

DOUBLE DESCENT AND STATISTICAL THEORY

Bui Gia Khanh & Luong Van Tam

Department of Physics

Hanoi University of Science, Vietnam National University

Hanoi, Vietnam

{fujimiyaamane}@outlook.com, tam0309cnvl@gmail.com

Dang Tri Trung

Monash University, Department of Computer Science

Melbourne, Australia

trungdc0708@gmail.com

ABSTRACT

The analysis of learning action, machine learning and related practices in the field theoretically has been made by utilizing Computational Learning Theory (CoLT) Valiant (1984) and Statistical Learning Theory (SLT) Vapnik (1999). These two theories, while overlapped, provides a general framework in analysing and justifying learning actions and learning model constructions, aiding in the formation of modern practices. One of the famous insight using such framework is the *bias-variance tradeoff* Geman et al. (1992), which states that model complexity and generality inversely affect each other, thus guarantee the need for a safe bracket between them. However, recent literatures, Belkin et al. (2019) has indicated the fallout of such dilemma by the new phenomenon called *double descent*, where there exists an interpolation threshold that renders the current statistical justification for bias-variance inconsequential to a given region. Various ‘anomaly’ has also been detected similarly, with varying degree of sophistication and potential. In this paper, we would like to review through progress made in this particular field of research, and the phenomena potential explanations up until now.

1 INTRODUCTION

The theory of machine learning and its modern practice has been developed and researched, of a substantial portion by empirical and heuristic approach, either by advancing new practices, architectures or by try-and-test modification. From its early onset of a regression estimator $\theta(x, y)$ on the linear regression problem, machine learning has developed substantially. Certain model concept with great successes includes regression-classification model, Bayesian modelling, generative learning model, support vector machine, Gaussian processes, and more. Of all such, the more formal and complex model architecture created, is the concept of a *neural network*. With increasingly sophisticated architecture, heuristic approach become popular, the fast-paced advancement of the field comes with new method, new results, new observations, and its far-reaching application which led to even bigger and larger scale deployment, there has been questions about the formation and status of a theoretical ground, a rigorous matter on the side of **theoretical machine learning**.

While rigorous and well-formulated in a sense, classical and theoretical machine learning was dwarfed by the modern advancement of machine learning as a whole, leading to several anecdotal problems regarding the interpretation of phenomena, the re-evaluation of the theory to fit the more updated analysis, and explanation to more sophisticatedly designed system. This and many more, plus as present, many of such advancements and improvements are heuristic, and the general theory and conceptual understanding remain limited, led to the choice to often opt for analogies and empirical workaround. Ultimately, this resulted in multiple ‘failures’ in explaining, interpreting, and predicting behaviours of new phenomena appeared in the modern landscape of machine learning. Thereby, we suggest some particular insights and analysis, and a fix within interpretation of the phenomena.

Our main focus is to review on the topic of the umbrella term classical learning theory, and the recent phenomena observed named ‘double descent’. Statistical learning theory (SLT) and computational learning theory (CLT) Vapnik (1999); Mohri et al. (2012); Shalev-Shwartz & Ben-David (2014); Hajek & Raginsky (2021); Bousquet et al. (2020) has been prominent in constructing a well-rounded formal theory surrounding learning problems, models, and machine learners analysis. Valuable insights have been dissected from

treatment of statistical theory and mathematical modelling on models, including the *bias-variance tradeoff* Geman et al. (1992); Domingos (2000c), which serves as a bound for efficient learning and model configuration (or complexity). However, recently there has been observations of *double descent* Belkin et al. (2019); Schaeffer et al. (2023); Nakkiran et al. (2019); Lafon & Thomas (2024) which refute the famous tradeoff assumption, and hence brings question to the establishment of the theory, as well as several assumptions and insight in the framework. Further events and phenomena observed also includes grokking and triple, to n -descent Davies et al. (2023); d' Ascoli et al. (2020). Furthermore, many problems of defining and formalizing notions used in designing and implementing machine learning models are inconclusive, such as, for example, *model complexity* and others.

2 OUTLINE

The main contributions of the paper lie in the heart of the problem of *machine learning theoretical*, and specifically *double descent*. As such, we focus on the following:

3 RELATED WORK

On itself, the phenomena *double descent* has been investigated somewhat comprehensively, firstly introduced by Belkin et al. (2019), which gives it distinctive saddle escape illustration. Further analysis was made by several literatures, particularly attributed the existence of double descent to the concept of model complexity and inductive bias, and potential explanations of such. Nakkiran et al. (2019) expanded the phenomena into deep neural network models. Their conclusion is reached by considering the perturbation of a learning procedure $\mathcal{T}$ on the effective model complexity $\text{EMC}_{\mathcal{D},\epsilon}(\mathcal{T})$ defined in the paper, separating the eventual phenomena into regions of observations, thereby in one way or another, predicting the tendency of double descent. This is also the first, arguably, "concise" definition of double descent, even though its nature is an empirical definition, including the notion of model complexity. Recently, there is also the Neural Tangent Kernel (Jacot et al. (2018)) which can be used to explain double descent, however further researches are still required. Preliminary works are done by Lafon & Thomas (2024), Schaeffer et al. (2023), and Liu & Flanigan (2023) on the role of optimization in double descent. Davies et al. (2023) attempted to unifying grokking with double descent, theorized a possibility of similarity during the generalization phase transition of the inference period, and Olmin & Lindsten (2024) attempted to explain epoch-wise double descent within the model of two-layer linear network. However, it is mentioned that it can also be expanded into deep nonlinear networks. Further literatures on double descent varies, but fairly new, ever since their discovery articles in Belkin et al. (2019) and Nakkiran et al. (2019). Since then, we have Shi et al. (2024); Schaeffer et al. (2023); Quéty & Tartaglione (2023a); Lafon & Thomas (2024); Neal (2019); Davies et al. (2023); Quéty & Tartaglione (2023b); Liu & Flanigan (2023); Mei & Montanari (2020; 2019a); Adlam & Pennington (2020); Transtrum et al. (2025); Liu et al. (2021); Allerbo (2025); Pezeshki et al. (2021); Brellmann et al. (2024); Nakkiran (2019b); Mei & Montanari (2019b); Heckel & Yilmaz (2020); Nakkiran et al. (2020); Belkin et al. (2020); Yang & Suzuki (2024); Spiess et al. (2023); Nakkiran et al. (2021); Zhang (2024); Cherkassky & Lee (2024); Nakkiran (2019a). An even more exotic version of double descent, dubbed *triple descent* (or more), is discovered in d' Ascoli et al. (2020).

4 PRELIMINARIES

This section introduces and provides usable preliminary that will be used throughout this text, which include learning theory, practical consideration, algorithmic guarantees, and so on.

4.1 CLASSICAL PROBLEM SETTING

Most of the paper will argue about different features of statistical learning theory (Sterkenburg (2024); Vapnik (1999); Hajek & Raginsky (2021)). It is then imperative to discuss the background setting of such. We are given the observations, or dataset of the form $\mathcal{S} = (\mathcal{X}, \mathcal{Y}) \subset \mathbb{R}^n \times \mathbb{R}^m$, of all 2-tuple pairs, assumed to be sampled or observed and governed by a distribution $\mathcal{D}$.

$$\mathcal{S} = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\} \subset \mathcal{X} \times \mathcal{Y}$$

The dataset is assumed to be i.i.d. sampling according to $\mathcal{D}$, which is unknown by the priori. Hence, we have the first set of assumption.

Assumption 4.1. *The observational space is governed by a particular sampling distribution $\mathcal{D}$, for fixed y -response according to the concept $c \in \mathcal{C}$. The sampling process is further assumed to hold no further structure, hence is identically and independently distributed (i.i.d.).*

The learning problem is then formulated as followed. Given the machine learning model expressed a hypothesis h of the hypothesis class $\mathcal{H}$, the learning theory aims for creating a procedure to learn either elements of the concept class $\mathcal{C}$ of all concepts $c : \mathcal{X} \rightarrow \mathcal{Y}$, or the function class $\mathcal{F}$ of all functions $f : \mathcal{X} \rightarrow \mathcal{Y}$, which is usually set to $\{0, 1\}$ or $[0, 1]$. This distinction is trivial, hence, if it is clear, we will talk about the concept class $\mathcal{C}$ only as representative form. The learner $\mathcal{L}(h)$ consider the set of possible hypothesis $\mathcal{H}$, in which might not coincide with $\mathcal{C}$. It receives a partial image of sample $S = (x_1, \dots, x_2, \dots, x_n)$ drawn i.i.d. according to $\mathcal{D}$ as well as the label $(c(x_1), \dots, c(x_n))$. This constitutes the dataset $\mathcal{S}$, which are based on specific concept $c \in \mathcal{C}$ for the hypothesis to learn. The task is then to use (or *extract*) meaningful information to select a hypothesis $h_S \in \mathcal{H}$ that accurately mimic c , with marginal error Θ . The notation h_S stands for all hypothesis that can be inferred from the range of the dataset (the first argument, $\mathcal{X}$).

The marginal error Θ is considered of two parameters, the empirical error $\hat{R}(h)$ and the generalization error $R(h)$. We give the following definition of them.

Definition 4.1 (Empirical risk). *Given a hypothesis $h \in \mathcal{H}$, a target concept $c \in \mathcal{C}$, and a sample $S = (x_1, \dots, x_m)$. For some particular $\epsilon > 0$, the **empirical error** or empirical risk of h is defined by*

$$\hat{R}_S(h) = \mathbb{P}_{x \in S \sim \mathcal{D}}[\ell\{h(x), c(x)\} \geq \epsilon] = \frac{1}{m} \sum_{i=1}^m \ell\{h(x_i), c(x_i)\} \quad (1)$$

Definition 4.2 (Generalization risk). *Given a hypothesis $h \in \mathcal{H}$, a target concept $c \in \mathcal{C}$, and an underlying distribution $\mathcal{D}$ on $\mathcal{X}$. For some particular $\epsilon > 0$, the **generalization error** or risk of h is defined by*

$$R(h) = \mathbb{P}_{x \sim \mathcal{D}}[\ell\{h(x), c(x)\} \geq \epsilon] = \mathbb{E}_{x \sim \mathcal{D}}[\ell\{h(x), c(x)\}] = \int_{x \in \mathcal{D}} \ell\{h(x), c(x)\} dP(x) \quad (2)$$

In essence, the empirical risk measure the hypothesis-to-observation error, and the generalization risk captures the difference of the hypothesis to the actual concept. Notice that by doing this, we have implicitly assumed that the observations and the concept is not the same thing, under the majority of scenarios. For the observations to be *approximately equal* or sufficiently captures the concept, there then would have to be various criteria to fulfil. This can be illustrated as Illustration 1.

For fixed $\mathcal{H}$, for fixed and sufficiently large $\mathcal{S}$, and no observation (data) errors, the empirical risk is the generalization risk. These two measures between h and c constitute the learning problem, which can also be separated into both cases - either empirical learning or generalization learning, one to minimize $\hat{R}(h)$, and one to minimize $R(h)$.

Definition 4.3 (Empirical learning problem). *We present the formal form of the empirical learning. Suppose we have a **target**, $c \in \mathcal{C}$, where $\mathcal{C}$ is an arbitrary concept class that captures targets of the same type. Suppose we are provided a set of observations $\mathcal{S}$. The problem is to use certain algorithm $\mathcal{A}$ using $\mathcal{S}$, to obtain a hypothesis h^* for a fixed $\mathcal{H}$ such that:*

$$R(h^*) = \min_{h \in \mathcal{H}} \hat{R}(h) = \min_{h \in \mathcal{H}} \mathbb{E}_{x \sim \mathcal{D}, x \in \mathcal{S}} \ell\{h(x), c(x)\} \quad (3)$$

The generalization best h is also called in literature as the *Bayes model*, such that it reduces the Bayes risk that is the infimum of the generalization risk, $R^* = \inf_{h \in \mathcal{H}} R(h)$. The hypothesis h^* is often called the *empirical best*, for it being the minimal, finite hypothesis of the lowest loss evaluation on the entire observation space $\mathcal{S}$. There exists no certified assumption regarding whether h^* aligns with the minimal generalization error.

Definition 4.4 (Generalization learning problem). *We present the formal form of the generalization learning problem. Suppose we have a **target**, $c \in \mathcal{C}$, where $\mathcal{C}$ is an arbitrary concept class that captures targets of the same type. Suppose we are provided a set of observations $\mathcal{S}$. Supposed we have an algorithm $\mathcal{A}$ that for fixed hypothesis space $\mathcal{H}$, equation 3 holds true. The problem is to use certain algorithm $\mathcal{A}'$ such that, under limited*

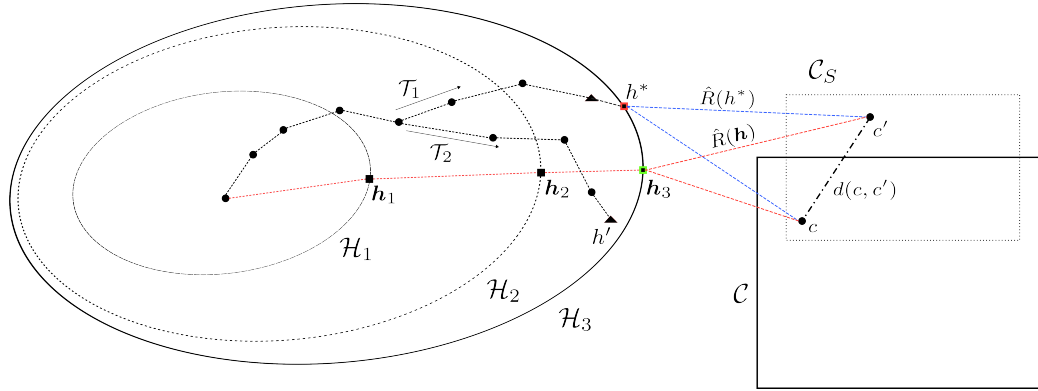


Figure 1: **Illustrative dynamic of the learning problem.** For an incremental hypothesis space sequence $\mathcal{H}_1, \mathcal{H}_2, \mathcal{H}_3$, we aim to obtain the procedure $\mathcal{T}$ that would either reach the *generalization solution* $\mathbf{h}$, or the *empirical solution* $\mathbf{h}^*$ for a particular hypothesis class, with respect to the concept c and its observed concept c' . Model selection hence dictates how an algorithm or procedure than choose the best possible hypothesis to approximate the generalization solution from the empirical solution set. Do notice that here we explicitly state that the data would create a **proxy concept** that overlaps with the concept class and of some arbitrary ‘distance’ from the true concept by $d(c, c')$. In case the two classes overlap, there still exists the arbitrary distance. Also, we can also observe the intuitive notion of increasing hypothesis class - while it indeed can help in getting closer to the concept class, the somewhat intrinsic property of current learning theory being the relative random procedure class make it probabilistically unstable.

availability, to obtain $\mathbf{h}$, satisfies:

$$R(\mathbf{h}) = \min_{\mathbf{h} \in \mathcal{H}} R(\mathbf{h}) \leq \{\epsilon\}, \quad \epsilon > 0 \quad (4)$$

For a set of risk bounds ϵ . If the setting is deterministic, then there exists $\epsilon = 0$ accordingly.

Then, we can partially say that generalization problem is an advancement from empirical solution. As such, empirical solution imposes the observed concept, and not the concept itself. This is reflected in modern machine learning landscape, by modelling the concept as distributions, and then using notions such as KL-divergence to measure such relative disparity.

Optimization problem 1: Empirical-generalization learning dynamics (non-proxy)

Data: $(x_i, y_i) \in \mathcal{S}, \Gamma \supset \mathcal{S}$, objective $\ell(y, f(x))$, $\nabla(\cdot)$ differentiable gradient field on ℓ .

Result: Expected optimized model to the minimal error on the objective $\ell(\cdot, \cdot)$, dependent on $\mathcal{S}$.

begin

$\Gamma \supset \mathcal{S}$, the general set;

$R(\cdot), \hat{R}(\cdot) \leftarrow \ell$ as objective functional;

for Fixed $\mathcal{H}$, with $\mathbf{h} \in \mathcal{H}$, **for** $\epsilon > 0$ **do**

 Find $\mathbf{h} \rightarrow \mathbf{h}'$, using algorithm $\mathcal{A}$ for minimization objective, such that

$$\mathcal{A} \left(\min_{\mathbf{h} \in \mathcal{H}} \hat{R}_{\mathcal{S}}(\mathbf{h}) + R(\mathbf{h}) \right) \leq \epsilon \quad (5)$$

 for procedure $\mathcal{A}(\cdot, \cdot)$ of structural constraints.;

end

end

return Optimized model $\mathbf{h}$ on arbitrary objective space (ℓ, ∇) .

These two stages of learning procedure comes of as the intrinsic design of machine learning setting. As opposed to interpolation, where empirical learning problem would have to be put into first priority to achieve the best possible point-through model, machine learning leans on approximation more and a separated criterion that is only visible in the learning setting - the fact that we are using c' to extract c . This disparity is exactly the **generalization problem**, of which is then also the goal of machine learning, to approximate the general solution using the observed solution. One then can ask what is the difference between the two notions, and

when we can guarantee that both will ‘converge’ to the same point. The process of **model selection** is then the procedure that will optimize $h \in \mathcal{H}$ to a given target fixture, such as the empirical best.

One of the major assumption or observation from the point of the hypothesis class, is that the hypothesis cannot know the generalization error. Thereby, the more efficient and often used procedure/algorithm in training, is then the *empirical risk minimization* (ERM) method, minimizing only the empirical risk to the empirical best; then, certain measures to ‘generalize’ this heuristically is apprehended on the hypothesis. The process of model selection is also the place that we get the concept of **bias-variance tradeoff**.

4.1.1 UNIFORM CONVERGENCE

Suppose that for a concept class $\mathcal{H}$, we can then partition it to $\mathcal{H} = \bigcup_{k \in \mathbb{N}} \mathcal{H}_k$ for finite k hypotheses. Assume that they satisfy the uniform convergence (Shalev-Shwartz et al. (2010)):

$$\sup_{\mathcal{D}} \mathbb{E}_{S \sim \mathcal{D}^m} \left[\sup_{h \in \mathcal{H}} [R(h) - R_S(h)] \right] \xrightarrow{m \rightarrow \infty} 0 \quad (6)$$

This uniform convergence holds for a learning problem, if the empirical risks of hypotheses in the hypothesis class converges to their population risk uniformly, with a distribution-independent rate. The result of this is that a problem then can be considered learnable with the ERM rule, for any given characterization. We state the following definition.¹

Definition 4.5. A learning problem is learnable if there exist a learning rule $\mathcal{A}$ and a monotonically decreasing sequence $\epsilon_{cons}(m)$ such that $\epsilon_{cons}(m) \xrightarrow{m \rightarrow \infty} 0$ and

$$\forall \mathcal{D}, \quad \mathbb{E}_{S \sim \mathcal{D}^m} [F(\mathcal{A}(S)) - F^*] \leq \epsilon_{cons}(m), \quad F^* = \inf_{h \in \mathcal{H}} R(h) \quad (7)$$

A learning rule $\mathcal{A}$ for which this hold is denoted as a *universally consistent learning rule*.

There are certainly a fundamental class of rule or algorithm $\mathcal{A}$ to resolve this particular uniform convergence of classes. One of the major assumption or observation from the point of the hypothesis class, is that the hypothesis cannot know the generalization error. Thereby, one strategy is the *empirical risk minimization* (ERM) method, minimizing only the empirical risk to the empirical best; then, certain measures to ‘generalize’ this heuristically is apprehended on the hypothesis.

Definition 4.6. A rule $\mathcal{A}$ is an *ERM (Empirical Risk Minimizer)*, denoted $\mathcal{A}_{ERM}$ if it minimizes the empirical risk $\hat{R}(h) = (1/m) \sum_{c \in \mathcal{C}} \ell(h, c)$, over the observational space S ,

$$\hat{R}_S(\mathcal{A}_{ERM}(S)) = \hat{R}_S(h_S) = \inf_{h \in \mathcal{H}} \hat{R}_S(h) \quad (8)$$

Hence, the ERM tries to obtain $\text{ERM}(h') = \arg \min_{h \in \mathcal{H}} \hat{R}(h)$.

The problem of overfitting still somewhat resides in the setting of the method. Hence, usually, we have an implicit term in addition, called the *regularizer*, of which is as followed.

Definition 4.7. A rule $\mathcal{A}$ is an *SRM (Structural Risk Minimizer)*, denoted $\mathcal{A}_{SRM}$ if it minimizes the empirical risk $\hat{R}(h) = (1/m) \sum_{c \in \mathcal{C}} \ell(h, c)$, over the observational space S ,

$$\hat{R}_S(\mathcal{A}_{SRM}(S)) = \hat{R}_S(h_S) + \lambda r(h) = \inf_{h \in \mathcal{H}} \hat{R}_S(h) + \lambda r(h) \quad (9)$$

of the regularizing term $r(\cdot)$. Hence, the SRM tries to obtain $\text{ERM}(h') = \arg \min_{h \in \mathcal{H}} \hat{R}(h)$, while taking into consideration a forcing term r , usually defined on the model complexity.

In this paper, we would like to presume the ERM method in question as the famous *gradient descent*. We refer to Achlioptas; Ruder (2017) for general view of the algorithm, and Zhang (2019) for a source on particularly gradient descent algorithm on deep learning models.

¹The reason why $r(\cdot)$ is not characterized in this preliminary section, is because of their uncertainty of effect. Hypothetically, $r(\cdot)$ is an additional structure (bias) of which apply a new gradient field in conjunction to the loss gradient field itself. This addition is uncertain of its content and analysis, since it depends on various stochastic dynamic of the model. We reserve from using regularization for now because of such reason.

4.2 OVERFITTING AND UNDERFITTING

Following the above framework of statistical probabilistic learning theory, one understanding about the concept of underfitting and overfitting (similar to James et al. (2013)) can be reached, using the notion of *partitioning* of observation space. Such work is typically done because of **practical condition**, in which the following constraints is applied.

Assumption 4.2. *No knowledge of $c \in \mathcal{C}$ is provided.*^a

^aFew points need to be said about this assumption. This assumption represents the *worst-case possible* of a system dynamic. From the perspective of a designer, inherent leakage of information can be provided in the design of a model. In such, case we are operating in the mode of a *resource-constrained, closed* game, in which the model with further assumptions of fixed class, operates on. The problem space itself is inherently large and penalize any additional structural changes. If one is to occur, we usually mention it as *implicit bias*, but another name that would be more fitting would be *structural attractor*, after the notion of gradient attractor in chaotic systems.

Without such knowledge, $R(h)$ cannot be computed, accordingly. We specify such approach as *heuristic*, as it is only an approximation, and for better definition, we define it as followed.

Definition 4.8 (Heuristic empirical risk). *Given a dataset S of size m , supposedly of the concept $c \in \mathcal{C}$, and the hypothesis $h \in \mathcal{H}$, then the **heuristic empirical risk** take a partitioning $k < m$ with $k/m \geq 1/2$, hence defining the heuristic empirical loss on k , denoted $\hat{R}_k(h)$.*

Definition 4.9 (Heuristical generalization risk). *Given a dataset S of size m , supposedly of the concept $c \in \mathcal{C}$, and the hypothesis $h \in \mathcal{H}$. Fixed $\hat{R}_k(h)$, for the final optimization of algorithm $\mathcal{A}$, then the **heuristic generalization risk** take the remaining 2-partition of portion $p = k - m$, hence defining the heuristic generalization loss on p , denoted $\hat{R}_p(h_{\mathcal{A}})$.*

The two heuristic definition however, can be noted to not be equal to the true generalization risk - empirical risk pair in reality. Nevertheless, one can then define overfitting and underfitting in such manner.

Definition 4.10 (Underfitting). *A model h is called **underfitting** if for a constant $\epsilon > 0$ chosen for the setting, then $\hat{R}_k(h_{\mathcal{A}}) > \epsilon$, $\hat{R}_p(h_{\alpha}) > \epsilon$ over the entire observation space.*

Usually, underfitting is defined to be such that test error is smaller than training error partition, however, we do not think that is a good definition. Especially since, at least in this setting, the empirical error under algorithm $\mathcal{A}$ is reduced to the representative smallest error that $\mathcal{A}$ can achieve on that particular model, within particular setting of h , then come the heuristic generalization error. Considering also that $k/m \geq 1/2$, that is, at least the training partition is half of the observation set, then only under very specific case will $\hat{R}_p(h_{\alpha})$ be smaller than $\hat{R}_k(h)$, statistically speaking. Thence this line of thought also reinforces the idea that both of them will be larger than the threshold ϵ . Usually, heuristically, this threshold of error is 0.3, normalized. Next, we define the notion of overfitting.

Definition 4.11 (Overfitting). *A model h is called **overfitting** if there exists a specific constant $\epsilon > 0$ chosen for the setting, such that $\hat{R}_k(h) < \epsilon$, $\hat{R}_p(h_{\alpha}) > \epsilon$ over the entire observation space.*

Namely, this mean that the static learning makes the generalization error worse. This draws the analogy of the student - studying explicitly for the past paper tests, yet unaware of the larger margin of questions in the large set of the test paper pool, leading to subpar performance. Similarly, overfitting is theorized to be fitting too fit to the reduced observation space - hence when it comes to the generalization observation space, it fails within such heuristic. The notion works on the mask of the risks, $\hat{R}_k$ and $\hat{R}_p(h_{\alpha})$.

4.2.1 PRACTICAL PROXY

In practice, Assumption 4.2 means we cannot have $c \in \mathcal{C}$, or its probabilistic interpretation $\mathcal{D}$ such that $x \sim \mathcal{D}$, or in the extreme non-structured case, $(x, y) \sim \mathcal{D}$. In such case, the typical practical fix relies on the partitioning of the available partitioning space, that is, the empirical-generalization proxy. For large enough singleton count of $|\{(X_k, Y_k)\}|$, we assume that $\hat{R}_{\text{Gen}}(h) \approx R(h)$. Thus, the algorithm can be stated in a form as followed.

Optimization problem 2: Empirical-generalization learning dynamics (proxy)**Data:** $(x_i, y_i) \in \mathcal{S}, y_i = \{\pm 1\}$, hinge objective $\ell(y, f(x))$, $\nabla(\cdot)$ differentiable gradient field on ℓ .**Result:** Expected optimized model to the minimal error on the objective $\ell(\cdot, \cdot)$, dependent on $\mathcal{S}$.**begin** $((X_1, Y_1), \dots)_n \leftarrow \text{Partition}_n(\mathcal{S});$ $\{(X_j, Y_j)\} \leftarrow \text{Train};$ $\{(X_k, Y_k)\} \leftarrow \text{Gen for } k \neq j;$ **for** Fixed $\mathcal{H}$, with $h \in \mathcal{H}$, for $\epsilon > 0$ **do**(Optimistically) achieve $h \rightarrow h'$, using algorithm $\mathcal{A}$ for minimization objective, such that

$$\mathcal{A} \left(\min_{h \in \mathcal{H}} \hat{R}_{\text{Train}}(h) + \hat{R}_{\text{Gen}}(h) \right) \leq \epsilon \quad (10)$$

for procedure $\mathcal{A}(\cdot, \cdot)$ of structural constraints.;**end****end****return** Optimized model $h_{\mathcal{A}}$ on arbitrary objective space (ℓ, ∇) .

The assumption above of the generalization set to the generalization truth error is apparent in the usual system theorem, Theorem 4.3.

Theorem 4.3 (Mohri et al. (2012)). *For fixed $\mathcal{H}$, for fixed and sufficiently large $\mathcal{S}$, and no observation errors, the empirical risk is the generalization risk:*

$$\mathbb{E}_{S \sim \mathcal{D}^m} [\hat{R}_S(h)] = R(h) \quad (11)$$

Proof. By linearity of the expectation, and the fact that we assume the sample is given i.i.d., we can write:

$$\mathbb{E}_{S \sim \mathcal{D}^m} [\hat{R}_S(h)] = \frac{1}{m} \sum_{i=1}^m \mathbb{E}_{S \sim \mathcal{D}^m} [\mathbb{1}_{h(x_i) \neq c(x_i)}] = \frac{1}{m} \sum_{i=1}^m \mathbb{E}_{S \sim \mathcal{D}^m} [\mathbb{1}_{h(x) \neq c(x)}],$$

for any x in sample S . Thus,

$$\mathbb{E}_{S \sim \mathcal{D}^m} [\hat{R}_S(h)] = \mathbb{E}_{S \sim \mathcal{D}^m} [\mathbb{1}_{h(x) \neq c(x)}] = \mathbb{E}_{x \sim \mathcal{D}} [\mathbb{1}_{h(x) \neq c(x)}] = R(h).$$

which yields the desired result. $\square$

The above procedure still attains a lot of assumptions, of which we settle for the next section. Optimization procedure 1 and procedure 2 define the precursor tradeoff in the problem space, or rather, the heuristic-generalization tradeoff. We can state it as the following.

Theorem 4.4 (Heuristic-generalization tradeoff, model selection). *An algorithm $\mathcal{A}$ on the system $(\mathcal{H}, \mathcal{C}, \mathcal{A})$, for the expected empirical heuristic $\hat{R}_{\text{Train}}(h)$ and generalization heuristic $\hat{R}_{\text{Gen}}(h)$ on $\mathcal{S}$, on the measure landscape of $\mathcal{L}[\ell]$ for $\hat{R}(\cdot)$, of which $\mathcal{A}$ runs separately on $\mathcal{A}'(\text{Train}, h)$ and $\mathcal{A}'(\text{Gen}, h)$ for the above heuristic, there exists*

$$\left\{ \arg \min_{h \in \mathcal{H}} \hat{R}_{\text{Train}}(h), \arg \min_{h \in \mathcal{H}} \hat{R}_{\text{Gen}}(h) \right\} \in \mathcal{L}[\ell] \quad (12)$$

as **convex attractor** on the measure landscape. Then, algorithm $\mathcal{A}$ solves the generalization if for the transition $\mathcal{T} : \mathcal{L}_{\text{Train}}[\ell] \rightarrow \mathcal{L}_{\text{Gen}}[\ell]$ for separated loss-landscape, it satisfies for $\mathcal{A}(S, h) \rightarrow h'$, that

$$h' = \arg \min_{h' \in \mathcal{H}} \left(\hat{R}_{\text{Train}}(h) + \hat{R}_{\text{Gen}}(h) \right) \quad (13)$$

This tradeoff relies on the precursor concept of the **attractor** landscape, of which is observable within the objective-space expansion from $\mathcal{L}_{\text{Train}}[\ell]$ to $\mathcal{L}_{\text{Gen}}[\ell]$ of the heuristic space. The partition $\{(X_j, Y_j)\}$ and $\{(X_k, Y_k)\}$ covers the practical notion of cross-validation, though the cross-validating procedure itself works much different, as it covers multiple re-shuffling, i.e. unordered re-partition of the default permutating groups.

5 PROBLEM STATEMENTS

Before continuing further, we must review the problem statement that led to the problem of double descent. We will have a look at the theory, the original conflict, and the illustrative phenomenon in action. In general, the learning setting in consideration of statistical learning theory is often supervised learning, for guarantee of a plausible upper bound result.

5.1 BIAS-VARIANCE TRADEOFF

In classical sources, James et al. (2013); Goodfellow et al. (2016); Hajek & Raginsky (2021); Mohri et al. (2012); Shalev-Shwartz & Ben-David (2014), bias-variance tradeoff is a classical principle used in the categorization of technique called *model selection*. Specifically, this refers to the progress of choosing the best predictor h^* that minimize the generalization error that is the main goal of a particular machine learning model, during the process of training and inference. Originally, the theory that bias and variance often come in to conjunction is **Estimation theory**, of which there exists an estimator $\hat{\theta}$ that use a set of observations, to estimate the concept, or the underlying mechanics of certain system that outputted the observation set, by the **probabilistic perspective** - that is, it is governed by appearance by an independently and identically distributed process of parameterized probability $\theta \in \Theta$. Here, parameterized means being expressed by a set, often finite, of parameters, by E. L. Lehmann (1998); Paninski (2005). It is only when Geman et al. (1992) introduced it to machine learning that we have an equivalent structure that is similar to bias-variance in machine learning.

5.1.1 PRECURSOR (GEMAN ET AL., 1992)

The original definition of bias-variance tradeoff by Geman et al. (1992) is first constructed using the mean-square error, which is regarded as a normal measure in the real encoding space. Their approach is to justify bias-variance via decomposition of the loss function ℓ , for such to find an alternative reasonable form of such loss landscape. Suppose of a regression problem to construct a hypothesis function $f(x)$ from $(x_1, y_1, \dots, x_N, y_N)$ for the purpose of generalization - that is, predicting unseen variational values for different pair $(x_j, ?)$ such that $? = y_j + \epsilon$ for a conceivable implicit error. To be explicit about the relation of this problem, or f on the given data $\mathcal{D} = \{(x_i, y_i) \mid i \leq N\}$, denote $f(x; \mathcal{D})$ instead of f , the natural mean-square measure as a predictor is:

$$\mathcal{M}(f, y) = \mathbb{E} [((y - f(x; \mathcal{D})))^2 \mid x, \mathcal{D}] \quad (14)$$

for $\mathbb{E}[\cdot]$ the expectation wrt to a distribution P . Decomposing the right-hand side, we have:

$$\mathcal{M}(f, y) = \mathbb{E} [((y - f(x; \mathcal{D})))^2 \mid x, \mathcal{D}] = \mathbb{E} [(y - \mathbb{E}[y \mid x])^2 \mid x, \mathcal{D}] + (f(x; \mathcal{D}) - \mathbb{E}[y \mid x])^2 \quad (15)$$

Here, $\mathbb{E} [(y - \mathbb{E}[y \mid x])^2 \mid x, \mathcal{D}]$ does not depend on $\mathcal{D}$, but simply the statistical variance of y given x . The term $(f(x; \mathcal{D}) - \mathbb{E}[y \mid x])^2$ is considered a natural measure of effectiveness on $\mathbb{R}^n$ as a singular predictor of y . Now, for $\mathbb{E}_{\mathcal{D}} [(f(x; \mathcal{D}) - \mathbb{E}[y \mid x])^2]$ which depends on the training set $\mathcal{D}$ in its computation, is decomposed into the form of *bias-variance decomposition* terms, by derivation:

$$\mathbb{E}_{\mathcal{D}} [(f(x; \mathcal{D}) - \mathbb{E}[y \mid x])^2] = \underbrace{\{\mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D})] - \mathbb{E}[y \mid x]\}^2}_{\text{bias term}} + \underbrace{\mathbb{E}_{\mathcal{D}} \{(f(x; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D})])^2\}}_{\text{variance term}} \quad (16)$$

We summarize this in the following statement.

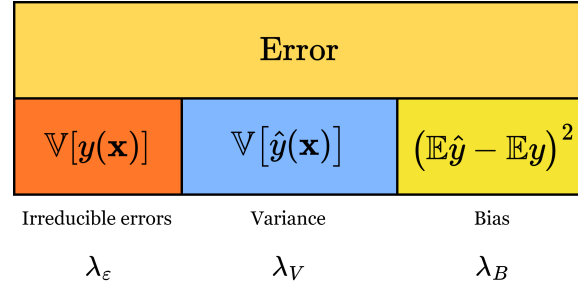
Theorem 5.1 (Bias-variance decomposition). *Suppose the model $f(x; \mathcal{D})$ for the data $\mathcal{D} = (x_i, y_i)$ and its parameter x is defined. For y_i of the target concept's responses y , and consider a regression problem with the loss measure $\mathcal{M}(f, y)$ of mean squared risk, the following statement is true:*

$$\mathbb{E}[\mathcal{M}(f, y)] = \mathcal{B}(f, y) + \mathcal{V}(f, y) + \mathbb{E}[\mathbb{E} [((y - f(x; \mathcal{D})))^2 \mid x, \mathcal{D}]] \quad (17)$$

for $\mathbb{E}[\cdot \mid x, \mathcal{D}]$ any expression with dependencies on x and $\mathcal{D}$. The bias and variance term is subsequently expressed by

$$\mathcal{B}(f, y) = \underbrace{\{\mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D})] - \mathbb{E}[y \mid x]\}^2}_{\text{bias}}, \quad \mathcal{V}(f, y) = \underbrace{\mathbb{E}_{\mathcal{D}} \{(f(x; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D})])^2\}}_{\text{variance}} \quad (18)$$

Figure 2: **Decomposition of the error term into 3 parts.** Respectively, the irreducible error $\mathbb{V}[y(\mathbf{x})]$, the variance (blue) and the bias (dark yellow). All of them are assumed to take up 100% of the error observed in such composition. The proportion is included using the three coefficient $\lambda_\epsilon, \lambda_V, \lambda_B$ where $\lambda_\epsilon + \lambda_V + \lambda_B = 1$.



The above decomposition principle is often expressed into a form where there exists the intrinsic noise Brown & Ali (2024):

$$\mathbb{E}_D \left[\mathbb{E}_{xy} \left(y - \hat{f}(x) \right)^2 \right] = \mathbb{E}_x \left[\left(y^* - \mathbb{E}_D[\hat{f}(x)] \right)^2 \right] + \mathbb{E}_x \left[\mathbb{E}_D \left(\hat{f}(x) - \mathbb{E}_D[\hat{f}(x)] \right)^2 \right] + \mathbb{E}_{xy} \left[(y - y^*)^2 \right] \quad (19)$$

In all, almost all representation of bias-variance decomposition are the same. For simplicity of notation, we adopt the similar form in the standard case of the paper Adlam & Pennington (2020):

$$\mathbb{E} [\hat{y}(\mathbf{x}) - y(\mathbf{x})]^2 = (\mathbb{E}\hat{y}(\mathbf{x}) - \mathbb{E}y(\mathbf{x}))^2 + \mathbb{V} [\hat{y}(\mathbf{x})] + \mathbb{V}[y(\mathbf{x})] \quad (20)$$

A main common theme of criticism toward bias-variance tradeoff is the fact that the decomposition is much more general, and intrinsic for the class of *mean squared loss*. However, when considering the naturalness of an error measure and then its direct applicant, mean square error on real space of sufficient support and measure comes of as a very natural choice of an error measure, at least in consideration of the regression setting; for that, we then can also define classification as another top-layer above a regression's similar real continuous space, i.e. a decision layer. Furthermore, it can also be shown Brown & Ali (2024); Pfau (2013) that it also holds for the class of Bregman divergence measure.

What does the decomposition say about model selection principle in general? In general, bias-variance is typically presented of its decomposition only, and the asymptotic prediction of its behaviours. In fact, one of the reason that it became the rule-of-thumb for ML practitioner, as well as generally statistical learning (Lafon & Thomas (2024) provides a quite rigorous treatment of bias-variance tradeoff in the section on statistical learning theory) solidify the trade-off as a particular model selection principle. Generally, this tradeoff can be summarized as followed:

Theorem 5.2 (Bias-variance tradeoff). *For the expected loss of any given hypothesis h , the bias $\mathcal{B}(f, y)$ and variance $\mathcal{V}(f, y)$ is inversely proportional, that is, $\mathcal{B}(f, y) \propto \lambda^{-1} \mathcal{V}(f, y)$ for some proportionality λ that may or may not be constant. In the most general case possible, $\lambda = -1$ on the entire error range.*

The tradeoff is then of inverse proportionality. Indeed, statistically, we have such tradeoff on a statistical framework in a more concrete sense. For the bias to increase, variance will increase, of which the criterion is inverse - we would like to have more bias but lower variance, and vice versa, according to such theory.

5.1.2 EXPANSION AND VARIATIONS

The bias-variance decomposition and tradeoff is not general. Indeed, works, by Geman et al. (1992); Sharma & Aiken (2014); Domingos (2000d); Adlam & Pennington (2020); Yang et al. (2020) and more all attempted to reconstruct and reformulate bias-variance, and it did leave out a picture of uncertainty regarding the concept. Specifically, there are certainly different way to do the bias-variance decomposition, as classified in Adlam & Pennington (2020) paper. Some of this are particularly specialized in *neural network* training, basing on the main structure of modern machine learning in neural network formalism.

1. **Semiclassical approach:** The semiclassical approach is fairly simple. We can rewrite to another variation of the classical approach to be

$$\mathbb{E}[\ell(h)] = \mathbb{E}_x \mathbb{E}_\epsilon [\hat{y}(x) - y(x)]^2 = \mathbb{E}_x (\mathbb{E}_\epsilon [\hat{y}(x)] - y(x))^2 + \mathbb{E}_x \mathbb{V}_\epsilon [\hat{y}(x)] \quad (21)$$

and instead, consider the *structural conformation* of the decomposition, or rather, taking the expectation within $\theta \in \Theta$ of the structural description of $h \in \mathcal{H}$. Then, we shall have

$$\mathbb{E}_{\text{SC}}[\ell(h)] = \mathbb{E}_x \mathbb{E}_\theta \mathbb{E}_\epsilon [\hat{y}(x) - y(x)]^2 = \mathbb{E}_x \mathbb{E}_\theta (\mathbb{E}_\epsilon [\hat{y}(x)] - y(x))^2 + \mathbb{E}_x \mathbb{E}_\theta \mathbb{V}_\epsilon [\hat{y}(x)] \quad (22)$$

Do note that the order matters, since the expectation can be constant of a given ordering. And expected value over constant is just the constant.

2. **Symmetric decomposition:** This approach is fairly difficult, we refer to the paper itself of its own justification on the decomposition. The symmetric decomposition relies on the shape of the input, $\mathcal{X} = \{P, D, \epsilon\}$, of which you can remove the ϵ term and retain some decomposition on P and D per distribution.

5.1.3 GENERALIZED DECOMPOSITION

According to Domingos (2000b;a), the bias-variance decomposition can usually be treated as being controlled by three factor error, with their respective coefficients $\lambda = (\lambda_1, \lambda_2, \lambda_3)$ for such:

$$\begin{aligned} \mathbb{E}_{\mathcal{D}} [d((f, \mathbb{E}[y | x]))] &= \lambda_1 \text{Bias}(f, y) + \lambda_2 \text{Var}(f, y) + \lambda_3 \epsilon(\mathcal{D}) \\ &= \lambda_1 \underbrace{\{\mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D})], \mathbb{E}[y | x]\}}_{\text{bias term}} + \lambda_2 \underbrace{\mathbb{E}_{\mathcal{D}} \{(f(x; \mathcal{D}), \mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D})])\}}_{\text{variance term}} + \underbrace{\lambda_3 \epsilon}_{\text{irreducible error}} \end{aligned}$$

where $\epsilon(\mathcal{D})$ is the irreducible error of the system, depends (theoretically) on the intrinsic imperfection of the dataset $\mathcal{D}$. This method of which the bias-variance-irreducible error are ‘fit’ into the total error by three coefficients $\lambda_1, \lambda_2, \lambda_3$ instead. This does not only scale up the B-V-IE triplet, but also leaves out the inexplainable gap between the scaling, and the actual normal measure.

5.1.4 BIAS-VARIANCE ALGORITHM

5.1.5 FUNCTIONAL APPROXIMATION ALGORITHM

Usually, the optimization problem is evaluated empirically of its bias-variance metric. That is, we test singular models successively, and then choose the generally best — usually within bias-variance tradeoff. This can be expressed as a function for theorem 5.2.

5.1.6 APPROXIMATION-ESTIMATION TRADEOFF

Another closely related notion to bias-variance is the concept of approximation-estimation tradeoff. We refer to Mohri et al. (2012); Lafon & Thomas (2024) for some analysis and mentions.

Let $\mathcal{H}$ be a family of functions mapping $\mathcal{X} \rightarrow \{1, -1\}$. This is the particular case of **binary classification**, in which can be straightforwardly extended to different tasks and loss functions. The *excess error* of a hypothesis $h \in \mathcal{H}$, is the difference between its error $R(h)$ and the Bayes error R^* . This can be decomposed to be the following:

$$R(h) - R^* = \left(R(h) - \inf_{h \in \mathcal{H}} R(h) \right) + \left(\inf_{h \in \mathcal{H}} R(h) - R^* \right) \quad (23)$$

The first bracket contains the **estimation error**, and the second bracket contains what is called the **approximation error**. The estimation error depends on the hypothesis h selected. It measures the error of h with respect to the infimum of the error achieved by hypotheses in $\mathcal{H}$, or that of the best-in-class hypothesis h^* when that infimum is reached. The approximation error measures how well the Bayes error can be approximated using $\mathcal{H}$. It is a property of the hypothesis set $\mathcal{H}$, a measure of its richness.

Model selection consists of choosing $\mathcal{H}$ with a favourable trade-off between the approximation and the estimation error. However, in the most general case, this will be done, but not in practice, as it requires the underlying distribution $\mathcal{D}$ to be known to determine R^* , which is not possible. In contrast, the estimation error can be bounded, or can be analysed, using particular metric and analysis. It is however, worth to note that bias-variance and approximation-estimation have a very complicated relationship, for example, in Brown & Ali (2024) shown that they are indeed not the same, and is in fact two different decomposition, where one is the others’ component.

5.2 DOUBLE DESCENT

Nevertheless, of bias-variance and the analogous statistical learning theory concept, the target is the same. It is the dilemma of which is presented in **Occam’s razor**, for choosing the sufficient model of good complexity, or bias, for tradeoff of its generalization ability, or variance. Then there must exist a sweet spot between the axis of bias and variance, since they are as exhibited above inversely proportional to each other. However, double

descent seemingly broke the status quo, and insists on an interesting phenomenon - under the same setting, if we ‘crank’ the complexity high enough, we will then reach a point then called the **interpolation threshold**, such that the trend reverse and the error rate, instead of being theorized to go up, goes down to a certain line of lower bound.

The first identification of the double descent phenomena dated back to the paper of Belkin - Belkin et al. (2019), in which the title is literally “reconciling” modern machine learning practice and the bias-variance tradeoff. In modern machine learning practice, or state-of-the-art developments, models are now bigger than ever. If to notice, we will see that currently models are inherently large, for example, a normal large language model will have from 900 millions (900M) to a few billions, for example 10 billions (10B) parameters. That is not taking into account the overall dynamics and structure of the model, which dictates the operating range and efficiency of the model itself. These model, based on the neural network architecture are somewhat trained to exactly fit (or interpolate) the data, almost certainly so that it turn from a prediction setting to an estimation setting. By statistical learning theory, this would be considered overfitting, and yet, they often obtain very high accuracy on test data.

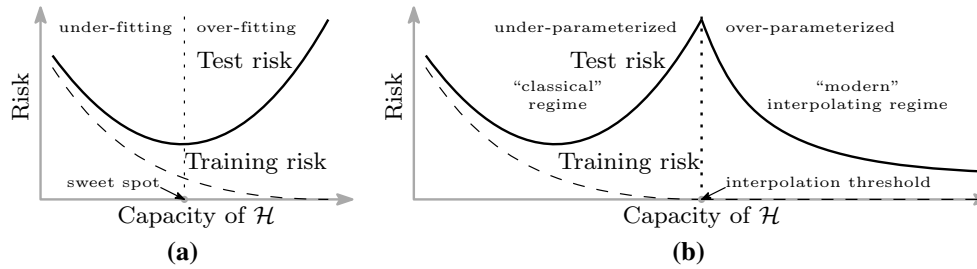


Figure 3: **Curves for training risk (dashed line) and test risk (solid line).** (a) The classical *U-shaped risk curve* arising from the bias-variance trade-off. (b) The *double descent risk curve*, which incorporates the U-shaped risk curve (i.e., the “classical” regime) together with the observed behaviour from using high capacity function classes (i.e., the “modern” interpolating regime), separated by the interpolation threshold. The predictors to the right of the interpolation threshold have zero training risk. Reproduced from Belkin et al. (2019).

The main finding that Belkin found is a pattern for how the apparent performance on unseen data depends on model capacity and the mechanism underlying the emergence of double descent. When function class capacity is below the “interpolation threshold”, learned predictors exhibit the classical *U-shaped curve* from Figure 3. The ‘modern’ interpolating regime marks the opposite trend to the right, where the risk starts to decrease up to a lower bound, which then can be called the *optimal descent bound*.

The bottom of the *U* is achieved at the sweet spot which balances the fit to the training data and the susceptibility to over-fitting: to the left of the sweet spot, predictors are under-fit, and immediately to the right, predictors are over-fit. When we increase the function class capacity high enough (e.g., by increasing the number of features or the size of the neural network architecture), the learned predictors achieve (near) perfect fits to the training data—i.e., interpolation. Although the learned predictors obtained at the interpolation threshold typically have high risk, we show that increasing the function class capacity beyond this point leads to decreasing risk, typically going below the risk achieved at the sweet spot in the “classical” regime. (Belkin et al. (2019))

At this point, we would like to also clarify the term interpolation threshold in this case. Usually, under the pretext of overfitting and underfitting, interpolation threshold means the following:

Definition 5.1 (Interpolation threshold). *Assuming a typical supervised learning setting, for a hypothesis h within class structure $\mathcal{H}$, and a supposed concept c of the class $\mathcal{C}$. Suppose $[X, Y] \subset c \in \mathcal{C}$ being the dataset, or observable space of c on which h is tested on. Under the **empirical generalization learning** procedure, partitioning $[X, Y]$ into $[X, Y]_1$ and $[X, Y]_2$, for error evaluated on the first set, $\hat{R}_1(h)$ called training error; and the second set, $\hat{R}_2(h)$ called test error; with optimization applied on the first set, then evaluated statically on the second, the **interpolation threshold** with respect to a particular complexity measure of h , $\mathcal{M}(h)$, is the point $\mathcal{M}_0(h)$ where it is possible for $\hat{R}_2(h)/\hat{R}_1(h) \approx 1$, for $\hat{R}_1(h) \ll \epsilon$, $\hat{R}_2(h) \ll \epsilon$, with $\epsilon \geq 0$.*

This basically means that what is learnt, with respect to the model complexity that gauge *how much* and *what* the model can learn, perfectly fits the concept c itself, from the observation space on its own. We can say that, that specific sense, we gain the assumption that the observation space is *fully representative* of the concept class, and the concept class is then fully realized by the model. Such point where that is possible, or there then such behaviour is possible, is called the interpolation point. The definition relies on the statement ‘where it is possible’, hence it still, does not give us a particularly sound notion on to how and what constitute such possibility. The term empirical generalization learning comes from the fact that the procedure in practice of estimating and optimizing train-test-validation set in itself, is an imperfect empirical approximation of generalization testing. While the fact that we cannot gauge generalization error, in a purely blind black-box setting is settled (Mohri et al. (2012); Sugiyama (2015); Shalev-Shwartz & Ben-David (2014)), the method is then to try to mimic such by using generalization-via-hidden observations. To facilitate this, deliberately, the empirical learning procedure is only dynamic during training session, and static throughout the whole testing and validation testing section, and further on.

By default, in general theory, such interpolating point is dangerous. Not only because it is inherently difficult to know exactly what the dataset constitute, whether it is actually representative of the concept that gives rise to it, or the structural information, noises, and interferences in between. It is hence impossible, to a certain point, to truly know the effectiveness of the model beside interpolating the current dataset which forms the observable details.

Another prominent discovery to look at is Nakkiran et al. (2019), on the double descent of deep learning models. The paper serves as the first observation upon which double descent is observed on modern, deep learning theoretic models, especially complex one like Transformer or ResNet networks.

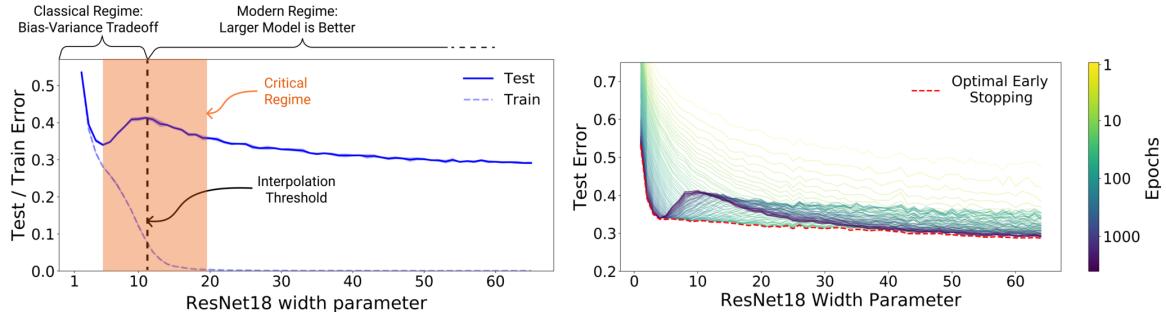


Figure 4: **Left:** Train and test error as a function of model size, for ResNet18s of varying width on CIFAR-10 with 15% label noise. **Right:** Test error, shown for varying train epochs. All models trained using Adam for 4K epochs. The largest model (width 64) corresponds to standard ResNet18. Resued from Nakkiran et al. (2019).

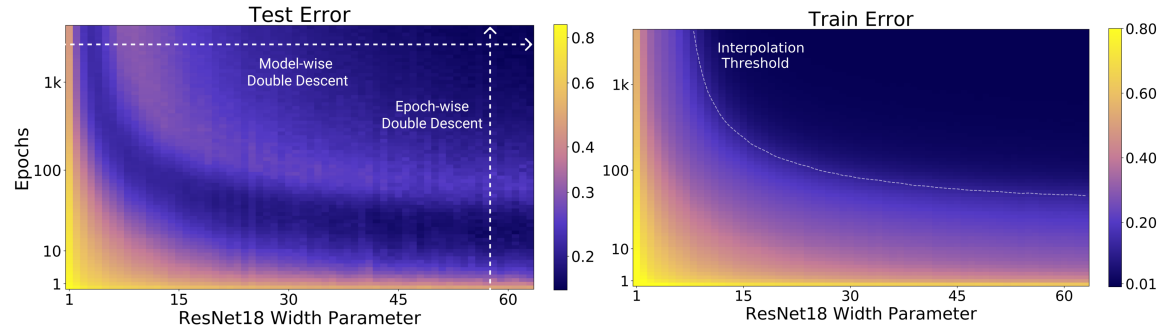


Figure 5: **Left:** Test error as a function of model size and train epochs. The horizontal line corresponds to model-wise double descent—varying model size while training for as long as possible. The vertical line corresponds to epoch-wise double descent, with test error undergoing double-descent as train time increases. **Right** Train error of the corresponding models. All models are Resnet18s trained on CIFAR-10 with 15% label noise, data-augmentation, and Adam for up to 4K epochs. Reused from Nakkiran et al. (2019)

They define *effective model complexity* of $\mathcal{T}$ (w.r.t. distribution $\mathcal{D}$) to be the maximum number of samples n on which $\mathcal{T}$ achieves on average ≈ 0 training error. This is an entirely empirical definition, similarly per definition as VC-dimension.

Definition 5.2 (Effective Model Complexity). *The Effective Model Complexity (EMC) of a training procedure $\mathcal{T}$, with respect to distribution $\mathcal{D}$ and parameter $\epsilon > 0$, is defined as:*

$$\text{EMC}_{\mathcal{D},\epsilon}(\mathcal{T}) := \max \{n \mid \mathbb{E}_{S \sim \mathcal{D}^n} [\text{Error}_S(\mathcal{T}(S))] \leq \epsilon\}$$

where $\text{Error}_S(M)$ is the mean error of model M on train samples S .

Using this definition, their main hypothesis can then be stated as the following three-fold regions:

Hypothesis 5.1 (Generalized Double Descent hypothesis, informal). *For any natural data distribution $\mathcal{D}$, neural-network-based training procedure $\mathcal{T}$, and small $\epsilon > 0$, if we consider the task of predicting labels based on n samples from $\mathcal{D}$ then:*

Under-parameterized regime. *If $\text{EMC}_{\mathcal{D},\epsilon}(\mathcal{T})$ is sufficiently smaller than n , any perturbation of $\mathcal{T}$ that increases its effective complexity will decrease the test error.*

Over-parameterized regime. *If $\text{EMC}_{\mathcal{D},\epsilon}(\mathcal{T})$ is sufficiently larger than n , any perturbation of $\mathcal{T}$ that increases its effective complexity will decrease the test error.*

Critically parameterized regime. *If $\text{EMC}_{\mathcal{D},\epsilon}(\mathcal{T}) \approx n$, then a perturbation of $\mathcal{T}$ that increases its effective complexity might decrease or increase the test error.*

It is to notice that this definition is also only observational. By that, it only outlines the specific region of interest where the paradigm shift is identified through experimental results. It has no effective predictive power, or rather descriptive power more than setting up hypothesis with respect to the effective model complexity, and some arbitrary perturbation. Nakkiran et al. (2019) also noticed this difficulty in providing a theoretical definition and theorem regarding such hypothesis, as said in their manuscript. The existence of the critical regime is also not defined.

The behaviour itself has been particularly investigated, in certain settings. For example, before Belkin et al. (2019), Advani & Saxe (2017) investigated the generalization error in neural networks of high-dimensional measure. Of such, various behaviours that bear similarity to double descent can be observed. Belkin et al. (2018) expanded on his previous research on the particular subset of kernel learning models, providing a theoretical analysis of specifically the Laplacian kernel on standard neural network. Mei & Montanari (2020) investigated random feature regression in such regard, and also found similar results.

The fact that double descent is not well-defined itself is a problem on its own. Up to the author's knowledge and of myself, there has been no effective definition or given description that outlines specifically how double descent can be structured. Instability in reproducing experiments and the effective range of the phenomena remains a question on its own.

6 NO-FREE-LUNCH

There is an interesting observation that can be made using the above formalism of the learning theory. From what we can see, if we can guarantee that $\hat{R}(h) \approx R(h)$ by their resolution, then we apparently can generalize the model fairly well, since it is approximately looking at a clear picture of the concept itself. Thereby, Theorem 4.3 is guaranteed in such case.

However, this theorem is fairly optimistic. By default, it assumes that the observations provided on c , given by the distribution $\mathcal{D}(x)$ represents such mapping $c : X \rightarrow Y$ is not fallible, that is, the observation is of $c(x)$ itself. Furthermore, it assumes of a fixed dataset with sufficient size, suggesting only in such case that for $|S| \rightarrow +\infty$ of all discrete, non-overlapping sample size, then $\hat{R}(h) = R(h)$ and hence optimization can be conducted on the exact measure of the general concept itself. This can be considered the *best-case scenario*, in a sense. In reality, this is not always a guarantee.

When talking about learning capacity of a model, we often think of its ability to perform on certain task class, for example, approximating $c \in \mathcal{C}$ of class $\mathcal{C}$ using class $\mathcal{H}$. Even if we accept the setting of Theorem 4.3, there always exist a fundamental theorem called *no-free-lunch* that state that, even of the fixed problem class $\mathcal{C}$ itself, $h \in \mathcal{H}$ can still fail.

Theorem 6.1 (No-Free-Lunch). *Let A be any learning algorithm for the task of binary classification with respect to the 0 – 1 loss over a domain $\mathcal{X}$. Let m be any number smaller than $|\mathcal{X}|/2$, representing the training set size. Then, there exists a distribution $\mathcal{D}$ over $\mathcal{X} \times \{0, 1\}$, such that:*

1. *There exists a function $f : \mathcal{X} \rightarrow \{0, 1\}$ with $L_{\mathcal{D}}(f) = 0$.*
2. *With probability of at least $1/7$ over the choice of $S \sim \mathcal{D}^m$ we have that $R_{\mathcal{D}}(A(S)) \geq 1/8$.*

Of course, this theorem is not perfect, and is often expressed within a weaker version, as seen of its binary expression. Furthermore, it assumes black-box model style, which is typical in machine learning assumption of worst-case learning. Nevertheless, under said assumptions, we can consider perfect learning to be impossible. The NFL theorem for hypothesis classes essentially says that without assumptions about the structure of the target function, no hypothesis class is universally better than any other. It is also considering a hypothesis set of fixed $\mathcal{H}$, with no sustained variation. A disadvantage of this theorem is the fact that it does not take into account of several aspects, for example, the structure and implementation of the hypothesis object itself, the narrow vision of the objective with objective function optimization, and so on. Nevertheless, it implies that no algorithm or hypothesis is superior to other, average over all possible structure. This type of average can be considered simply as either internal average or external average, depends on interpretation of matching $\mathcal{H} \mapsto \mathcal{C}$. That is, superiority in-class and superiority out-of-class that the algorithm is configured of, for example, between $h_1, h_2 \in \mathcal{H}$ or $h_1 \in \mathcal{H}_1$ and $h_2 \in \mathcal{H}_2$, for $\mathcal{H}_1 \not\subseteq \mathcal{H}_2$. The pushback however, then, comes in the form of double descent, is that within the same-class differential, specific version of $h \in \mathcal{H}$ outperforms all the other configuration, of which then happen more than once - which is why we said it is 'double'.

While it is fairly observable that the bias-variance setting is a special case of NFL - fixed objective function, infinite hypothesis class, we can observe that even within such case, NFL breaks down completely. Thereby, in said case, NFL should not be of concerned with us fairly much. But it raises an important point about analysing the space of structural hypothesis and objective landscape.

6.1 SENSITIVITY OF HYPOTHESIS

Some intuition or observation can be made using NFL examples. While it is true that of the subset problem on infinitely many hypotheses and singular objective function class NFL does not hold simply because of double descent and bias-variance tradeoff, it highlights the need for sensitivity analysis of the hypothesis class. Specifically, about the learning setting itself.

NFL is concerned with the abstract structure of $\mathcal{H}$ and $\mathcal{C}$. In other word, it is not about the model itself, but the communication-in-learning between the two. As such, factors in between can affect how the problem converges and so on, especially with the practical proxy-generalization method of partitioning dataset into testing and training quartile.

7 PREVIOUS ATTEMPTS

Ever since Belkin et al. (2019), despite the general chaotic view of double descent that seems to hold up pretty well, many researches have been conducted throughout, trying to observe or explain it, as we briefly mentioned Nakkiran et al. (2019) as one of such main paper. Before attempting to do anything, and to further contribute with hypotheses forward, we will have to resolve to review previous coursework on such matter. Most of those works include some on theoretical analysis, some on dissection of optimization method then speculation of possible hindsight, some on experiments and observable effects, and so on.

As such, the first thing to do is to contrive the consensus on 'what' constitutes double descent, at least per observable results, by delivering a conjecture.

Conjecture 7.1. *On the landscape of double descent, model complexity versus generalization error, there exists a fundamental point $\mathcal{M}_0$ of model complexity axis, where $dR(h)/d\mathcal{M} < 0$ for $\mathcal{M} > \mathcal{M}_0$, which is the interpolation point.*

While this is not perfect, it provides us with a general idea on where to look for double descent. With this, we are ready to follow up the literatures in provision of said questions.

7.1 DISCOVERY OF DOUBLE DESCENT

While double descent is discovered by Belkin et al. (2019) and further on modern deep learning landscape of Nakkiran et al. (2019), it remains that the conclusion is not reached if there exists some fundamental models that would not have double descent, the consistency of double descent through different models entirely and so on. This is mainly because of a particular 'shock effect', since before double descent, bias-variance holds the

candle as the main principle of choosing and optimizing models to its optimal saddle point. Suddenly by then we observe double descent, which introduces the interpolating threshold $\mathcal{M}_0$, that breaks this principle. There is then the open question on how far-reaching is double descent in consideration of different architectures, and this is reflected in several papers above. As of now, the main structures that have been verified includes

- **Classical models** Yang & Suzuki (2024); Nakkiran (2019b); Schaeffer et al. (2023); Lafon & Thomas (2024) (linear regression and classical polynomial), Li & Sonthalia (2024) (linear regression upon least square), Liu et al. (2021), Zhang (2024); Allerbo (2025) (kernel methods/regressions), Brellmann et al. (2024) (Classical MSBE-LSTD² optimization), Harzli et al. (2023) (Gaussian process). Some papers also have experiments on Random Fourier Feature Networks, and so on, but not substantial (including Belkin et al. (2019) paper).
- **Deep neural networks:** Luo & Dong (2023); Yang & Suzuki (2024); Pezeshki et al. (2021) (CNN, VGG, ResNet, DenseNet), Nakkiran et al. (2019) (Transformer), Shi et al. (2024) (GNN).

Most of the experiments are concerned of seeing and analysing double descent, basic analysis and experimental observations, test case scenarios for different models and setting and so on. All of the above models are then considered to be, concretely, reported of observing double descent, through various testing scenario on them. Review papers and literatures can be found as Schaeffer et al. (2023); Hu (2021), and Shi et al. (2024) on specifically, graph neural network.

7.2 THEORETICAL ATTACKS

Because of its intrinsic nature, where double descent breaks the long-standing principle of learning selection, theoretical resolutions has been employed to tackle such problem. Indeed, for example, Cherkassky & Lee (2024) and Lee & Cherkassky (2022) aim toward such direction by understanding the behaviours of double descent using the classical VC-theory approach. However, such papers only introduce, or relate double descent to VC-theory via phenomenological approach - for example, reducing the VC-dimension while fixing or reducing training error, or by examining a potential proxy between the VC-bounds and potential factors encapsulated in the quantities involved. They however, do raise voices of certain inherent problems lies within the theory of learning and in general, learning models.

Beyond such, certain discovery and inquiries help in having a clearer picture on the behaviour of double descent through various structures. For example, Hastie et al. (2019), under pretext of studying high-dimensional ridgeless (ℓ_2 norm) interpolation, analyse the case of *isotropic features*, *correlated features*, misspecified model (feature observation inhibited), on ridgeless interpolation of linear model, and soon as nonlinear model and latent space model. While strong, their results are restricted to a specific class of assumptions and structure, and is doubtful to be extended further. Allerbo (2025) investigated and find a particular double descent trace when changing kernel during training, specified in the term of the minimum allowed bandwidth. The kernel bandwidth $\sigma \geq 0$ of a specific kernel filter determines the information smoothing, hence implies the complexity abides by the kernel structure, as well as determine the smoothness of information indicted by the term.

The sentiments that are shared in between researchers on the topic of double descent vary, but have some common roots, most of them coming from the difficulty to fully assess double descent and its factor components. While double descent concerns the evaluation of *generalization (test) error*, and the *model complexity* notions, the error in question is often implicitly defined or loosely considered in one way or another, and model complexity has no general consensus on its definition. In fact, this points out a very potential, if not already mismatch in the theoretical development. While optimizations and phenomenological aberration of models, as well as building and constructing models at will are well-developed, the definitions on which they are defined on is not fully understood, if not undefined. As of now, literatures on model complexities like et al.; Hu et al. (2021b); Janik & Witaszczyk (2021); Truong (2025); Luo et al. (2024) introduce their variations or a copula of different definitions altogether, where some are concerned with its effective complexity, some are concerned with its expressive or structural complexity with different ways to define the model underneath, from parameters to units, to random standard variables (Neal et al. (2018)). There is also the rift in consideration between classical theory, of which mainly is for classical models, and modern deep neural network structures of which do not conform to such analysis.

7.3 PROBLEMS

There are a lot of problems regarding how we treat the problem, how we see it, and how we actively analyze it. Most of them lies in the space of statistical learning theory, since bias-variance on itself has variations and justification that is seemingly very natural in statistical theory, like the other version name *approximation-estimation error tradeoff*. All of them contribute somewhat to the current outlook, and so far, it has been pretty

²Mean-Squared Bellman Error and Least-Squared Temporal Difference

bad, than what is being advocated of. Nevertheless, let us state them out, even if missing some particular details.

PROBLEM 1. AUTHORITATIVE STATEMENTS

Despite its prevalence in particular research interest, double descent itself has no definite statement to describe its particular behaviour. This is also true for a list of other phenomena, such as *grokking* (Power et al. (2022)), *emergence behaviours* (Wei et al. (2022), typically discussed in large language models, which is understandable of large-scale specific ruling system), *implicit bias* (Soudry et al. (2024)), and *neural scaling law* (Kaplan et al. (2020); Wei et al. (2022)). There is also *catastrophic forgetting* (van de Ven et al. (2024)) which is still fairly native of large systems; nevertheless, those phenomena will be of target later on (or discussed in later chapters). The fact remains that we only *partially understand* these particular results with conjectures and theories to support the facilitation of new insights, new analytical lens, and nothing much. Without a clear definition of what is even double descent, sometimes, we cannot proceed, lest to say even solving it. This is particularly troublesome, since the current theory is so messy and not rigid that many even turns to exotic choices, like topological informatics learning theory, *Homotopy Theoretic and Categorical Models* (Manin & Marcolli (2024)), and several more. So this is definitely one of the main problem to pushback on.

PROBLEM 2. MODEL COMPLEXITY

The notion of model complexity can be said to be often not so well-defined. There are attempts by Hu et al. (2021a); Luo et al. (2024); Barceló et al. (2020); Molnar et al. (2020a); Janik & Witaszczyk (2021). The problem of model complexity is not so apparent until the inherent complex nature of models, typically in deep learning models (based on the MLP architecture) is observed to cannot be treated in similar way as classical models. In Hu et al. (2021a), most deep learning models are based on, and investigated of their complexity through measures like the expressive capacity, the effective capacity, and so on; aside from other teams and researchers investigated it by using computational complexity, or functional decomposition. One promising aspect of this is Molnar et al. (2020a), where we are able to decompose the functional form a specific model into variations of its complexity.

Nevertheless, it is reasonable to say there are no conclusive measure or definition of a model's complexity. Considering that double descent also appears outside the range of deep learning model, and is somehow consistent in classical models (for example in models tested by Belkin et al. (2019)). This is particularly troublesome, as a theoretical analysis requires such definition, especially when the relation of interest is directly involved in such manner. This is a particular aspect needed to be resolved if there are to be progress made in this investigation. Additionally, the concept of overparameterization and underparameterization is also a problem, especially since its definition is not well-defined, hence become a problem in analysis by identifying the classification of the problem setting.

PROBLEM 3. MODEL STRUCTURE

In modern practice, the theoretical and formal treatment, rigorous consideration of different model structures and complexity is not realized. Partially, this is due to the bloom of machine learning and artificial intelligence from the early 2000s. Currently, there has been no conclusive theoretical definition or formulation about the structure of different models, typically can go under the name of mathematical modelling hypothesis, in a way that unify certain properties between architectures. Heuristics are also often employed in several newer models and widely-utilized structures typically seen in large-scale systems, leading to difficulty in formalization and factorization of affectants.

PROBLEM 4. THEORETICAL INSUFFICIENCY

The setting surrounding the learning theory, machine learning models is generally missing of its rigours and analytical properties. Most of the learning setting and model's training-testing setting are often narrowly stated in theoretical sense, and there are a lot of diffusing details that might or might not affect the model's perturbation without knowing. This in a certain way leads to the complexity in analysing different problem setting and experimental results, since the architecture used, the setting considered, model in questions, configuration (either custom or on system side) are not consistent overall.

Furthermore, a lot of terms, definitions are often hand-waived in papers or in discussions. This also led to the point that in Nakkiran et al. (2019), they have to somehow 'reinvent' another type of model complexity itself, albeit unsatisfying of a definition, is still a new kind of definition per insufficiently of such. while it is welcome to introduce new definitions, the fact is that it is subjectively inconclusive of terms, definitions and thereof leads to the perception and situation where multiple studies can be found, but all of them are defined on different ground.

Analysing statistical learning theory and overall landscape of statistical learning theory, and the learning theory as a whole, encountering new problems like double descent also revealed its weakness, and ultimately, perhaps one of the reason why it is ineffective against such problem. First, there are simply too many assumptions made, too many formulations made during said process. Furthermore, there are also unclear notions and concepts, of which make it even harder to analyse or fully formalize. Secondly, there are inherent conflicts and uncertainty within those theories, formulations and notions by itself. For example, in one sense, the No-free-lunch theorem is considered to be representative and true, whilst also simultaneously being considered the opposite of such. And amidst abnormality behaviours of the old bias-variance formulation, we also find distinctive weakness in our theory, for example, the concerning difficulty in defining the notion of *model complexity* in various contextual ways. To analyse double descent, perhaps we also need a new theory or formulation to support it.³

Another problem with classical analytical solution is that it is very *loose*, almost too loose to even considered of such. But sometimes it is too *rigid* in a sense such that it simply does not work at all. Indeed, basing our works and foundational assumptions on top of approximation theory is not quite good at all, because they are very limited. Overall, we can say it is not equipped for handling similar situation. For example, almost all bounds that is subjected of classical learning theory only guarantee particular existential theorem, under very simplified and specific situation without the virtue of extending it to a more general setting.

This is particular event or rather pattern that we often see in learning theory development, or machine learning theory in general - the ‘**mathification**’ of a theory. Even though we need mathematics on either end, the approach is impractical or simply wrong, believing mathematics to be the singular thing that defines learning theory. This has many fallacies that can then be attributed to a lot of factors and whatnot, of several factors that plague this analysis even further than just the problem of double descent. For now,

1. Epistemological limits and interpretations (Barbierato & Gatti (2024)) - We ultimately lack understanding of a lot of things. While those ‘theorems’ are very nice in learning theory, the real picture is that it is not real learning, for the word learning are not even defined, as such is to compare them to *human learning* on itself. Even by then, theorems are severely limited. Certain voices also concern of similar problem, including Lipton (2018); Doshi-Velez & Kim (2017); Molnar et al. (2020b) (Molnar et al. (2020b) shortly consider the misleading interpretation question instead), and philosophically, with several pushbacks on structural anecdotes, Dreyfus (1965; 1972); Dreyfus & Dreyfus (1986); Suchman (1987); Brooks (1991); Searle (1980); McCarthy & Hayes (1969); Harnad (1990). On the more modern side, of contemporary critiques, Pearl (2009); Marcus (2018); Sutton (2019). Of the Chinese Room Argument, perhaps we can look into already copulated passages.
2. Applying wrongly, and is used to impress and not to explain anything (Lipton & Steinhardt (2018)) - mathematics is used to *impress* certain demographic of reviewers and readers, to provide a sense of rigours, to further enhancing the image of formal theory to the point that such theory, even if wrong, can be considered fairly correct by the sheer volume of practitioners believing in such. Such is to say the *mathiness* is turning things into ideology more than rigours itself, of which we can take a tangent to see in economic theory, one of the place to adopt a large portion of machine learning statistics, the pushback against such cursed devolution (Romer (2015); Syll (2024)). Such can also be seen in generally most double descent analysis, in which there exists many formalisms yet not definite result on double descent.
3. Reproducibility and acute false claims (Kapoor & Narayanan (2022)) - In general, what we have done cannot be recreated, in certain way, and of certain too optimistic setting that is seemingly unrealistic - particularly in adoption toward practical means of actions. Certain theoretical assumptions and formulations are too stricts, of which means the increment of hypothesis fluctuation with removal of such constraints.

³Furthermore, most of the general solution and bounds created by statistical learning theory is often in a very simplistic system. For example, if we are to utilize the Rademacher complexity measure (Mohri et al. (2012)), most of the time we will have to compute it through the growth function $\Pi_{\mathcal{H}}(m)$ for m points, on the standard finite hypothesis class $\mathcal{H}$ such that

$$\Pi_{\mathcal{H}}(m) = \max_{x_1, \dots, x_m \in \mathcal{X}} |\{(h(x_1), \dots, h(x_m)) \mid h \in \mathcal{H}\}| \quad (24)$$

Most of our problems resolve to binary classification, or rather, the problem of pattern recognizing discrete, reduced classification form. It is not so sure for now if all problems can be reduced to such way, so we cannot draw conclusive analysis that is not diluted of mathematical formulation for complex systems. That is not to count the computationally intensive operation required to calculate the supposedly classical measure, while not entirely of itself holds any substantial reasonable information about the internal dynamics of the model itself.

While most of those are not directly related to our problem - suffice to say the general narrative is so, it particularly indicates a very serious error in the theoretical development itself, and the scope of its reach has been perhaps confirmed at hand. Attempts to solve this problem might need to be an entirely new framework, of something that unify the theory at-hand, or by relinquished attendances to the current dilemma using old and trusted foundation, yet will inevitably fail to resolve further outcome of the solution since its onset is beyond the classical theory. Such is to say, we need an entire solution to cover this particular track. One is promising of them all, being the *Neural Tangent Kernel* (Jacot et al. (2018)), but other than that, nothing much came through.

7.4 CLASSICAL ANALYSIS

Classically, we can see that the problem of double descent is not inherently a problem that can be explained only by considering model complexity and the error rate. As such, it requires a framework that consider multiple factors together, and interpreting different hypothesis. Nevertheless, classical theory does indeed point to certain factors or aspect that should be of concern. For example, we have seen plenty of analysis that indicates that *sample complexity* might be the biggest factor, considering the behaviour is intrinsically operable and perhaps observable in almost any model, except for certain model that is specialized of the data structure itself. Even by then, the error still can be obtained given large enough model disparity to the concept class. There are attempts to, indeed, analyze double descent from a VC perspective (Lee & Cherkassky (2022)), however, the paper falls into the same pitfall as others in their interpretation and analysis.

Nevertheless, even though we have said plenty of the disadvantage of the classical theory in terms of statistical learning, their insight is valuable in determining the possibility of dynamic in question.

7.4.1 PAC-LEARNING SETUP

The PAC-learning process (Shalev-Shwartz & Ben-David (2014); Mohri et al. (2012); Hajek & Raginsky (2021)) connects three complexity measure - the sample space complexity, representation complexity, and optimization complexity, to the evaluation of the statistical learning process. It refers to the categorization of various learning procedure that satisfies the Probably Approximately Correct criteria in its name, as such being followed.

We assume no structure of h and c . They can be functions, partial functions, relations, complex algorithms, or others. In a typically learning setting, we also have the argument of *preliminary knowledge*, presented in literatures of the term *inductive bias*. For example, if the concept class $\mathcal{C}$ is assumed, then we say the setting is **model-specific**. If there exists no hard assumption on $\mathcal{C}$, then the learning setting is said to be **model-free**. Then, the PAC-learning set for model-specific setting is defined as followed.

Definition 7.1 (PAC-learning). *A concept class $\mathcal{C}$ is said to be **PAC-learnable** if there exists an algorithm A and a polynomial function $\text{poly}(\cdot, \cdot, \cdot, \cdot)$ of 4-argument such that for any $\epsilon > 0$, $\delta > 0$, for all distribution $\mathcal{D}$ on $\mathcal{X}$ and for any target concept $c \in \mathcal{C}$, the following holds for any sample size $m \geq \text{poly}(1/\epsilon, 1/\delta, n, \text{size}(c))$:*

$$\Pr_{S \sim \mathcal{D}^m} [R(h(S)) \leq \epsilon] \geq 1 - \delta$$

*for a given error measure $R(h_S)$. If A further runs in $\text{poly}(1/\epsilon, 1/\delta, n, \text{size}(c))$, then $\mathcal{C}$ is said to be **efficiently PAC-learnable**. When such algorithm exists, we call A a **PAC-learning algorithm**.*

This can be easily extended into the model-free case, and further on, which is left in the appendix section. The reason for the argument $\text{size}(c)$ is as followed. Consider a class of concepts defined by the satisfying assignments of certain formulae. A concept from this class that satisfies such formulae, can be represented by a formula f , a truth table, or any given formulae that is tautologically equivalent formulae f' to f . PAC-learning bound argues in the sense of *computational complexities*, and *space complexities*. Specifically, the term $\text{poly}(1/\epsilon, 1/\delta, n, \text{size}(c))$ hopes to bound the required learning process to polynomial time, specified by 4 parameters. Here, n is the *input dimension*, $\text{size}(c)$ is the encoding complexity of the target concept, $\epsilon > 0$ is the *accuracy bound*, and δ is the confidence bound - the probability that h fails.

PAC-learning procedure gives, by assumption of a **consistent learner**, two different generalization bounds for two different cases. We first define the notion of a consistent learning algorithm, or consistent learner, for a concept class $\mathcal{C}$ and hypothesis h .

Definition 7.2 (Consistent Learner). *We say that an algorithm is a **consistent learner** for a concept class $\mathcal{C}$ using hypothesis class $\mathcal{H}$, if for all n , for all $c \in \mathcal{C}_n$ and for all m , given*

$$S = \{(x_1, c(x_1)), (x_2, c(x_2)), \dots, (x_m, c(x_m))\}$$

as input, $x_i \in X_n$ outputs, then the algorithm L outputs a hypothesis $h \in \mathcal{H}_n$ such that

$$h(x_i) = c(x_i), i = 1, \dots, m$$

A consistent hypothesis is the hypothesis that is consistent with all the training data provided to it from the concept c . The two bounds then contain the debate between consistent hypothesis (i.e., for example, admitting no error on training data, almost perfect - maybe even so), and inconsistent hypothesis, of which have general defects from the actual concept, or even different by a margin. This equates to observational noise σ , typically used in simulation of noisy feature and spread, and also for randomization. Inconsistent then means the further this particular noise is increase, the higher the lower bound of the error can be for any particular measure, in comparison to the true concept c in consistent case.

PAC-learning famously leaves two results on the learning bound of the learning setup.

Theorem 7.1 (Learning bound - finite $\mathcal{H}$, consistent case). *Let H be a finite set of functions mapping from $\mathcal{X} \rightarrow \mathcal{Y}$. Let A be an algorithm that for any target concept $c \in H$ and i.i.d. samples S returns a consistent hypothesis H_S , such that $\hat{R}(h_S) = 0$. Then for any $\epsilon, \delta > 0$, the inequality $\Pr_{S \sim D^m}[R(h_S) \leq \epsilon] \geq 1 - \delta$ holds if*

$$m \geq \frac{1}{\epsilon} \left(\log |H| + \log \frac{1}{\delta} \right)$$

This sample complexity result admits the following equivalent statement as a generation bound: for any $\epsilon, \delta > 0$, with probability at least $1 - \delta$,

$$R(h_S) \leq \frac{1}{m} \left(\log |\mathcal{H}| + \log \frac{1}{\delta} \right) \quad (25)$$

Proof. See Mohri et al. (2012) for the original proof. □

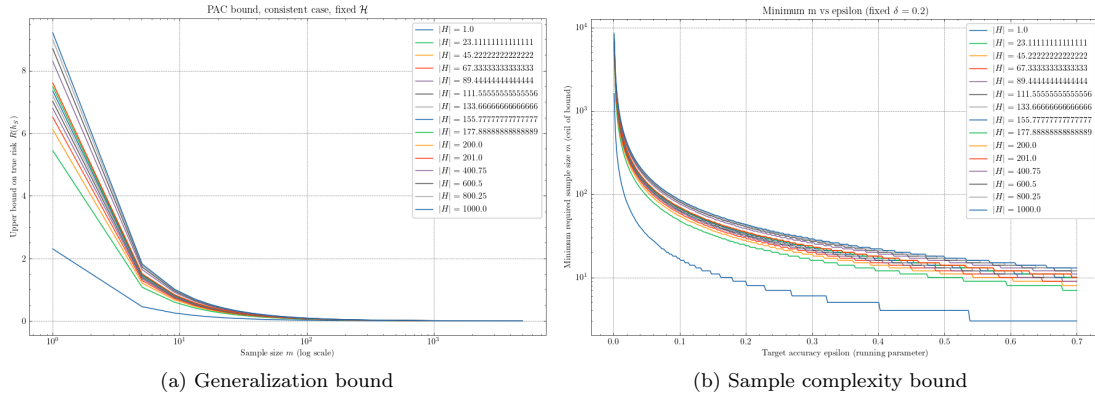


Figure 6: The learning bound, both generalization bound $R(h_S)$ and the sample complexity bound m , from theorem 7.1

Theorem 7.2 (Learning bound - finite $\mathcal{H}$, inconsistent case). *Let $\mathcal{H}$ be a finite hypothesis set. Then, for any $\delta > 0$, with probability at least $1 - \delta$, the following inequality holds:*

$$\forall h \in \mathcal{H}, \quad R(h) \leq \hat{R}_S(h) + \sqrt{\frac{\log |\mathcal{H}| + \log 2/\delta}{2m}} \quad (26)$$

Proof. Appendix section. □

For a looser bound, we can ultimately use an ϵ -consistent learner criteria in such case. PAC-learning setting raises some peculiar insight toward the problem itself. First, is the learning bound itself of which raises the importance of several factors — the dataset size, m , of which is used as the **total resources required** for approximating such function. Furthermore, the set complexity $\mathcal{H}$ is also used, albeit questionably inconsistent

in the term $\log |\mathcal{H}|$, which implies at least consideration on the cardinality of the hypothesis class. In a finite domain, we notice that PAC-learning of consistent case is exactly the case of interpolation, assuming non-smooth and noisy data, and the inconsistent case describe the wider range of approximation-based case, but does not consider an approximation-aware bound. The design of $|\mathcal{H}|$ also does not allow for certain exact hypothesis class size measurement, except for combinatorial case where $\mathcal{H}$ is indeed finite. Nevertheless, the bound indeed indicates that for m increase, the hypothesis class often do much better in general.

7.5 VC-THEORY

7.6 COMPLEXITY NOTIONS

Assume the working space is the real field $\mathbb{R}^n$. Let us recall a model $h \in \mathcal{H}$ of certain hypothesis class $\mathcal{H}$. Then, the **complexity** refers to the complexity of the hypothesis h itself; there is also the notion of class maximal complexity for $\mathcal{H}$. Complexity in this case refers to the mass of the model, or rather, to answer the following questions:

1. What is the effective expression range of a hypothesis?
2. What is the hypothesis structured in?
3. If the model is constructed by parameters, how many parameters are there?
4. How many components are there in specific model h , and how would another model $h' \in \mathcal{H}$ differs from h ?

Those questions are not so easily answered. Specifically, analysis often puts the hypothesis class $\mathcal{H}$ as a class of functions. So, there exists the class $\mathcal{H}_n$ of n -th degree polynomials, or else. Effective complexity in such case means the range of expression a function can give - for example, for n -th polynomial class, we already have the Weierstrass's theorem that tells us that any continuous function of closed interval can be approximated by a polynomial of arbitrary order. These expressions depend on the field or space the hypothesis class lives in, for example, if the hypothesis is configured to work in the set of all functions in $\mathbb{F} = \mathbb{R}$ of algebraic field, then we call it the algebraic family. There are various hypothesis classes in such case, and usually, not a lot of them can be analysed and represented in the same way, and hence also the analysis of the 'size' of the hypothesis, also between different hypothesis classes, prompted the usage of a more general notion, called **representation complexity**⁴.

We would like to recite the preliminary setting of this section. The hypothesis class $\mathcal{H}$ has two types of classification: either infinite hypothesis class $\mathcal{H}^\infty$ or finite hypothesis class $\mathcal{H}^n$. Based on the details of the expression of representation strings (Kearns & Vazirani (1994)), an infinite hypothesis class contains infinitely many specifiers of the hypothesis structure. Note that this does not correlate to an increase in parameter, as we will see with the case of axis-aligned rectangle - for only four parameters (l_1, l_2, h_1, h_2) we can specify all axis-aligned rectangles with their value taken from the field $\mathbb{F} = \mathbb{R}$. Finite hypothesis class is then can be understood of the number of configuration, or in discrete form combinatorial pairs that can be presented of the hypothesis class, for example, a string of all n -length binary numbers $\{x_1, \dots, x_n\}, x_i \in \{0, 1\}$. A subtlety that would then have to discover, is the increment in size of the hypothesis to accommodate either finite or infinite number of specifying parameters - one example can be sampled from is the class of all support vector machine (SVM, Vapnik (1999)), which for now we treat it as it is. The concept class can then also be treated similarly. Here, we assume a setting where unless specified, we have knowledge of the concept class structures, but not the hypothesis.

⁴Representation complexity refers to the number of components needed to express h and generally, any hypothesis in $\mathcal{H}$. This is often the parameters count, since the parameters themselves are used in constructing the function in algebraic system. Thus, again, an n -th degree polynomial might have $2(n+1)$ parameters, for $n+1$ variational input (or variable), and $n+1$ coefficients. Notice that we specifically at the beginning, use the notion of algebraic structure $\mathbb{R}$ because of it. If we restrict ourselves to the expression, for example, for $\mathbb{B} = \{0, 1\}$, then it can be expressed in various ways in logic theory, such as conjunctive normal form (CNF), disjunctive normal form (DNF), and else. This forbid the analysis based on the effective process itself, but rather the number of components required, hence taking them as complexity of construction.

Table 1: Descriptions of complexity measures applied to models and learning processes, from Hajek & Raginsky (2021); Sugiyama (2015); Mohri et al. (2012); Kearns & Vazirani (1994) and Vapnik (1999) with some custom notions.

Type	Notation	Purposes	Notes
Representation	$\mathcal{M}_r$	Complexity of encoding a model (size of representation, alphabet-dependent).	Model-level measure.
Expressive	$\mathcal{M}_e$	Complexity of the model’s observable behaviour (operational expressivity of the mapping it implements).	Model-level measure.
Structural	$\mathcal{M}_s$	Complexity of the model’s internal architecture (how components compose and interact).	Model-level measure.
Structural mass	$\mathcal{M}_\Sigma$	Aggregate representation size derived from representation complexity across the model.	Sum of $\mathcal{M}_r, \mathcal{M}_e, \mathcal{M}_s$
Sample	$\mathcal{M}_{p \in \mathcal{S}}, \mathcal{M}_p$	Number of samples required to reach a prescribed performance level (statistical sample-count).	Process-level measure.
Sample space	$\mathcal{M}_\mathcal{S}$	Intrinsic complexity of the data distribution / sample domain (richness, diversity, structured-ness).	Global process measure.
Optimization	$\mathcal{M}_\epsilon$	Complexity of the learning algorithm (convergence rate, computational/iteration cost, sensitivity) or worst-medium-best learning cost.	Process-level measure.

Notes: The notion of structural mass is convoluted. In certain setting, it is simply the parameter space $\theta \in \Theta$, but in some setting, there exists fundamental difference than identically weighted parameter space complexity, as seen in multi-basis polynomial models, not taking into account homogeneous deep learning networks. Optimization complexity coincides with PAC-learning bounds for polynomial runtime, and *sample complexity* sometimes can only be the dimension of the input/observation space.

The above complexity, aside from sample space-related measure, can also be applied onto the concept class’s members itself. We also take the class complexity in such case, as the argument that maximize those measure: that is, the largest value of the measure, taken on the class as the class’s complexity.⁵⁶⁷⁸⁹

The next complexity measures that can be taken of, comes from the learning process structure itself. Usually, in statistical learning theory (Hajek & Raginsky (2021); Mohri et al. (2012); Shalev-Shwartz & Ben-David (2014)), we often use the setting that is related to **supervised process**. This usually means the process can be configured in a feedback loop, or induced response toward the model - often come from the observational space, where the concept that is set to be the target of the process - then called as learning process - is conjured. For now, we reserve this to the classical learning theory and process, and will not inquire further of such direction.

The learning process assumes an operational space containing the structure, for example, $\mathbb{R}$ -encoding space of real values, and a set of **samples** living in such space, denoted $\mathcal{S}$. Those samples are often referred to as the **observation set** of a particular objective of learning. As mentioned above, $\mathcal{S}$ belongs to the concept c , assumed to be of certain concept class $\mathcal{C}$. $\mathcal{S}$ is obtained by using an observing process to create or generate the result. This process can be determined or considered in multiple way, however, generally can be considered by having a deterministic interpretation (using an explicit function $c(x)$) or by probabilistic approach (modelling a distribution $\mathcal{D}$). Hence, we gain another type of complexity, with regard to the process called **sample complexity** - usually defined of cardinality - the amount of sample needed for particular purpose, for particular objective. For the complexity category up to the structure of the learning setting by itself, we call it the **sample space complexity**.

⁵Expressive complexity measures the variety of input-output behaviours the model can implement.

⁶Structural complexity is decomposable into component-wise measures (e.g., per-layer, per-module) and useful for architectural comparisons.

⁷Sample complexity depends on hypothesis class, loss, and desired error/confidence. Further clarification must be made of a grounded definition for utilization.

⁸Sample space complexity infers the statistical measure over the sample space; affects learnability and effective sample size.

⁹We also note that the notion of optimization is usually hard to pin down analytically, often characterized empirically or by worst-case bounds.

One final complexity that is worth mentioning is then the **optimization complexity**, measure on the action of the learning algorithm. By such, it investigates the construction of an optimization algorithm solely dependent on its size of the algorithm, and other non-process related measures. Optimization complexity can also be related or expressed, in situation of time complexity, which is typical in computing system.

One might ask why we need to determine and categorize complexity measures and the need to generalize and differentiate of each one. Indeed, while in usual use cases, such complexity notion can be considered sufficient in controlling and gauging the model itself. However, just as ? indicated in the section on deep learning double descent, such complexity might eventually, of some regions, inadequate to express and evaluate the complexity of the model in its operations anymore. Such transition is unanswered, and while the founding of complexity notion is still fragmented, it is indeed difficult to not paying attention specifically to those notions themselves.

7.6.1 REPRESENTATION COMPLEXITY

Representation complexity works in assumption of the wider concept class $c \in \mathcal{C}$. However, it can also be applied onto the hypothesis class $\mathcal{H}$. To define it however, we need to define the concept of a representation scheme (Kearns & Vazirani (1994)).

Definition 7.3 (Representation scheme). *For a given concept class $\mathcal{C}$, a **representation scheme** for such concept class is a function $\mathcal{R} : \Sigma^* \rightarrow \mathcal{C}$, where Σ is the finite alphabet of symbols. We call any configuration $\sigma \in \Sigma^*$ such that $\mathcal{R}(\sigma) = c$ a **representation** of c , under $\mathcal{R}$. For any given c , the representation scheme might not be unique.*^a

^aPerhaps representation scheme can be further realized by applying a Borel σ -algebra on top, and put on it a perspective measure to handle the notion of representation size. For example, in the traditional example of axis-aligned rectangle, one such representation scheme could be the real number schema $\mathcal{R} : (\Sigma \cup \mathbb{R})^* \rightarrow \mathcal{C}$, which then can utilize the extended notion of a Lebesgue measure over $\mathbb{R}$ and its spaces.

The representation of a specific concept or hypothesis classes can be different. Consider the learning problem of a Boolean string. The representation can take form as the **conjunctive normal form** (CNF) or **disjunctive normal form** (DNF), with their expressiveness on equal, however string length varies exponentially, as seen by Wegener (1987); Miltersen et al. (2005); Darwiche & Marquis (2002). Since the learning algorithm has access only to the input-output behaviour of c , in the worst case, it must assume that the simplest possible mechanism is generating this behaviour. Within this assumption, we can now formulate the **representation size** or **representation complexity** of particular concept, denoted $size(c)$.

Definition 7.4 (Representation complexity). *Given a concept $c \in \mathcal{C}$, the **representation complexity** of c is defined to be the size of the smallest representation of the concept c in the underlying representation scheme $\mathcal{R}$, that is,*

$$\mathcal{M}_r(c) = \min_{\mathcal{R}(\sigma)=c} \{size(\sigma)\} \quad (27)$$

for any string σ of the representation scheme $\mathcal{R}$ that still represents c .

Under such definition, the more "complex" the concept c is with the respective chosen representation scheme, the larger $size(c)$ is. This is also where the concept of the concept class $\mathcal{C}$ comes from - it comes from the *representation class induction* that we have in mind some fixed concept classes we study by their representation scheme. We note that, however, the definition works within the least-lower bound limit - we choose the infimum. In the worst-case scenario, the representation potential mismatch between h and c might be very large, hence the *effective representation* might not be the same. We also operate on the assumption that different representations are equal, even though practically, we do not have the exact form of c to be sure that problems can reduce from one representation to a smaller one - which forms the basis of the **manifold hypothesis**¹⁰. Hence, the following assumptions are needed.

Assumption 7.3. *For every concept $c \in \mathcal{C}$, there exists the representation family $\{\mathcal{R}\}$ of all representation with characteristic shared by each concept c . For each c , there exists one or more representation $\mathcal{R}$ such that its representation complexity can be defined. A weaker assumption assumes that for c of one or more representation*

¹⁰The **manifold hypothesis** can be loosely expressed in the following statement by Lawrence Cayton
[...] the dimensionality of many data sets is only artificially high; though each data point consists of perhaps thousands of features, it may be described as a function of only a few underlying parameters. That is, the data points are actually samples from a low-dimensional manifold that is embedded in a high-dimensional space.

scheme, the more complex representation scheme can always be reduced to a lower representation scheme of minimal complexity.

7.6.2 OPTIMIZATION COMPLEXITY

Optimization complexity refers to the runtime in solving the learning problem by itself. Taken for example, between polynomial and linear regression analytical-probable solution solving time, the complexity lies around $\mathcal{O}(np^2 + p^3)$ for the structural complexity defined by $p = d + 1$ parameter count (including bias) via ordinary least square (OLS). However, this measure itself is not enough, and is not relevant to our problem, partially because of its operational level usage in runtime, which is often attributed to the learning algorithm convergence bound itself.

Still, we should give our own idea of such optimization complexity, called simply the ϵ -optimization complexity, rephrased from famous variations already established in the theory of optimization (hence, not a novel definition). The idea of the procedure is simple. We simply define k -ensembles on a fixed landscape $\mathcal{S}$, and calculate the worst-case, medium-case, and best-case scenario gauge of them. Before that, we need a notion on which to configure what to optimize.

Under the optimization setting, we are concerned of the optimization problem on time, $\mathcal{M}_{\epsilon,t}$, the structural mass $\mathcal{M}_{\epsilon,\Sigma}$ or decomposition of the structural mass, constrained on arbitrary ϵ . Of all those above measure, the choice of choosing specific complexity measure can be applied for the following definition.

Definition 7.5 (ϵ -optimization). *For random initialization optimization algorithm $\mathcal{A}_R(\mathcal{H})$ on class $\mathcal{H}$ of criteria R , Let the procedure $\mathcal{P}(n, \mathcal{A}_R, \mathcal{H}, M)$ of n -ensembles, and setting M , for $\epsilon > 0$, denote the complexity measure as $\mathcal{M}_\epsilon$, and the set of structural optimization method, $\mathcal{M}' = \{\mathcal{M}_{\epsilon,t}, \mathcal{M}_{\epsilon,\Sigma}, (\mathcal{M}_{\epsilon,t}, \mathcal{M}_{\epsilon,\Sigma})\}$ we have:*

- **Worst-case** ϵ Given ϵ , then the worst-case scenario means $\mathcal{M}_\epsilon(h) \leq \epsilon$ for $h \in \mathcal{H}$ for the maximum average of any $\mathcal{M} \in \mathcal{M}'$.
- **Medium-case** ϵ Given ϵ , then the worst-case scenario means $\mathcal{M}_\epsilon(h) \leq \epsilon$ for $h \in \mathcal{H}$ for the average of all ensembles of any $\mathcal{M} \in \mathcal{M}'$ that satisfy the ϵ -criteria.
- **Best-case** ϵ Given ϵ , then the worst-case scenario means $\mathcal{M}_\epsilon(h) \leq \epsilon$ for $h \in \mathcal{H}$ for the minimum average of any $\mathcal{M} \in \mathcal{M}'$ that satisfy the ϵ -criteria.

7.6.3 SAMPLE COMPLEXITY

The sample complexity can be defined in a fairly simple manner. Per usual, sample complexity, in context of learning systems, is defined as the number of training samples that it needs in order to successfully learn a target function. Certain definitions can be found, in for example, Huang et al. (2025); Wegel et al. (2025); Mei et al. (2022) or in a more specific context of active online learning, Balcan et al. (2010). However, such definition is usually too simplistic of the sample complexity itself, or compressed any reasonable information or properties are there of the sample set. Nevertheless, we should still recall the copulated definition,

Definition 7.6 (Sample complexity). *A measure $\mathcal{M}_S$ gauges the sample complexity of a learning setting on sample set S , if, the learning algorithm $\mathcal{A}(h, c)$ can guarantee, for $M_S : \mathcal{P}(S) \rightarrow \mathbb{N}$, such that M_S outputs a particular value such that for S_i the i th ordered power subset of S , then*

$$\mathbb{P}_{S_i} [\hat{R}(h) \leq \epsilon] \geq 1 - \delta \quad (28)$$

If, it can guarantee instead, such that,

$$\mathbb{P}_{S_i} [|\hat{R}(h) - R(h)| \leq \epsilon] \geq 1 - \delta \quad (29)$$

We call it the generalization sample complexity on empirical data.

Again, we emphasize the usage of this sample complexity term, however, either we *decompose it* (which is not possible), or we will need to define another notion for *sample complexity* of a given learning setting entirely. Furthermore, statistical metric of the sample set should also be considered. For example, a *statistical-aware sample complexity* can be defined by the (ϵ, μ) -sample complexity measure on Gaussian spread distribution, as followed.

Definition 7.7 $((\epsilon, \mu)$ -sample complexity). *A measure $\mathcal{M}_S[\epsilon, \mu](h, c)$ gauges the sample complexity of a learning setting on sample set S , if, for the sample distribution on particular ϵ, μ , of a random permutation*

process, the learning algorithm $\mathcal{A}(h, c)$ can guarantee, for $M_S : \mathcal{P}(S) \rightarrow \mathbb{N}$, such that M_S outputs a particular value such that for S_i the i th permuted power subset of S , then

$$\mathbb{P}_{S_i} \left[\hat{R}(h) \leq \epsilon \right] \geq 1 - \delta \quad (30)$$

This definition sought to connect, in a sense, partially, the statistical analysis of the sample space to the complexity term with respect to the hypothesis h . Even though we use the statistical set (ϵ, μ) of normal distribution process, a different set can be used, and hence changing the definition accordingly.

7.7 HYPOTHESIS

In learning and researching of the progress and treatment of double descent, several hypotheses, most of the time observations and insights such, is formed. Hence, there are several hypotheses for what to look out for, as well as the formulation needed to interpret certain phenomena and observations. This ranges from topics of re-evaluating the statistical learning theory, estimation and refactor of assumptions (or *inductive bias* when designing models), to different interpretations and representations that can better explain, model, or gives certain insights.

7.7.1 STABILITY OF BIAS-VARIANCE MEASURE

Stability of bias-variance measure is questionable, as it is in literature (Domingos (2000d)), loosely defined by defining a 3-dimensional vector $\lambda = (\lambda_1, \lambda_2, \lambda_3)$ for such:

$$\begin{aligned} \mathbb{E}_{\mathcal{D}} [d((f, \mathbb{E}[y | x]))] &= \lambda_1 \text{Bias}(f, y) + \lambda_2 \text{Var}(f, y) + \lambda_3 \epsilon(\mathcal{D}) \\ &= \lambda_1 \underbrace{\{\mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D})], \mathbb{E}[y | x]\}}_{\text{bias term}} + \lambda_2 \underbrace{\{(f(x; \mathcal{D}), \mathbb{E}_{\mathcal{D}}[f(x; \mathcal{D}))\}}_{\text{variance term}} + \underbrace{\lambda_3 \epsilon}_{\text{irreducible error}} \end{aligned}$$

where $\epsilon(\mathcal{D})$ is the irreducible error of the system, depends (theoretically) on the intrinsic imperfection of the dataset $\mathcal{D}$. Assessment on stability of such decomposition is recommended.

7.7.2 ALTERNATIVE MEASURES

We believe that the bias-variance decomposition is a poor estimation and measure for controlling, validating and choosing model from, at least from the perspective of analysing missing details. Specifically, bias and variance term, as well as their supposed expression of decomposition in the standard loss measure of model effectiveness is not representative, nor expressive of the actual, underlying notion of *model complexity* $C(f)$ and its effectiveness. Furthermore, bias-variance decomposition is not straight-forward of others type of loss functions, except only for some classes of loss function (loss on Bregman divergences and exponential class). This sentiment is shared in Brown & Ali (2024).

7.7.3 INDUCTIVE BIAS

Inductive bias obviously plays a role, as mentioned by some literature¹¹, though we don't know exactly how the inductive bias can be formed to play any role in the central dynamic of the model operating process. Certain assumption, for example, the *convexity of the loss function* might actually affect the model as it is, and introduce unwanted patterns in the following process.

7.7.4 RANDOMNESS AND PROBABILISTIC SETTING

Random initialization of parameters, as well as the initial conditions of the model initialization process might also be taken into account. Obviously, this is not a new insight, and is rather a very old one, however, this approach still has varied potential for interpreting double descent. Scaling up/down this effect might result in new insight, as well as inspecting the uncertainty and diffusion patterns in large-scale model deployment, in which from an end-to-end perspective requires several space transformation operations in successions.

7.7.5 KNOWLEDGE MASKING

The very ambiguous idea or treatment of statistical learning, as well as machine learning, is the idea of masked. Or, paraphrased, *hidden information* regarding the underlying process of estimation. Specifically, for example, we tend to assume things to not be able to grasp - either the probability $p_X(c)$ of the concept class, the distribution $\mathcal{D}$ specified to such input distribution, or the concept class $\mathcal{C}$, Markov properties, etc. We would like to formalize this notion as soon and as such in the most direct way, as it is one source of the many assumptions that we have in concession of theoretical results.

7.7.6 INTRINSIC DATA FACTORS

The effect that data has on the model should also be taken into account, as of fact. As far as we are concerned of, the shape, practical system that the dataset gives the setting, the factors running into the dataset, the choice of the dataset being sampled, selected in partitions, and other factors must also be analysed. Without such, we leave 'gaps' in which sensitive, potentially chaotic behaviours might occur frequently and would not be identified, or being classified into irreducible errors, static of the learning setting. As such, we might as well list a few potential points of interest.

¹¹Most notably, Lafon & Thomas (2024), of which raised particular point about the need of learning with inductive bias as a structural presumption. Additionally, the paper also includes an analysis on the inductive bias of gradient descent, though solutions and particular application on double descent explanation is doubtful.

1. **Data distribution:** the distribution of the dataset itself affects how the concept is represented in the space. If the representation space created by using the data is large, but sparse, then the condition would make for some behaviours to occur more easily. This is one degree of freedom of the system setting. If the dataset spread is low, yet the data is very volatile, then it will lead to inconclusive picture of the actual concept, hence introducing new behaviours on the now unknown region of the distribution. As such, the majority of those unknown spaces will treat every point there as a potential outlier instead.
2. **Data complexity:** The complexity of the data itself is of interest. In a controlled setting of response-inducing evaluation, identifying behaviours of the model with a glimpse on the data complexity - how the actual data works with the internal mechanics accompanied - is of specific interest, as it can affect some of the majorative effects and chaotic behaviours that occur in the learning process, especially stochastic process and probabilistic optimization patterns.
3. **Data partitions:** The partitioning of data in the previously mentioned sample partitioning section might give rise to previously observed behaviours, as well as explaining particular volatility when changing the test partitioning set. The ordering of the dataset also helps in such regard.

It is also worth to note that since we emphasize the use of dataset partitioning to replicate the empirical-generalization guarantee, it is also then indicative that such partitioning would have its separate way of calculating test error on itself.

Intuitively, it can be illustrated as followed. Suppose of given two datasets $\mathcal{S}_1, \mathcal{S}_2$, we can trace a pattern, if, for assumption, the partition is $k = 10$. If the algorithm $\mathcal{A}$ attempt empirical learning optimization specifically, and best optimize on individual partition $\mathcal{S}_1^{(i)}$, then the path toward each and individual partition would potentially be very volatile, depends heavily on the partition randomization, and others factor leading to such. If the algorithm is justified in that sense, to ‘decelerate’ the optimization to a not so tight fit, the generalization would be much better. This is also the reasoning for the technique of **regularization**, and generalization task. However, again, such procedure cannot help but having volatile dataset itself as a factor of uncertainty. More specifically, the result might work for $\mathcal{S}_1$, but for another dataset $\mathcal{S}_2$, assuming no intersection, that is, $\mathcal{S}_1 \cap \mathcal{S}_2 = \emptyset$, then optimization based on one dataset would not be optimal in another, more so can be counterproductive. This is also used of the reasoning for **cross-validation** technique, of which aims to stabilize the partitioning. However, analytical analysis suggests a wider role of such, in the wider setting of learning theory, particularly in double descent later on.

8 ANALYSIS

For understanding and analysing double descent and bias-variance trade-off, and furthermore in later section on identifying double descent, we would like to use several test models specifically for exhibiting bias-variance, as well as testing hypothesis and forming theoretical conjectures. This is because of the troublesome nature of the phenomena itself, which proves difficult to analyse without sufficient experimental observations. Furthermore, specific analysis on these types of model might prove useful for generalization toward more difficult and complex structure.

Our major focus is on three types of model. First two are classical models and system, linear-polynomial models on the real space $\mathbb{R}^n$. Second, is the type of **support vector machine** (SVM) models to highlight the analysis toward data-based fundamental analysis. The third one is the class and collection of different *neural network structures* (MLP, or variations). Our goal is then to expansively analyse, formalize, making conjectures and hypothesis and test them, to find, force double descent out, or identify it on the learning landscape. We note that structures of those models are not well-generalized, hence analysis from one might not apply or apply only in small conjunction. This approach is rather quite helpful in its own merit, as its choice and preference will gauge our parameters than we gauge the phenomena of the parameter it induces.

8.1 GRADIENT-BASED OPTIMIZATION

In our usual experiment, we would most specifically, use a closely-regarded family of gradient-based algorithms, such as gradient descent (Cauchy (1847); Robbins & Monro (1951); Widrow & Hoff (1960); Hestenes & Stiefel (1952); Fletcher & Reeves (1964)), mirror descent, natural descent, and so on (Nemirovski & Yudin (1983); Amari (1996; 1998); Yang & Amari (1998); Rattray et al. (1998); Gunasekar et al. (2021); Luong et al. (2016); Xiao & Zhang (2014); Sohl-Dickstein (2012); Raskutti & Mukherjee (2013)). While implementation and details of the specific algorithm can be different, they all shares some of the same concepts of which making them possible for optimization in the specified iterative domain. The most general formulation possible is the following algorithm on convexity:

Algorithm 3: Gradient descent on general convex function

Data: $(x_i, y_i) \in \mathcal{S}$, η , $\nabla(\cdot)$ operator.
Result: Expected optimization to the minimal error on $\ell(\cdot, \cdot)$.
begin
 $x_1 \leftarrow$ randomly initialized points;
 for $t \leftarrow 1$ **to** T **do**
 $x_{t+1} \leftarrow x_t - \eta \nabla f(x_t)$
 end
end
return $\hat{x} = 1/T \sum_{t=1}^T x_i$

Nevertheless, one should be cautioned of the intrinsic assumption on which gradient descent of this type assumes. It assumes *convexity*, which can only be possible, if the landscape itself is supposedly, assumed to contain such global minima. Said global minima in a richer language can be described as the epistemic collapse of the learning structure, under the arbitrary optimization space. This is a *very serious assumption*, and should it fail, as seen in a lot of non-populated or general observation space, the entire procedure is invalid. It is also troublesome as to rely on the singular objective (numerically encoded) space in which it is trivial to realise that there are indeed, local minima. This scaling, with respect to both locality and global consideration, must be considered carefully, since *every single contribution* might distort the experiment in a very unexpected way.

8.2 LINEAR FUNCTION CLASS $h(u)$

Linear-linear scenario learning is one particularly well-studied case of representing double descent or not, usually through simplicity in analysis (of the similar ease of proof compared to binary classification). Models in this setting can be said to share the same *parameter space* with no additional structure or scaling mask (like in monomial basis). The model that is representative of this preliminary observation is from Lafon & Thomas (2024), specifically, linear regression (function approximation) with Gaussian noise, and of univariate case. Consider the family class $(\mathcal{H}_p)_{p \in \llbracket 1, d \rrbracket}$ of linear functions $h : \mathbb{R}^d \mapsto \mathbb{R}$ where exactly p components are non-zero ($1 \leq p \leq d$), we have the following model.

Definition 8.1. For $p \in \llbracket 1, d \rrbracket$, $\mathcal{H}_p$ is the set of functions $h : \mathbb{R}^d \mapsto \mathbb{R}$ of the form:

$$h(u) = u^T w, \quad \text{for } u \in \mathbb{R}^d$$

With $w \in \mathbb{R}^d$ having exactly p non-zero elements. The complexity here itself can be interpreted as the effective random initialization, trimming down data of hidden details instead - this is indicated by the amount of information it observes given d dimension, and p -hypothesis dimension. ^a

^aAccording to the *manifold hypothesis*, there are chances where the actual concept can be interpreted and mimicked by seeing that it lies on a lower-dimension manifold inside its actual expressive dimensional space.

Using such model, they consider the setting of minimizing the customized mean square loss,

$$\min_{w \in \mathbb{R}^d} \frac{1}{2} \|Xw - Y\|^2 \rightarrow \min_{w \in \mathbb{R}^p} \frac{1}{2} \|X_{\sim p}w - y\|^2 \quad (31)$$

of the sub-problem on p cardinality, where again p is the dimensionality or independent degree of freedom. This results in the following theorem that 'mimic' double descent on this linear regression model.

Theorem 8.1 (Double descent on linear regression). Let $(x, \epsilon) \in \mathbb{R}^d \times \mathbb{R}$ independent random variables with $x \sim \mathcal{N}(0, I)$ and $\epsilon \sim \mathcal{N}(0, \sigma^2)$, and $w \in \mathbb{R}^d$. we assume that the response variable y is defined as $y = x^T w + \sigma\epsilon$. Let $(p, q) \in \llbracket 1, d \rrbracket^2$ such that $p + q = d$, $X_{\sim p}$ the randomly selected p columns sub-matrix of X . Defining $\hat{w} := \phi_p(\hat{w}, [\hat{w}])$ with $(\hat{w}) = X_{\sim p}^+ y$ and $[\hat{w}] = 0$.

The risk of the predictor associated to $\hat{w}$ is:

$$\mathbb{E}[(y - x^T \hat{w})^2] = \begin{cases} (\|w_{\sim q}\|^2 + \sigma^2)(1 + \frac{p}{n-p-1}) & \text{if } p \leq n-2 \\ +\infty & \text{if } n-1 \leq p \leq n+1 \\ \|w_{\sim p}\|^2(1 - \frac{n}{p}) + (\|w_{\sim q}\|^2 + \sigma^2)(1 + \frac{n}{p-n-1}) & \text{if } p \geq n+2 \end{cases} \quad (32)$$

The point $p = n$ is representative of the interpolation threshold from underparameterization to overparameterization.

As said, this theorem is designed to mimic the double descent behaviour on the range of linear regression with Gaussian noise. While there exists a proof for this particular theorem, to rule out possible falsification, experiment to verify the claim of this particular result will have to be conducted.

We present two parts - first are those fundamentals predicted by the theorem, and second are verifications of the theorem with our own version of testing. The first experimental setting is then conducted with fixed d , varying p for each varying n samples. Since this is a least square result, preliminary run is conducted using one-shot error-risk measure on least square closed-form approximation. Note that $p \leq d$, hence we are regarding the domain of the term underparameterized and equal parameterization setting. The result is presented in Figure 7. As we see from this particular experimental result, the theorem predicted a loose, left-peak absent, double descent curve estimation for a variety of test case with more and more samples in increment. There exist some fundamental conclusion that we have about the shift of the interpolation threshold upon to $p = n$, which is reflected in the piecewise prediction. However, this pattern breaks down for n higher than specific constant, for this case, starting from $n \approx 100$ to beyond. From here, at least from testing scenario, the test indicates that predictions suggest a breakdown of the double descent estimation. Beyond this, a simple semi-chaotic convergences is observed, from $n = 106, 112, 121$.

Interestingly, verifying this notion of easy enough to simplify it into what the statement holds under particular notion. Namely, as we find out, the above double-descent replicating formula can actually be realized by the parameter-wised error, comparing the parameter between the least-square solution and the actual concept. The formula for this is expressed by the exposed MSE on parameter function, for $\beta_{p,k}$ the coefficient value estimated for the k -th selected feature when the model is restricted for randomization of p features,

$$\text{MSE}_{\text{param}} = |\hat{w} - w|_2^2 = \sum_{j=1}^d (\hat{w}_j - w_j)^2 \quad (33)$$

where $\hat{w}_j$ is chosen such that it is equal 0 for $j \notin S$, otherwise $\beta_{p,k}$ for $S = \{i_1, \dots, i_p\}$ is the set of selected feature indices. Nevertheless, it is then found dubious why this is true, as for the original statement of Lafon & Thomas (2024) indicates correlation to the prediction error of the result, $x^T \hat{w}$ instead. The result of evaluating $\text{MSE}_{\text{param}}$ is presented in Figure 8.

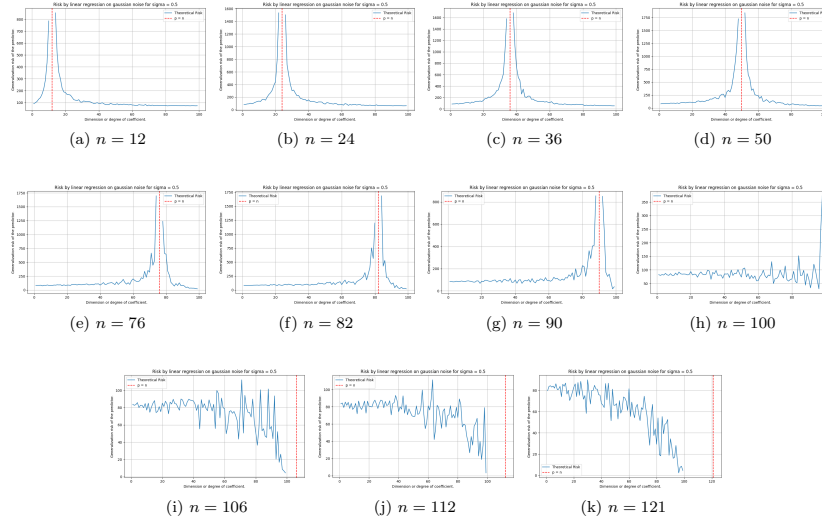


Figure 7: Theorem 8.1 behaviours on randomized setting. For this model, we have $d = 100$, $\sigma = 0.5$, and variational n . Full test cases are for $p = [1, 100]$, $n = \{12, 24, 36, 50, 76, 82, 90, 100, 106, 112, 121\}$ accordingly. The test function of the concept itself is the same linear model, but with different input only.

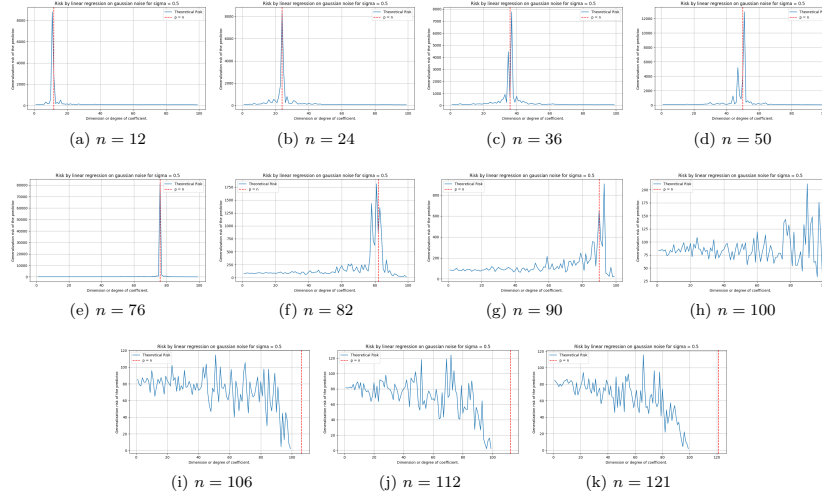


Figure 8: Contrary to Theorem 8.1, this is the behaviours on randomized setting for standard parameter-wised mean square error measure (MSE-on-parameter). For this model, we have $d = 100$, $\sigma = 0.5$, and variational n . Full test cases are for $p = [1, 100]$, $n = \{12, 24, 36, 50, 76, 82, 90, 100, 106, 112, 121\}$ accordingly. The test function of the concept itself is the same linear model, but with different input only.

If, however, we turn our attention to the statement itself, of which refers to the prediction error MSE_{pred} , defined by the equation

$$\text{MSE}_{\text{pred}} = \frac{1}{n} \left\| y - X_p \hat{\beta} \right\|_2^2 = \frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - x_p^{(i)T} \hat{\beta} \right)^2 \quad (34)$$

on h and c accordingly, a strong case of this happens to be not following the pattern indicted by the theorem. Rather than being the interpolation point of return, $p = n$ is now the convergence point of the prediction error instead. Test result is shown in Figure 9.

Thence, the theorem gives us two things - the interpolation point, in this setting, refers to the parameter error for their double descent behaviour. However, on the prediction results' error measure, the interpolation point

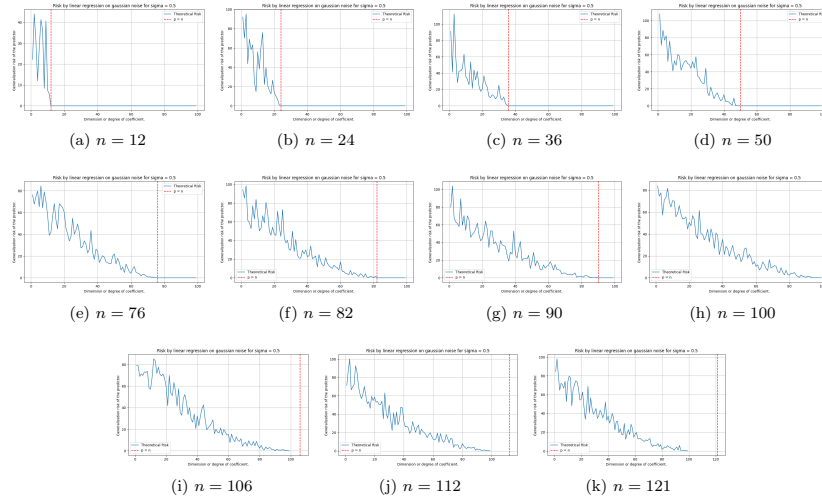


Figure 9: Contrary to Theorem 8.1 and both figure 8, this is the behaviours on randomized setting for standard parameter-wised mean square error measure (MSE-on-prediction). Especially, we do not see double descent pattern in this setting, as it is. For this model, we have $d = 100$, $\sigma = 0.5$, and variational n . Full test cases are for $p = [1, 100]$, $n = \{12, 24, 36, 50, 76, 82, 90, 100, 106, 112, 121\}$ accordingly. The test function of the concept itself is the same linear model, but with different input only.

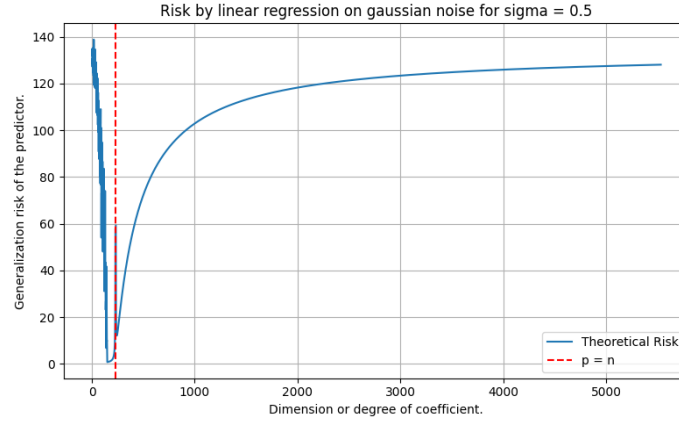


Figure 10: Similar experiment according to setting and result of Theorem 31, where p is in the range $[1, 5523]$. Here, we can see small double descent modelled exactly at the interpolation threshold, while the later region exhibit similar phenomenon as bias-variance tradeoff.

correlates to the point where the defined generalization risk in such case equals zero. These patterns hold up to certain point. However, the test case is only up to $d = p$. An unexplored test case for $p > d$ indicate the surprising fact according still, to the theorem: the generalization risk takes double descent as a region with disturbance in the middle, but bias-variance appears afterward. This is illustrated in Figure 10, where the same theorem is applied but in the domain which is usually called as *overparameterized region*.

Such can then be seen as a prediction that double descent in certain case, can be extrapolated to be the transition between the two curvature of bias variance, or the abnormal point on the error curve measure. Such must then be verified, both in the experiment's correctness, and so on that lead to such result. Nevertheless, if true, it would mean that double descent indeed has another implication of itself.

8.3 LINEAR FUNCTION CLASS EXTENSION

The current theory prompted for a more general treatment of the linear function class, as in-line with exploring reasonable existing alternatives in interpretation (Nakkiran (2019a)).

Let us states some hypothesis about the previous experimental cases. First, is the lack of the begin-tail of the prediction on double descent of linear random cutoff features model. This can be explained informally as to be resulted from the uniformity of the operating space. Specifically, all parameters of the $\mathbb{R}^n$ parameter space, with respect to the dual space on such $f : \mathbb{R}^n \rightarrow \mathbb{R}$ projection is defined, are of the same order of magnitude. That is, for $f_{i,k}(x), f_{j,k}(x)$ of arbitrary parameter indexing on the set k of shared parameter choice, and $i \neq j$ being the distinct choice, then we say $f_{i,k}(x) = \mathcal{O}(f_{j,k}(x))$ of the growth of both function on value ranges. Approximately speaking, the choice of parameter does not dramatically growth to a point of no return, like in later examples of polynomial approximation model. Such special model of multi-variable linear projection of scalar output also reinforces the numeric growth of the numerical encoding expression on the hypothesis model. The behaviour of the MSE-on-parameter, the internal risk, is also explainable using the same notion, such is that for a black-box case approximation, such linear model does not allow for average parameter deviation per approximation to be oversaturated and skewed (for example, the degree of polynomial directly counteract this, by separating expressive complexity by growth degree). How such hypothesis can be evaluated, we will have to make it for the general extension experiment, which we will have to conduct.

Let us define the more general class of linear model, denoted $\mathcal{H}_L$, in the following structure of the linear model. We also note that the previous linear class $\mathcal{H}_p$ only solves for the problem of underparameterized, hidden parameters p' .

Definition 8.2 (General linear model). *Let us define, for $p \in \llbracket 1, d \rrbracket$, $\mathcal{H}_L$ is the set of functions $h : \mathbb{R}^{d+1} \times \mathbb{R}^q \rightarrow \mathbb{R}^q$ of the form:*

$$h(\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k) = \begin{cases} [f_j(\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_k)]^\top \mathbf{w} + b & \text{general case} \\ [u_1, u_2, \dots, u_n]^\top \mathbf{w} + b & q = 1, f_j(\{u_i\}) = u_i \\ [(u_1 \odot u_2 \odot \dots \odot u_n)_j]^\top \mathbf{w} + b & q > 1, f_j(\{u_i\}) = \mathbf{u}_1 \odot \dots \odot \mathbf{u}_n \\ [(u_1 \oplus u_2 \oplus \dots \oplus u_n)_j]^\top \mathbf{w} + b & q > 1, f_j(\{u_i\}) = \mathbf{u}_1 \oplus \dots \oplus \mathbf{u}_n \end{cases} \quad (35)$$

where $\odot$ is the Hadamard product, $\oplus$ the vector addition, and $\mathbf{w} \in \mathbb{R}^d$ the weight matrix. A subset $\mathcal{H}_{L,p}$ then consists of all set of functions h with $w \in \mathbb{R}^d$ having exactly p non-zero elements, fixed between all iteration.

Preliminary model learning then will relies on theorem 8.1 for its initial prediction, using least square approximation results. The testing is adversarial and consistent, of which we have $c \in \mathcal{H}_L$ and $h \in \mathcal{H}_{L,p}$ for $p = d$, which can then further be modified. Because of their equal expressive capacity, respect to independent consideration of the parameter space $\mathbf{X}$, we define the complexity, which will be used later on, simply as

$$\mathbb{C}(h) = \frac{1}{\|\bar{\mathbf{x}} - \underline{\mathbf{x}}\|} \left[1 + p + q + \frac{\mathbb{E}_{i \in S}(w_i x_i) + b}{\|\bar{\mathbf{x}} - \underline{\mathbf{x}}\|} \right], \quad S = \{p_i\} \quad (36)$$

Functional prediction on the classical theorem is presented below, for $n = \{1, \dots, 241\}$ samples, and with $p \in [1, 200]$. We set the target degree to the maximal 200, since the theorem is not defined on the overparameterized region.

A design of the above class will require, in general, a much less straightforward implementation than the least square method, at least when it comes to circumstances and case fielding. The previous experimental result was done using a straightforward, single-time calculated theorem. To verify such, we need to adopt the non-zero class in a way that is static of the entire training procedure, and so on, for larger and larger q . First, we discuss of the mismatch calculation and thereof taken from the iterative error segmentation. In our cases, we have h and c with *model mismatch*, that is, there might exist certain parameter that h has, but the concept c of its originality does not, and vice versa for the concept model of which the hypothesis does not have, or attain certain parameter of interest. This prompts us to make use of a type of **penalty** for such case. With respect to the mean square error on parameter, from the perspective of the supervisor, we have two ways to implement this, either of the truncate or padding mode. The truncate mode simply calculates parameter expression of those that can be compared. By default, for the learning problem to be consistent of the space, there must be at least one parameter that is not inhibited. Then,

$$\text{MSEP}_{\text{truncate}}(h, c) = \frac{1}{k} \sum_{i=1}^k (p_i - q_i)^2 \quad (37)$$

For $K = \min(n, m)$ of the overlap, or can be changed to be the set $k = \{(h_i, c_i)\}$ of which matches each other. Such can be changed to absolute error, if one does not want to inflate the error measure beyond its distance metric with respect to certain axis. The padding mode simply takes into account of all missing or

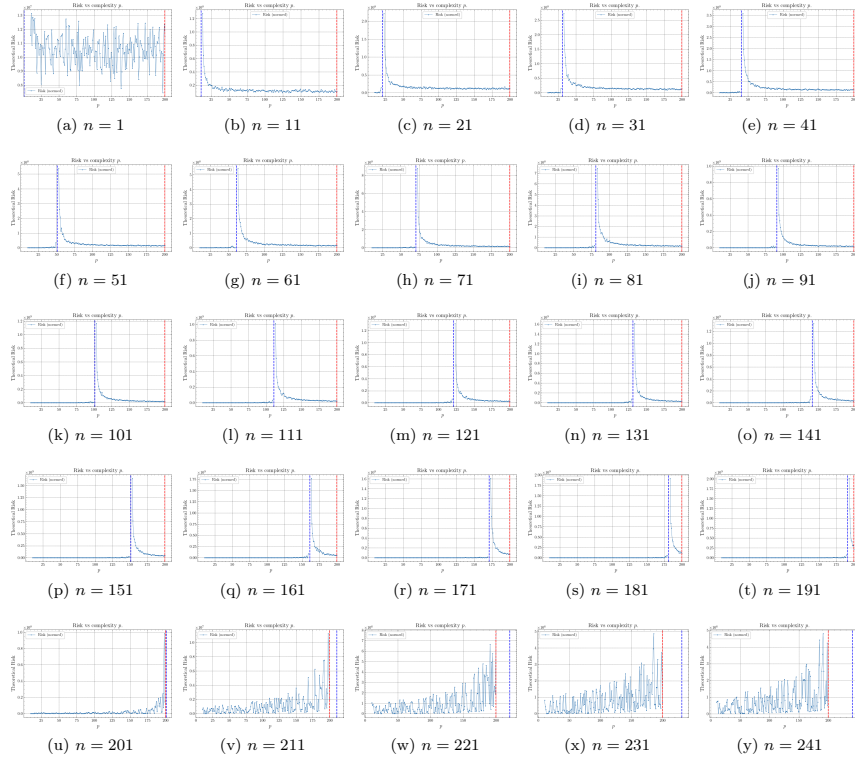


Figure 11: Extension general experimentation, using the prediction made of theorem 8.1 for verification. Here, $q = 1$ as the general basis case of verification.

surplus information and of the optimization phase. By initialization, from the hypothesis side, all surplus parameters are padded with input 1. Hence, the measure is defined as

$$\text{MSEP}_{\text{pad}}(h, c) = \frac{1}{k} \sum_{i \in k} (p_i - q_i)^2 + \lambda \left(\frac{1}{n-k} \sum_{i \notin k} p_i^2 + \frac{1}{m-k} \sum_{i \notin k} q_i^2 \right) \quad (38)$$

The total error then is simply the above error, is simply calculated toward all layers possible or any architectural design parameters. This design will be also used in subsequent testing, including polynomial, in which the effectiveness is enhanced significantly.

8.3.1 CASE 1. $q = 1$

8.4 POLYNOMIALS

Polynomial space is one of those famous example that is often cited of the bias-variance tradeoff, for example, in Goodfellow et al. (2016); Sugiyama (2015). Indeed, it is considered, usually, a very effective and high-volatility construct class. Let us reiterate what we know about polynomial class.

There exists two type of elementary polynomial classification, based on their (i) variable count or (ii) its domain range. We first define the univariate polynomial function.

Definition 8.3 (Univariate polynomial). *A function of a single variable t is a polynomial on its domain if we can put it in the form*

$$a_n t^n + a_{n-1} t^{n-1} + \cdots + a_1 t + a_0 t^0 \quad (39)$$

where $\{a_i\}$ are constants, and of a singular variable t .

From this, we can extend to polynomial of multiple variables. In general, a function $f(x, y)$ is a *multivariate polynomial* in x, y if and only if it can be represented as a finite sum of terms of the form $cx^k y^m$, where c is a coefficient and k and m are nonnegative integers. The number $k + m$ is called the degree of the term, and the degree of the polynomial $f(x, y)$ is equal to the highest degree of its own terms. We can then generalize multivariate polynomial and univariate polynomial in the same kind.

Definition 8.4 (Polynomial, multivariate generalization). *Let K be a field. We then denote $K[x_1, \dots, x_n]$ the space of all polynomials in n variables, $x_1, \dots, x_n$. A **multivariate monomial** is then an expression $x_1^{\alpha_1} \cdots x_n^{\alpha_n}$ where $\alpha_1, \dots, \alpha_n$ are integers. The degree of such multivariate polynomial is the largest degree of terms of any monomial term.*

Effectively, we can see that multivariate polynomial can take on many forms, and also unconventional forms. In order to capture any reasonable details, most of the time focus are on symmetric polynomials $f(x, y)$ in which $f(x, y) = f(y, x)$, or homogeneous polynomials in which all the terms are of the same degree.

Given such, operationally, polynomial is used for expressing other functions, due to their usefulness because of such simplicity and versatility. In fact, applications ranges, even to a polynomial basis on n -dimensional space, $(p_1, \dots, p_n) \in K[x_1, \dots, x_n]$; their growth rate being polynomial also added more to their strength of approximating fast increasing function, usually expressed and used in setting, for example, polynomial growth class $\mathcal{O}(p(x))$ in computer science. Two applications stand out for such properties: function approximation of arbitrary criteria ℓ , or polynomial interpolation on arbitrary points. Those two are interconnected, though their objective and object of concern is much different.

We can define the application of analytical polynomial on arbitrary function by using the criteria on approximation. Usually, this is expressed in the theory of approximation, and requires a guarantee that polynomial can indeed approximate to arbitrary accuracy, of arbitrary function on specific interval. This is the statement of Weierstrass theorem.

Theorem 8.2 (Weierstrass Approximation Theorem). *Let $f \in C[a, b]$ of all continuously differentiable functions on the closed interval $[a, b]$. Then, for every $\epsilon > 0$, there is a polynomial p such that $\|f - p\| < \epsilon$, for $\|f - p\| = \sum_{i=1}^{\infty} |(f(x_i) - p(x_i))|^a$*

^aWe again note the disparity between $h(x) - c(x)$ or $c(x) - h(x)$ for clarity. In essence, $c(x) - h(c)$ is looking from the perspective of the concept itself. While $h(x) - c(x)$ is comparison on difference from the perspective of the hypothesis.

Here, ϵ is defined to be the *global error* of the polynomial with respect to the arbitrary function, and with uniform approximation. Another variation works on the squared error, such that $\|f - p\| = \sum_{i=1}^{\infty} (f(x_i) - p(x_i))^2$, or the mean absolute

$$\|f - p\| = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n (f(x_i) - p(x_i))$$

Weierstrass Approximation Theorem provides tighter bound for the mean absolute error, but not the square error, or the mean square case. Nevertheless, they are natural measure on the uniform metric space $C(X)$ with measure $d(f, g) = \sup_{x \in X} |f(x) - g(x)|$. If said metric also use the L^p metric instead, which is defined by

$$d_p(f, g) = \left(\int_X |f(x) - g(x)|^p dx \right)^{1/p} \quad (40)$$

we can also see that it is weaker than the bound given by Weierstrass theorem. Generalization of Weierstrass theorem can be made by Stone-Weierstrass theorem instead, of which is state in real space as followed, though generally Weierstrass theorem is sufficed.

Theorem 8.3 (Stone-Weierstrass Theorem (real version)). *Let S be a locally compact Hausdorff space. Let A be the subset of bounded continuous real functions on X , or of subalgebras $\mathcal{C}(X, \mathbb{R})$ which contains a non-zero constant function, then A is uniformly dense in $\mathcal{C}(X, \mathbb{R})$ if and only if it separates points.*

The setting of polynomial interpolation is rather different from the setting of function approximation. The setting of interpolation gives an n samples set $\mathcal{S}$, of which in this case we restrict to $\mathbb{R} \rightarrow \mathbb{R}$. Then, we are required to fit exactly the entire dataset, such that the following setting is sufficed.

Definition 8.5 (Interpolation). *A function class $\mathcal{F}$ is said to **interpolate** a distinct dataset $\mathcal{S}_n = (x_n, y_n)$ of n samples, if there exists $f \in \mathcal{F}$ such that $\mathbb{E}||S_i - |f(x_i)|| = 0$. Such f is then called the **interpolating function** of $\mathcal{S}_n$.*

Under such definition, the question is then if the polynomial class $\mathcal{P}_n$ of any given polynomial with the highest degree n is sufficed to approximate such dataset. The answer can be found using unisolvence theorem.

Theorem 8.4 (Unisolvence theorem). *Given $n + 1$ bivariate data points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n) \in \mathcal{S}_n \subset \mathbb{R}^n$, with all x_i distinct, there exists a unique polynomial $p_n(x)$ of degree at most n such that $p(x_i) = y_i$ for $i = 0, 1, \dots, n$.*

In determining such problem, we can also say that there exists a unique n -saddle point function represented here as polynomial, such that to fit n points perfectly by the definition of the saddle point. The process in obtaining such formula is difficult, however, and finding the perfect interpolation polynomial would be computationally significant, and erroneous.

Definition 8.6 (Saddle point, Nie et al. (2021)). *Let $X \subseteq \mathbb{R}^n$, $Y \subseteq \mathbb{R}^m$ be two sets (for dimension $n, m > 0$) and let $F(x, y)$ be a continuous function in $(x, y) \in X \times Y$. A pair $(x^*, y^*) \in X \times Y$ is said to be a saddle point of $F(x, y)$ over $X \times Y$ if*

$$F(x^*, y) \leq F(x^*, y^*) \leq F(x, y^*), \quad \forall x \in X, \forall y \in Y \quad (41)$$

Polynomial regression is one such analytical solution to function approximation task, using Weierstrass theorem as its basis. From such, we can also observe that polynomial regression, and hence polynomial interpolation, are correlated and can be reduced to each other. Polynomial regression can be reduced to interpolation via stating the criteria such that, denoting it as ℓ , then $\ell(f, p) = 0$, and interpolation can be reduced to regression via the condition that $p(x_i) - y_i \leq \epsilon$ for arbitrary $\epsilon > 0$. Hence, the regression formulae lie between the two extreme: the Weierstrass setting and the interpolation setting.

We then have to identify such extreme and what kind of extreme they represent. Fortunately, we know this as the problem of **transparency**, in the setting of approximation, which certainly correlates to the learning theory. According to such, the Stone-Weierstrass represents the total transparency case, where the function f is known to the approximator with polynomial class. Interpolation represents total black box, where the goal is only to interpolate the sample point by itself. In between such interpretation, there exists a fundamental assumption, hence the structural assumption.

Assumption 8.5 (Transparency). *For any bivariate pair (x, y) , we assume each pair depends on and originate from a based concept $y = f(x)$.*

Two kinds of knowledge is then available, which correlates to the degree of freedom the observation space can have. First is the knowledge of f . This knowledge of f is encoded directly by the number of sample point and their distribution in certain case. Accordingly, the worst case is as followed.

Conjecture 8.1. *If $\forall x$ there exists y corresponding to such, and $(x, y) \in \mathcal{S}$, then we say we have total transparency to the concept f . If there exists only finite x that we can have y corresponding to, then we say we have limited transparency to the concept f . If we assume no knowledge of f , then we say the setting is called the **black-box setting**.*

Secondly is the action on the dataset itself. One of the main assumption that is usually observed is that

Assumption 8.6 (Fallacy). *We assume every bivariate pair $(x_i, y_i) \in f[\mathcal{S}_n]$, such that $f(x_i) = y_i$.*

However, usually, this is not the case, as we have **noise** in addition from the dataset. The origin of the noise cannot be tracked. However, we can interpret what happens when such noise is introduced to the dataset. The noise transitions the transparency of the learned setting, on the field $\mathbb{R}$, by utilizing the dense property of the real space. Assuming we operate on $\mathbb{Z}$. Then, each discrete noise will introduce abnormality point such that no function on $\mathbb{Z}$ will be able to match, up to certain distance on $\mathbb{Z}$. This is because on such space, there exists no point x_1, x_2 , for example, that lies between $d(x_1, x_2) = 1$. If the noise is randomized on ± 1 , then there will exist no function that can approximate such disparity better than a random fluctuating function. However, on $\mathbb{R}$, the real number is inherently dense, such that there will always be certain point x_3 such that $x_3 \in [x_1, x_2]$, by the density of real number. By the distance metric on $\mathbb{R}$, we define $d(X, Y) = |X - Y|$ for distance between two data points. Noise intrinsically increases such distance, such that $d'(X, Y) > d(X, Y)$. Hence, the space in which density can appear increase, thus there then exists more fluctuation, and thus more uncertainty between such two points. This notion can be increased from local uncertainty to global uncertainty, hence the *noisy dataset*.

We then can measure or aggregate some properties of the dataset itself, including the **uncertainty** of the dataset. We define the simple uncertainty of the dataset as followed.

Definition 8.7 (Dataset uncertainty). *Given a closed interval $[a, b]$, for the sample set $\mathcal{S}_n = (X, Y) \in [a, b]$, the uncertainty $\mathcal{U}(\mathcal{S}_n)$ is defined as*

$$\mathcal{U}(\mathcal{S}_n) = \left[\mathbb{E}_{x_i \in X} (x_{i+1} - x_i)^2 + \mathbb{E}_{y_i \in Y} (y_{i+1} - y_i)^2 \right]^{1/2} \quad (42)$$

We assume X, Y are totally ordered sets.

Hence, the assumption of fallacy assumes $\mathcal{U}(\mathcal{S}_n) = 0$, though in reality it is often $\mathcal{U}(\mathcal{S}_n) > 0$. With this, we can quantify how the dataset is behaving on the scale of the bivariate form typically seen. Noise intrinsically increases this distance, usually by the y -axis. This dataset uncertainty increases with multivariate dataset, for example, $(\mathbf{x}, y)$ of $\mathbb{R}^n \rightarrow \mathbb{R}$. We call the new metric as the *empirical uncertainty metric*. From this, we can have the theorem:

Theorem 8.7 (Uncertainty tradeoff).

Thirdly, is the treatment on such information. Even though information is known, such as from the assumption of transparency and the fallacy conjecture, it relies on the method in which the model uses that make use of such information. Polynomial interpolation effectively removes information about transparency, and thus the interpolation setting is a kind of **black-box absolute setting**. This setting is *extremely brittle*, as to see that if y_i is slightly wrong due to measurement noise, the interpolant oscillates wildly, which is explained by Runge phenomenon (Runge (1901); Corless & Severyi (2018); Boyd (2009)). Probability then arises from such setting naturally, though this particular degree of probabilistic setting must be quantified, which comes from the process that the model simulates upon. Regression is inherently not probabilistic, but treatment about the probabilistic assumption, for example, such that (X, Y) inherently as random variables. We can then separate this into three parts - the data-level probabilistic layer (data uncertainty), the model-level layer, and the algorithm-layer that determines the solution solver within such model. We can define that as followed.

Definition 8.8 (Probabilistic layer). *Given a learning setting with dataset $\mathcal{S}$ of bivariate pairs, $h \in \mathcal{H}$, $c \in \mathcal{C}$ as the hypothesis and concept, we have the following conceptual probabilistic setting:^a*

1. *Data-level probability: Sampling assumptions (observation sample points) such that there exists the distribution $\mathcal{D} \sim (X, Y)$, intrinsic data uncertainty $\mathcal{U}(\mathcal{S}_n)$.*
2. *Model-level probability: Structure of the dataset is controlled by the concept structure $\{\theta\} \in \mathcal{c}$, c is modelled as a target sampler $c \equiv \mathcal{D}$, knowledge about the structure of c , Bayesian prior-posterior assumptions, $\mathbb{P}(Y \mid X, \{\theta\})$. For Bayesian polynomial regression, the model-level*

probability is formulated as

$$\mathbb{P}(Y | X) = \int \mathbb{P}(Y | X, \theta) \mathbb{P}(\theta) d\theta, \quad \mathbb{P}(\theta_j) = \mathcal{N}(0, \sigma^2) \quad (43)$$

of which assumes aleatoric uncertainty and epistemic uncertainty from the weight.

3. Algorithm-level probability: Stochastic method, decision conditional (dropout, MCMC), ensembles criteria, Monte-Carlo initialization, variational inference.

^aCurrently, we have no way to **quantify** the ‘probabilistic degree’ of the system setting itself

Dynamically, in this paper, we would like to investigate this particular junction between uncertainties, to see what happens when the model transparency, the treatment of data, and so on, that exhibits the observations that we would receive from the model itself. The experimental details will infer such result.

Generally, we would be entitled to see particular trajectory specifically designed around the point of interpolating match, or $n = m$ of n degree of freedom, technically, and m data points. After this point, there exist two possibility of dynamic - either the model dilutes, in which case result in *double descent*, or the model fluctuate strongly afterward, leading to *bias-variance emergence* again later on. This conclusion however, would have to be analysed within the current framework of the possible interjection to its fluctuation strength. For example, in the monomial basis $\{x^i\}^n$, any given fluctuation contributes to the entire polynomial. In such case, fluctuation of the other $[0, n - k]$ of arbitrary k and large enough n would be unaccounted for by the scale. Conversely, restrict them to $[1, 0]$ range will reveal something about the system, however will destroy particular properties of the prediction, which is not recommended. Nevertheless, we must conduct the test before giving in any reasonable conclusion.

Previously, we said that we did indeed focus on the interpolation problem. Now, assuming we have given the interpolating algorithm $\mathcal{A}$ to approximate the interpolation, what will this look like? Firstly, solving this is *very computationally expensive*, because it is a dense, $(n + 1) \times (n + 1)$ linear system, and even more so if we consider the overparameterized outcome. However, this can be reduced further down if we attempt to solve them using convergence bound, but will be still very expensive. That and incremental soft interpolation is somewhat out of the question since it will be kind of the same. Aside from the insight, we would still have to know what to do. Then there exists two ways - we have some version of this to optimize of simply evaluating by gradient descent optimization, or, we can use somewhat analytical fit for the case of soft interpolation modification.

A theory on such problem can be obtained of its mechanics, as will be a fairly intuitive way in consideration of the theory of interpolation, as well as the technique of interpolation by itself. For a fixed-range interpolation threshold problem, a hard-encoded interpolation solution requires n -degree polynomial, of which has $n + 1$ degree of fitness, to fit $n + 1$ distinct points. Suppose all points are distinct of the set S which is finite, then this is the case. If we switch to the problem of interpolation but induce a more forgiving bound, there exists a possibility that it induces a relative *density averaging path* for the points, especially if we are considering the case where $p > n$ for p saddle points or degree of fitness, and n the number of point to interpolate. In a more nuanced formulation, we have

$$\text{Flatten} \left(\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^p \\ 1 & x_1 & x_1^2 & \cdots & x_1^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_p & x_p^2 & \cdots & x_p^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^p \end{bmatrix}_{n \times p} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_p \end{bmatrix} - \begin{bmatrix} f(x_0) \\ f(x_1) \\ f(x_2) \\ f(x_3) \\ \vdots \\ f(x_n) \end{bmatrix} \right) \leq \epsilon \quad (44)$$

or equivalently, $\text{Flatten}(\mathbf{X}_{n \times p} \mathbf{w}_n - \mathbf{Y}_n) \leq \epsilon$ where $\text{Flatten}(\cdot)$ is the compression of vector to scalar, or so $\text{Flatten} : \mathbb{R}^n \rightarrow \mathbb{R}$. This essentially is the problem of solving n points approximation, toward using only p saddle points available. If $\epsilon = 0$, it is well known that interpolation is impossible. However, if $\epsilon \neq 0$, there are possibilities, depends on the shape of the dataset, that there will exist a data-dependent property or more that indict a particular *density preference* toward those saddle points - or rather, the act of density averaging. This is reflected rather implicitly in the structure itself. In machine learning, the error measure is usually the mean average, such is to say that the criterion now would look more like $\mathbb{E}[\text{Flatten}(\mathbf{X}_{n \times p} \mathbf{w}_n - \mathbf{Y}_n)] \leq \epsilon$. Formally, we have the following hypothesis

Hypothesis 8.1 (Polynomial double descent). *For a polynomial $\{a_i x^i\}_n$ on Vandermonde basis, approximating on the set $\{(x_k, y_k)\}_m$, then*

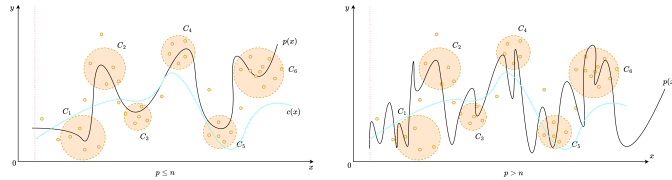
1. *For $n < m$, we expect the polynomial form factor to be inherently sparse, such that for an iterative approximation, for each p_i saddle point of the polynomial expansion form, there exists an ϵ -ball B_ϵ for $\epsilon > 0$ with at least one data point satisfying $(x_k, y_k) \in B_\epsilon$.*
2. *For $n = m$, we expect the polynomial to be able to fit all points, up to hard interpolation threshold such that $\hat{R}_S \approx 0$.*
3. *For $n > m$, we expect the polynomial form factor to be locally dense then turning to globally dense, over random ensembles. That is, for every point (x_k, y_k) there exists a β -ball B_β such that there is at least one saddle point of the polynomial belong to the ball.*

The shift that will be hypothesized to appear here is expected to be from sparse in the polynomial term sense, out to globally dense with enough model complexity, over an iterative process. One also main point of clarity is, at certain point right now, we haven't discovered the effect of error deviation, yet. For such, we theorize the following.

Conjecture 8.2 (Double descent). *Double descent happens, for a polynomial class model $h_n \in \mathcal{H}_p$, is upper bounded to certain error deviation $\epsilon > 0$ from the mean only of the generalization (test) dataset verification.*

Figure 12: **Intuition of Hypothesis 8.1.**

Effectively, in the overparameterized range ($p > n$), the problem turns into soft interpolation, where the polynomial is possible to be dense enough such that relative error remains internally low. This can also happen if the dataset itself is intrinsically large.



That is, the disparity between the test and training dataset helps in such regard. If the two dataset is partitioned in the sense that their deviation is practically the same (which can be done using synthetic generation for a control group), then this will amount to not much of it. After all, it will be the consequence of the algorithm and error landscape interpretation used.

For this, we propose another speculative observation beside such, as to consider adjourned transform of the Vandermonde basis to the Newtonian or similar basis, in which the effects and motion can be easily observed, as least easy. This is because aside from mapping down to $[-1, 1]$, normal polynomial on monomial basis gives suboptimal result because of intrinsic high-degree term domination.

This will become more relevant later on in the analysis.

8.4.1 POLYNOMIAL MODEL STRUCTURE

Now, for mathematical analysis and experimental design, we would like to state the current main structure of a polynomial model. Specifically, in such example, we are concerned of the shallow polynomial neural network, of which is stated as such in Arjevani et al. (2025). Notice that it is defined on the normal Vandermonde basis, where an expression of such polynomial model under Newtonian or Lagrangian basis is not so clear-cut of an analysis.

Definition 8.9 (Shallow polynomial neural network). *A shallow polynomial is a neural network of one layer that are functions $f_{\mathcal{W}}$, defined such as: $\mathbb{R}^d \rightarrow \mathbb{R}$ of the form*

$$f_{\mathcal{W}}(x) = \sum_{i=1}^r \alpha_i (w_i \cdot x)^d, \quad \mathcal{W} = (\alpha, w_1, \dots, w_k) \in \mathbb{R}^r \times (\mathbb{R}^n)^k, \quad (45)$$

where $d \in \mathbb{N}$.

A modification is then available, in which we simply add $b(\cdot)$ for random initialization process. We note that this introduces certain flexibility to the model initial interpolation curvature.

Definition 8.10 (Polynomial neural network, general). *The polynomial network based on Model 8.9 is modified as*

$$f_{\Theta}(x) = \sum_{i=1}^d \theta_i (\mathbf{w}^{\top} x)^i + b, \quad \Theta = (\{\theta_i\}, \{w_k\}, b) \quad (46)$$

From this, we can state the model within a testing scenario listed as such, to fully capture the structural foundation of the related testing model class. For the notion of complexity, naively, we employ two kinds of complexity measure. First is the usual complexity measure via parameter counts. Second, is the expressive complexity on the basis of the linear transformation space. That is, an expressive complexity that take into account the fact that $\mathcal{O}(wx^n)$ will always be stronger than $\mathcal{O}(wx^{n-1})$ for $x > 1$. All of which can be contained in a singular expression, as seen in definition 8.11. However, we would have to analyse what is happening inside the metric itself. In the equation for expressive complexity, the first term is the normalized linear parameters θ_i fixing and controlling the shift of the polynomial inning parameter, the second term is the sum of weight-aware complexity with degree factor, and the degree factor over operating range $\|x_{max} - x_{min}\|$ of the dataset, for the degree complexity. The final term is the last weight-aware complexity for the dominating degree n of the polynomial function. Let us begin with the gauge on coefficient norm, i.e. the term $\|\alpha_d\| + 1$. Here, we set 1 as baseline, and the arbitrary norm on α there coefficient effect. The terms coefficient in this expression matters because, for a function $f : [a, b] \rightarrow \mathbb{R}$, the Lipschitz constant L for polynomial f goes by

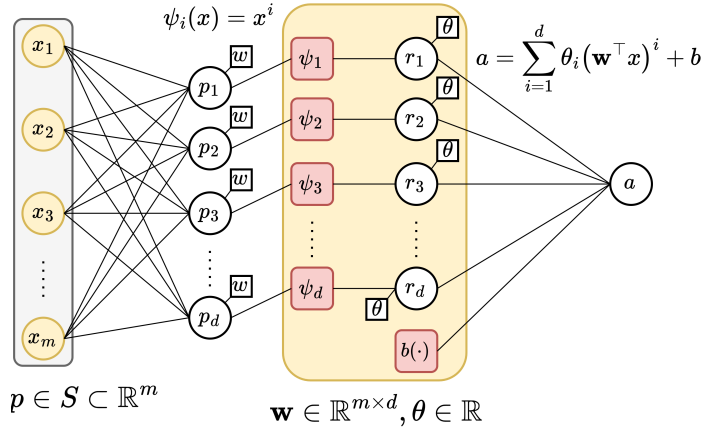


Figure 13: Illustration of polynomial model in definition 8.10.

$$\begin{aligned}
 L_p &= \sup_{x \neq y} \frac{|f(x) - f(y)|}{|x - y|} \\
 &= \sup_{x \neq y} \frac{|f_{\Theta}(x) - f_{\Theta}(y)|}{|x - y|}, \quad f_{\Theta}(x) = \sum_{i=1}^d \theta_i (\mathbf{w}^{\top} x)^i + b \\
 &\leq \sup_{x \in [a, b]} \|f'_{\Theta}(x)\|_2 \\
 &= \sup_{x \in [a, b]} \left\| \sum_{i=1}^d i \theta_i (\mathbf{w}^{\top} x)^{i-1} \mathbf{w} \right\|_2 \\
 &\leq \|\mathbf{w}\|_2 \sup_{x \in [a, b]} \sum_{i=1}^d i |\theta_i| |\mathbf{w}^{\top} x|^{i-1} \\
 &\leq \|\mathbf{w}\|_2 \sum_{i=1}^d i |\theta_i| \max\{|\mathbf{w}^{\top} a|, |\mathbf{w}^{\top} b|\}^{i-1}.
 \end{aligned} \quad (47)$$

Where $\|\mathbf{w}\|_2$ is the Euclidean ℓ^2 norm. The magnitude of $f'_{\Theta}(x)$ depends directly on the coefficient size α_i , and also on how big is x on the domain. All of this mean for large expression and value range on f_{Θ} , the Lipschitz constant of that model is accordingly large. Defining α -norm that way is a cheap proxy for Lipschitz constant, since it gauges stability, and rate of change of the functional expression itself.

Definition 8.11 (Polynomial expressive complexity). *For $x > 1$ or $x_i > 1$ of $\mathbf{x} \in \mathbb{R}^n$ as input. Denote the polynomial class as $\mathcal{P}_n$ of their highest degree n , then we define the total expressive complexity of $p \in \mathcal{P}_n$ as*

followed.

$$\mathbb{C}_E(p) = \mathbb{E} \left[\|\alpha\|_d + 1 + \frac{\sum_{i \in S \setminus \{p_n\}} w_i i}{\|p\| - p_n} + \frac{1}{\|\bar{x} - \underline{x}\|} \left(\frac{\sum_{i \in S} i}{\|p\|} + w_n n \right) \right], \quad (48)$$

where $S = \{p_j\}$, $\|p\|$ denotes the amount of different degree separable terms and $\|\bar{x} - \underline{x}\|$ is the observable range of the dataset.

8.4.2 TESTING RESULT

The testing scenario will be as followed. The following set of parameters are required for controlling the testing case, aside from different system measures. For parameter error, we reuse equation 38 but with degree aware terms.

Table 2: Controllable parameters and hyperparameters for polynomial class analysis

Information	Notation	Details
Hypothesis model	$h[\mathcal{W}_h, \mathbf{H}_h] \in \mathcal{H}$	Hypothesis model. For simplicity, it contains $\mathcal{W}$ as the configuration parameter set, and $\mathbf{H}$ the fixed hyperparameters
Concept model	$c[\mathcal{W}_c, \mathbf{H}_c] \in \mathcal{C}$	Concept or target model. Similar configuration as the hypothesis model
Complexity measure(s)	$\{\mathbb{C}(\cdot)\}$	Set of complexity measure of choice for any arbitrary model taken in.
Sample count	m	The number of sample available for hypothesis model of c , for $m = \mathcal{S} $ dataset
Hypothesis dimension (hyperparameter)	$[d_h]$	An explicit hyperparameter contains the list of all in-testing polynomial degree for the hypothesis model.
Concept dimension (hyperparameter)	d_c	An explicit hyperparameter of the concept's true dimension. Theorized such that $d_c = d_{h,i}$ for arbitrary i will reduce to possible low-error problem.
Data noise	ϵ_S	Dataset-wide noise with relative strength to the value of the sample point. This is to reduce vanishing noise with higher-degree polynomial sample points.
Learning parameters	epoch, lr , bs	Two main parameters for gradient-descent, iterative-based algorithm - the epoch, the learning rate, and the batch size.

Notes: We output to $\{\mathbb{E}[\ell]\}$ as the expected value of the error measure on h, c . There are a few things possible, but under approximation setting, we use MSE-on-parameter and MSE-on-prediction for two different insights: internal representation and external representation.

We note a few problems with both interpolation interpretation and polynomial regression of higher-degree terms. Firstly, is the intrinsic interpolation error and numerical, software issue error. Higher degree polynomial can be represented as solving for a wide Vandermonde system of n degrees for m sample points. Using monomial basis for interpolation, either for soft interpolation also exhibits instability and numerical garbage within the operations, as indicated by Shen & Serkh (2025). They can also be a potential source of double descent, as the majority of models tested around this range are not coded effectively for handling large data operations as such, and artifacts from such choices might be taken as the indication of double descent in some particular cases.

Aside from it, and aside from the fully-connected case, we have the particular testing scenario that can be expected of the ease of implementation. First, is the mundane implementation that is of gradually increasing degree mismatch. That is, for polynomial of h_p and c_p , one requires only d_h and d_c to figure out the structure as there will exist exactly $d + 1$ with the constant term, terms of which of finite many degrees. The second case is where for p chosen randomly, such that aside from such p entry, then the other degree or nodes of input are simply scattered dependent nodes, linear algebraically. That is, for a uniformly constructed d -degree network, choose at random p particular degree to be fixed, while the other of $d - p$ are assigned randomly of belonging to some $p_i \in \{p\}$ nodes given the random choice on p above. Linearly algebraically, scattered and dependent nodes then mean for example, node 4 might use node 35's degree, and vice versa (if they were both non-fixed). The dependency structure is deliberately randomized rather than organized. For overparameterized case, we simply design the hypothesis by the following line of justification. For a mapping $\mathbb{R}^d \rightarrow \mathbb{R}$, the mapping $\mathbb{R}^{d+n} \rightarrow \mathbb{R}$ for $n = 1$ requires additional information, since the source provision does not contain information to handle additional n dimensional input. Hence, we design it the following. For the hypothesis h , such unit of which does not fit to the original input space, the *internal input space* that h 'hypothesize'

contains, for the other n entries, $q \leq d$ per unit input. That is, each input of n overparameterized case receives at random, q inputs can be received, for example, $w_{n1}x_j + w_{n2}x_k$ for k, j specific input numbering. By the simplicity argument, then $q = 1$ for the simplest case. However, this creates a skewed polynomial network, of which informally, the provided structural bias or additional information, is the fact that there exists overparameterization. To prevent this, instead, the final case considers the most general structure — for each unit, each one can take q inputs controlled as hyperparameter of the network, then the set $\{p\}$ of chosen degree influence their degree choice, and is uniform if the sampling process it is not included.

9 AXIOMATIC FORMALIZATION

Concurrent theories are inherently unstable and with many variations, making them ill-suited for instant application on analysis of different models without losing major information about the structural details and expressions. Hence, we propose a novel analysis on the basis of the axiomatization of the learning setting, under various structural assumptions.

9.1 LEARNING THEORY FORMALISM

The classical theory of learning simply does not have much to offer in terms of a unified perspective, in which important factors are analysed of its realization. Specifically speaking of such, classical theory does do worse in many parts, for example, the aspect of transparency given the model, supervisor and the concept, of which their roles are not fully understood or formulated correctly, both in controlled setting and extension to replication of real life examples. Thereby, the problem requires a newer theory of learning to come forward. A new learning theory must extend beyond what constitute the basic learning model, such as the idealized PAC-learning, and statistical learning setting in general. More specifically, we want to design a learning theory that is more akin to **modelling theory** of its internalization, but also attain the learning of machine learning theory.

Within the constraints of such theoretical framing, we can then construct a different notion in which the generalization and the empirical learning problem can possibly relate to each other, at least from the statistical framework. Let us consider the problem of mathematical modelling in such sense. Beside the model class construction, which will be discussed in further sections,

9.2 THE IDEA OF NEURAL NETWORK FORMALISM

Previously, we have been discovering, examining past results and testing on models of different types, generally classical models. While decisive, each modelling setting requires different analysis and is often not so streamline. Hence, we propose a general analysis on the structure of a feed-forward style neural network architecture, as usual, for further analysis and identification.

A neural network (Goodfellow et al. (2016); Zhang (2023)) N is typically specified by the dual $(L, [S])$ where L is the number of processing layer of a neuron, and $[S]$ is the list of neurons configuration per layer - for example, the number of neurons per layer, and so on. For such network, the definition can be simple as followed

Definition 9.1 (Standard multilayer network, Zhang et al. (2023)). *We define a K -layer fully-connected deep neural network with real-valued output. Let $m^{(0)} = d$ and $m^{(K)} = 1$ for m the width of the network, and d the shape of the input vector. We then recursively define:*

$$x_j^{(0)} = x_j \quad (j = 1, \dots, m^{(0)}), \quad (49)$$

$$x_j^{(k)} = h \left(\sum_{j'=1}^{m^{(k-1)}} \theta_{j,j'}^{(k)} x_{j'}^{(k-1)} + b_j^{(k)} \right) \quad (j = 1, \dots, m^{(k)}), \quad k = 1, 2, \dots, K-1 \quad (50)$$

$$f(x) = x_1^{(K)} = \sum_{j=1}^{m^{(K-1)}} u_j x_j^{(K-1)} \quad (51)$$

where the model parameters can be represented by $w = \{[u_j, \theta_{j,j'}^{(k)}, b_j^{(k)}] : j, j', k\}$ with $m^{(k)}$ being the number of hidden units at layer k ; $\theta \in \mathbb{R}^m$ the weight of the neuron.

Here, each neuron is denoted by their expression as $f(\mathbf{W}^\top \mathbf{x} + b)$ for the activation function f . We can simplify a lot of those structures above into the neural network schemes. For example, the linear function class case can be simplified to a single $(2, [n, 1])$ neural network, such as:

$$x_j^{(0)} = x_j \quad (j = 1, \dots, n) \quad (52)$$

$$x^{(1)} = \sum_{j'=1}^n w_{j'} x_{j'}, \quad (53)$$

without the activation function h . The polynomial neural network can be represented by

$$x_j^{(0)} = x_j \quad (j = 1, \dots, m^{(0)}), \quad (54)$$

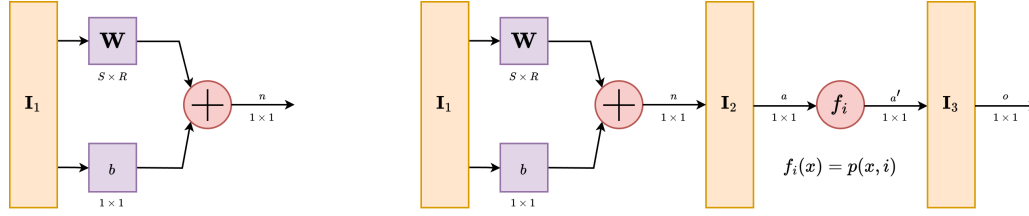


Figure 14: Flow diagram of the shallow neural network structure in linear and polynomial case, following the notation of Demuth et al. (2014) in *Neural Network Design*. The Support Vector Machine flow diagram looks similar to the case of polynomial, except a decision function at the end of the output, and changing the function f into the kernel function $K(x', x)$.

$$x_j^{(1)} = h \left(\sum_{j'=1}^{m^{(0)}} \theta_{j,j'}^{(1)} x_{j'}^{(0)} \right) + b_j^{(k)} \quad (j = 1, \dots, m^{(1)}), \quad h(x_j) = x^j \quad (55)$$

$$f(x) = x_1^{(2)} = \sum_{j=1}^{m^{(1)}} u_j x_j^{(1)} \quad (56)$$

for a neural network of shape $(3, [n, n, 1])$, and of polynomial custom scaling function. For support vector machine, this is even harder to do, since its definition varies between different notion of the model itself. However, it is interesting because we can treat it as an arbitrary generation of a neural network structure. There are papers about formalizing SVM in the sense of neural network, including making it to be an infinitely-wide shallow neural network, for example, Chen et al. (2021); Wang et al. (2019); Zhang & Wang (2014), however, the structure that is formed from the SVM learner can be expressed as followed:

Definition 9.2 (Support vector machine setting). *For any given optimization problem on input space $x_i \in \mathbb{R}^d$, and label $y_i = \{\pm 1\}$, the shallow neural network resultant from the training procedure is always of the form:*

$$x_j^{(0)} = x_j \quad (j = 1, \dots, m^{(0)}), \quad (57)$$

$$x_j^{(1)} = \sum_{j' \in m^{(1)}} \theta_{j,j'}^{(1)} x_{j'}^{(0)} K(x_{j'}^{(0)}, x) \quad (58)$$

$$f(x) = \text{sign}(x_1^{(2)}) \quad (59)$$

for a specific kernel $K(x, x')$.

Those three fundamental conversions to neural network raise interesting insight into the neural network formalism. For linear and polynomial shallow neural network, we design the network and then provide it with a training scenario, evaluated of the proxy generalization. SVM however can be considered an adaptive data construction, where the neural network is constructed from the dataset or observation set itself, using minimization on RKHS and with constraints. This prompts exactly as such, on one side the pre-optimization construction, and one is the post-construction optimization of a neural network architecture. This interpretation is surprisingly useful when talking about neural network's unit-like structure. Hence, from such evidence prompts us to analyse a neural network, not in only the sense of managing its layers, but also by its unit components together.

This way of analysing unit-wise gives us much more interpretability, and also potential for a newer kind of network specification. As far as we are concerned of, a neural network can represent an active *mathematical modelling system*. This then indicates, for example in adversarial learning case between the target neural network tailored to specific concept, and the learning hypothesis network, then the process of learning itself can be loosely interpreted as a concept of mathematical modelling representation learning, even in the case of black-box model, and not just simply phenomenological modelling of statistical data. To do this, we will examine the structure from the perspective of a **generator**.

9.3 SUPPORT VECTOR MACHINE

Support vector machine, first formally originated from Vapnik (1999), is a model inherently, specifically defined for pattern recognition task, and binary classification via an *optimal separating hyperplane*. There are two variants for SVM, namely, for linear and nonlinear hyperplane, but the main general aspect of SVM is its reversal of the direction on model building - instead, it aims to minimize the structure of a model, using the data-dependent structure, or *support vectors*.

The SV machine implements the following two precursor ideas: It maps the input vectors x , supposed of the setting, into a high-dimensional feature space Z through some nonlinear mapping, chosen a priori. In this space, an optimal separating hyperplane is constructed. By statistical learning theory, Vapnik restricted the function class (as for infinite hypothesis it is impossible to learn) to the class of hyperplanes by

$$\langle \mathbf{w} \cdot \mathbf{x} \rangle + b = 0; \quad \mathbf{w} \in \mathbb{R}^n, b \in \mathbb{R} \quad (60)$$

which $\mathbf{w}$ is the controlling weight, $\mathbf{x}$ is the input space to the space of all binary $\{-1, +1\}$ category. Hence, this basically divide the input space into two: one part containing vectors of the class -1 and the others being $+1$. If there exists such a thing, then it is said to be *linearly separable*. To find the class of a particular vector $\mathbf{x}$, we use the following decision function

$$f(\mathbf{x}) = \text{sgn}[\langle \mathbf{w} \cdot \mathbf{x} \rangle + b] \quad (61)$$

This is called the **hyperplane classifier** class¹². The above form of finding final classification can then be presented in dual form, which then depends only on dot products between vectors, as

$$f(\mathbf{x}) = \text{sgn} \left(\sum_{i=1}^{\ell} y_i \alpha_i \langle \mathbf{x} \cdot \mathbf{x}_i \rangle + b \right) \quad (62)$$

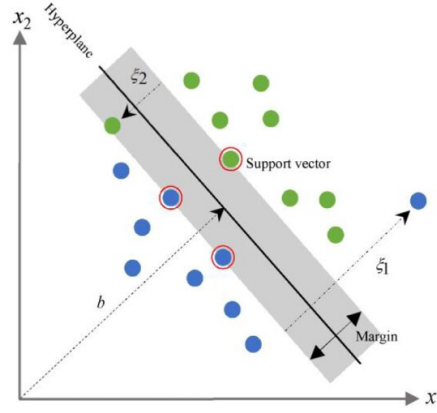


Figure 15: Illustration of support vector machine (SVM) on the observable latent space, with the goal to generalize the optimal separation hyperspace in linear separable data or via a proxy kernel space, as introduced. Reused from Khouma et al. (2019).

where $\alpha_i \in \mathbb{R}$ is a real-valued variable that can be viewed as a *measure* of how much informational value $\mathbf{x}_i$ has (for x_i the support vectors we get from training)¹³. The second idea is of the *kernel method*. This particular method, useful and well-known of the time SVM was created, gain a more prominent position among other techniques, including neural network. Specifically, this method is characterized by the mapping of the input vectors into a richer (usually high-dimensional) feature space where they satisfy *linear separable* criteria. This prompted the possibility to solve nonlinear problem through such mapping, and indeed yields a nonlinear decision surface in the input space, which is linear in the feature space. A **kernel** (function) is then a function

¹²As can be understood simply from such, there exists many hyperplanes that can correctly classifies the classes. It has been then shown that the hyperplane that guarantees the best generalization problem performances is the one with the maximal margin of separation between two classes (Cristianini & Shawe-Taylor (2000)).

¹³The *support vector* of a particular hyperplane $\mathbf{y}$ is the collections of all points that lies closest to the hyperplane, which is specified as condition for maximization on both side of maximal margin vector machine.

$k(\mathbf{x}, \mathbf{y})$ that given two vectors in input space, return the dot product of their images in feature space such that $k(\mathbf{x}, \mathbf{y}) = \langle \phi(x), \phi(y) \rangle$ for the nonlinear mapping ϕ . The general form of SVM is then

$$f(\mathbf{x}) = \text{sgn} \left(\sum_{i=1}^{\ell} y_i \alpha_i k(\mathbf{x} \cdot \mathbf{x}_i) \right) \quad \ell \leq n \quad (63)$$

for any particular final categorization. One of the major problem for analysing SVM, is in the properties of it potentially having infinitely many parameters, depends on the size of the dataset. This makes the curve of both bias-variance and double descent not applicable in such regard - infinitely many parameters' complexity, yet still finite complexity class¹⁴.

9.4 FORWARD-BACKWARD CONSTRUCTION

With respect to the observable space, SVM and neural network, we can call the construction scheme of SVM a type of *backward construction* from data $\mathcal{S}$. Or rather, the following algorithm represents SVM construction problem:

Factory 4: Support Vector Machine - backward construction.

Data: $(x_i, y_i) \in \mathcal{S}, y_i = \{\pm 1\}$, hinge objective $\ell(y, f(x))$, $\nabla(\cdot)$ differentiable gradient field on ℓ .

Result: Expected optimized model to the minimal error on the objective $\ell(\cdot, \cdot)$, dependent on $\mathcal{S}$.

begin

$d \leftarrow \dim |\mathcal{S}|$;

$n \leftarrow |\mathcal{S}|$;

$\ell(y, f(x)) = \max(0, 1 - yf(x))$;

Kernalization $\phi : \mathbf{x} \rightarrow \mathbf{x}'$;

$f(\mathbf{x}) \leftarrow \mathbf{w}^\top \phi(\mathbf{x}) + b$;

Objective

Attain the minimization

$$\min_{w, b \in \mathcal{F}} \frac{1}{2} \|w\|^2 + C \sum_{1 \leq i \leq n} \ell(y_i, f(x_i)) \quad (64)$$

Ideally for $y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) \geq 1, \forall i, y_i \in \mathcal{S}$, penalized by ℓ ;

end

return Optimized model SVM(f) on objective ℓ of ∇ field.

Such a factory is reversed inherently, and with its formulation constrained in the form $\mathbf{w}^\top \phi(\mathbf{x}) + b$, of which is equivalent to a non-linear shallow neural network with infinite width (Wang et al. (2017); Chen et al. (2021)), but is not a full-fledged neural network in technicality, hence it is only a functional equivalence. That is, such result only indicates that for $N \rightarrow \infty$ of the number of neurons of a single-hidden layer neural network, $\sum_{j=1}^N a_j(w_j^\top x + b_j)$, then

$$\text{NN}(x) \approx \text{SVM}(x) = \sum_i^{\ell} \alpha_i k(x, x_i), \ell \leq n \quad (65)$$

The support vector, ℓ list, is inherently implicit of the factory itself. On the other hand, typical deep learning or neural network constructions are rather *forward-constructed*, that is, the model is in a sense, prefabricated then tested of its efficacy, and thus the effective performance of the model is limited by iterations of its restricted frame. We can easily visualize the factory of which neural network are constructed, different of the SVM factory case:

¹⁴From such observation, it would seem imperative that the notion of complexity based solely on the parameter counts of certain model construction is ill-equipped for certain flavours of machine learning model, in this case support vector machine (in the same type is the Gaussian Mixture Model). No double descent has been observed by empirical reports record per the author's knowledge.

9.5 THE COMPONENT PERSPECTIVE

Let us look at neural network in a more component perspective, or rather, similar to a **resource** combination and advancement game. Let us play this game within the following sense. Generalizing neural network means working with component and unit-type framework, since much of our current understanding relies on the basis of hard-coding the flow of operations (feed-forward), external update on global space (backpropagation), fixed domain of construction into horizontal scaffolding and vertical stretching (layers and neurons per such unit). However, to even now of this point, we haven't turned our attention to it of the fullest. In fact, the individual neuron itself is made up of components. If one want a systematic structural analysis, one will have to consider the components' description and metric as it is, before going forward in the interpretation thereof. Furthermore, perhaps there would be a way to link the learning process, right now being interpreted as external of the model, into the system that the model is considered. Such requires a prototype of a new kind of consideration in modelling theory, and as well to the fact that it would be well-supported by many bounds that are usually there in statistical learning theory, which makes this implication a non-trivial yet natural argument.

REFERENCES

- Panos Achlioptas. Stochastic Gradient Descent in Theory and Practice. *Lecture note, Stanford's AI*.
- Ben Adlam and Jeffrey Pennington. Understanding double descent requires a fine-grained bias-variance decomposition, 2020. URL <https://arxiv.org/abs/2011.03321>.
- Madhu S. Advani and Andrew M. Saxe. High-dimensional dynamics of generalization error in neural networks, 2017. URL <https://arxiv.org/abs/1710.03667>.
- Oskar Allerbo. Changing the kernel during training leads to double descent in kernel regression, 2025. URL <https://arxiv.org/abs/2311.01762>.
- Shun-ichi Amari. Neural learning in structured parameter spaces - natural riemannian gradient. In *Advances in Neural Information Processing Systems 9 (NIPS'96)*, pp. 287–293, 1996. Also cited as “Natural Riemannian Gradient”.
- Shun-ichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2):251–276, 1998. doi: 10.1162/089976698300017746.
- Yossi Arjevani, Joan Bruna, Joe Kileel, Elzbieta Polak, and Matthew Trager. Geometry and optimization of shallow polynomial networks, 2025. URL <https://arxiv.org/abs/2501.06074>.
- Maria-Florina Balcan, Steve Hanneke, and Jennifer Wortman Vaughan. The true sample complexity of active learning. *Machine Learning*, 80(2-3):111–139, 2010. doi: 10.1007/s10994-010-5174-y. URL <https://link.springer.com/article/10.1007/s10994-010-5174-y>.
- Enrico Barbierato and Alice Gatti. The challenges of machine learning: A critical review. *Electronics*, 13(2), 2024. ISSN 2079-9292. doi: 10.3390/electronics13020416. URL <https://www.mdpi.com/2079-9292/13/2/416>.
- Pablo Barceló, Mikaël Monet, Jorge Pérez, and Bernardo Subercaseaux. Model interpretability through the lens of computational complexity, 2020. URL <https://arxiv.org/abs/2010.12265>.
- Mikhail Belkin, Siyuan Ma, and Soumik Mandal. To understand deep learning we need to understand kernel learning, 2018. URL <https://arxiv.org/abs/1802.01396>.
- Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine learning practice and the bias-variance trade-off. *Proc. Natl. Acad. Sci. U.S.A.*, 116(32):15849–15854, August 2019. ISSN 0027-8424, 1091-6490. doi: 10.1073/pnas.1903070116. URL <http://arxiv.org/abs/1812.11118>. arXiv:1812.11118 [cs, stat].
- Mikhail Belkin, Daniel Hsu, and Ji Xu. Two models of double descent for weak features. *SIAM Journal on Mathematics of Data Science*, 2(4):1167–1180, January 2020. ISSN 2577-0187. doi: 10.1137/20m1336072. URL <http://dx.doi.org/10.1137/20M1336072>.
- Olivier Bousquet, Steve Hanneke, Shay Moran, Ramon van Handel, and Amir Yehudayoff. A theory of universal learning, 2020. URL <https://arxiv.org/abs/2011.04483>.
- JP. Boyd. Divergence (runge phenomenon) for least-squares polynomial approximation on an equispaced grid and mock-chebyshev subset interpolation. *Applied Mathematics and Computation*, 213(1):292–303, 2009. doi: 10.1016/j.amc.2008.12.087. Study of divergence behavior and mitigation by Mock-Chebyshev grid.
- David Brellmann, Eloïse Berthier, David Filliat, and Goran Frehse. On double descent in reinforcement learning with LSTD and random features. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=9RIbNmX984>.
- Rodney A. Brooks. Intelligence without representation. *Artificial Intelligence*, 47:139–159, 1991. doi: 10.1016/0004-3702(91)90053-M. URL <https://people.csail.mit.edu/brooks/papers/representation.pdf>.
- Gavin Brown and Riccardo Ali. Bias/variance is not the same as approximation/estimation. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=4TnFbv16hK>.

- Augustin-Louis Cauchy. General method for the resolution of simultaneous systems of equations. *Comp. Rend. Sci. Paris*, 25:536–538, 1847.
- Yilan Chen, Wei Huang, Lam M. Nguyen, and Tsui-Wei Weng. On the equivalence between neural network and support vector machine. *arXiv preprint arXiv:2111.06063*, 2021. URL <https://arxiv.org/abs/2111.06063>.
- Vladimir Cherkassky and Eng Hock Lee. To understand double descent, we need to understand vc theory. *Neural Networks*, 169:242–256, 2024. doi: 10.1016/j.neunet.2023.10.014. URL <https://doi.org/10.1016/j.neunet.2023.10.014>.
- Robert M. Corless and Leili Rafiee Seyyeri. The runge example for interpolation and wilkinson’s examples for rootfinding. *arXiv preprint*, 0(arXiv:1804.08561), 2018. URL <https://arxiv.org/abs/1804.08561>. Modern analysis of Runge’s phenomenon and conditioning.
- Nello Cristianini and John Shawe-Taylor. An introduction to support vector machines and other kernel-based learning methods. 2000. URL <https://api.semanticscholar.org/CorpusID:60486887>.
- Stéphane d’ Ascoli, Levent Sagun, and Giulio Biroli. Triple descent and the two kinds of overfitting: where & why do they appear? In *Advances in Neural Information Processing Systems*, volume 33, pp. 3058–3069. Curran Associates, Inc., 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/1fd09c5f59a8ff35d499c0ee25ald47e-Abstract.html>.
- Adnan Darwiche and Pierre Marquis. A knowledge compilation map. *Journal of Artificial Intelligence Research*, 17:229–264, 2002.
- Xander Davies, Lauro Langosco, and David Krueger. Unifying Grokking and Double Descent, March 2023. URL <http://arxiv.org/abs/2303.06173>. arXiv:2303.06173 [cs].
- Howard B. Demuth, Mark H. Beale, Orlando De Jess, and Martin T. Hagan. *Neural Network Design*. Martin Hagan, Stillwater, OK, USA, 2nd edition, 2014. ISBN 0971732116.
- Pedro Domingos. A unified bias-variance decomposition and its applications. In *Proceedings of the 17th International Conference on Machine Learning (ICML-2000)*, pp. 231–238. Morgan Kaufmann, 2000a. URL <https://homes.cs.washington.edu/~pedrod/papers/mlc00a.pdf>.
- Pedro Domingos. A unified bias-variance decomposition for zero-one and squared loss. In *Proceedings of the 17th National Conference on Artificial Intelligence (AAAI-2000)*, pp. 564–569. AAAI Press, 2000b. URL <https://homes.cs.washington.edu/~pedrod/papers/aaai00.pdf>.
- Pedro M. Domingos. A unified bias-variance decomposition for zero-one and squared loss. In *AAAI/IAAI*, 2000c. URL <https://api.semanticscholar.org/CorpusID:2063488>.
- Pedro M. Domingos. A Unifeid Bias-Variance Decomposition and its Applications. In *Semantic Scholar*, June 2000d. URL <https://www.semanticscholar.org/paper/A-Unifeid-Bias-Variance-Decomposition-and-its-Domingos/eled9d24db5e8f7ab326aeb797e965a94f5ad6d3>.
- Finale Doshi-Velez and Been Kim. Towards a rigorous science of interpretable machine learning. In *arXiv preprint arXiv:1702.08608*, 2017.
- Hubert L. Dreyfus. Alchemy and artificial intelligence. Technical Report P-3244, RAND Corporation, 1965. URL <https://www.rand.org/pubs/papers/P3244.html>.
- Hubert L. Dreyfus. *What Computers Can’t Do: A Critique of Artificial Reason*. Harper & Row, 1972. ISBN 0060110821.
- Hubert L. Dreyfus and Stuart E. Dreyfus. *Mind Over Machine: The Power of Human Intuition and Expertise in the Era of the Computer*. Free Press, 1986. ISBN 0029080606.
- George Casella E. L. Lehmann. *Theory of Point Estimation*. Springer Texts in Statistics. Springer-Verlag, New York, 1998. ISBN 978-0-387-98502-2. doi: 10.1007/b98854. URL <http://link.springer.com/10.1007/b98854>.

- Molavi et al. Model Complexity, Expectations, and Asset Prices | The Review of Economic Studies | Oxford Academic. URL <https://academic.oup.com/restud/article-abstract/91/4/2462/7222145?redirectedFrom=fulltext&login=false>.
- R. Fletcher and C. M. Reeves. Function minimization by conjugate gradients. *The Computer Journal*, 7(2): 149–154, 1964. doi: 10.1093/comjnl/7.2.149.
- Stuart Geman, Elie Bienenstock, and René Doursat. Neural networks and the bias/variance dilemma. *Neural Computation*, 4(1):1–58, 1992. doi: 10.1162/neco.1992.4.1.1.
- Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. *Deep learning*, volume 1. MIT Press, 2016.
- Suriya Gunasekar, Blake Woodworth, and Nathan Srebro. Mirrorless mirror descent: A natural derivation of mirror descent. In *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics (AISTATS 2021)*, volume 130, pp. 2305–2313. PMLR, 2021.
- Bruce Hajek and Maxim Raginsky. *Statistical Learning Theory*, volume 1. 2021. URL <https://maxim.ece.illinois.edu/teaching/SLT/>.
- Stevan Harnad. The symbol grounding problem. *Physica D: Nonlinear Phenomena*, 42:335–346, 1990. doi: 10.1016/0167-2789(90)90087-6.
- Ouns El Harzli, Bernardo Cuenca Grau, Guillermo Valle-Pérez, and Ard A. Louis. Double-descent curves in neural networks: a new perspective using gaussian processes, 2023. URL <https://arxiv.org/abs/2102.07238>.
- Trevor Hastie, Andrea Montanari, Saharon Rosset, and Ryan J Tibshirani. Surprises in high-dimensional ridgeless least squares interpolation. *arXiv preprint arXiv:1903.08560*, 2019.
- Rebekka Heckel and Murat Yilmaz. Early stopping in deep networks: Double descent and how to eliminate it. *arXiv preprint*, 2020. URL <https://arxiv.org/abs/2007.10099>. discusses epoch-wise double descent and mitigation strategies.
- Magnus R. Hestenes and Eduard Stiefel. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49(6):409–436, 1952. doi: 10.6028/jres.049.044.
- Erdong Hu. An overview of double descent and overparameterization. *independent*, (0), 2021.
- Xia Hu, Lingyang Chu, Jian Pei, Weiqing Liu, and Jiang Bian. Model complexity of deep learning: A survey, 2021a. URL <https://arxiv.org/abs/2103.05127>.
- Xia Hu, Lingyang Chu, Jian Pei, Weiqing Liu, and Jiang Bian. Model Complexity of Deep Learning: A Survey, August 2021b. URL <http://arxiv.org/abs/2103.05127>. arXiv:2103.05127 [cs].
- Baihe Huang, Shanda Li, Tianhao Wu, Yiming Yang, Ameet Talwalkar, Kannan Ramchandran, Michael I. Jordan, and Jiantao Jiao. Sample complexity and representation ability of test-time scaling paradigms, 2025. URL <https://arxiv.org/abs/2506.05295>.
- Arthur Jacot, François Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. Kernel view of wide-network behavior.
- Gareth James, Trevor Hastie, Robert Tibshirani, and Daniela Witten. *An introduction to statistical learning : with applications in R*. New York : Springer, [2013] ©2013, 2013. URL <https://search.library.wisc.edu/catalog/9910207152902121>.
- Romuald A. Janik and Przemek Witaszczyk. Complexity for deep neural networks and other characteristics of deep feature representations, 2021. URL <https://arxiv.org/abs/2006.04791>.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. URL <https://arxiv.org/abs/2001.08361>.

- Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in ml-based science. *arXiv preprint arXiv:2207.07048*, 2022.
- Michael J. Kearns and Umesh V. Vazirani. *An introduction to computational learning theory*. MIT Press, Cambridge, MA, USA, 1994. ISBN 0262111934.
- Ousmane Khouma, Idy Diop, Papa Alioune Fall, Mamadou Lamine Ndiaye, Sidi M. Farssi, Abdou M. Oussamatou, and Birahime Diouf. Novel classification method of spikes morphology in eeg signal using machine learning. *Procedia Computer Science*, 148:70–79, 2019. doi: 10.1016/j.procs.2019.01.010.
- Marc Lafon and Alexandre Thomas. Understanding the Double Descent Phenomenon in Deep Learning, March 2024. URL <http://arxiv.org/abs/2403.10459>. arXiv:2403.10459 [cs, stat].
- Eng Hock Lee and Vladimir Cherkassky. Vc theoretical explanation of double descent, 2022. URL <https://arxiv.org/abs/2205.15549>.
- Xinyue Li and Rishi Sonthalia. Least squares regression can exhibit under-parameterized double descent. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 25510–25560. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/2d43f7a61b57f83619f82c971e4bddc0-Paper-Conference.pdf.
- Zachary C. Lipton. The mythos of model interpretability. *Queue*, 16(3):31–57, 2018.
- Zachary C. Lipton and Jacob Steinhardt. Troubling trends in machine learning scholarship, 2018. URL <https://arxiv.org/abs/1807.03341>.
- Chris Yuhao Liu and Jeffrey Flanigan. Understanding the role of optimization in double descent, 2023. URL <https://arxiv.org/abs/2312.03951>.
- Fanghui Liu, Zhenyu Liao, and Johan A. K. Suykens. Kernel regression in high dimensions: Refined analysis beyond double descent, 2021. URL <https://arxiv.org/abs/2010.02681>.
- Jing Luo, Huiyuan Wang, and Weiran Huang. Investigating the impact of model complexity in large language models, 2024. URL <https://arxiv.org/abs/2410.00699>.
- Yifan Luo and Bin Dong. Double descent of discrepancy: A task-, data-, and model-agnostic phenomenon, 2023. URL <https://arxiv.org/abs/2305.15907>.
- Duy V. N. Luong, Panos Parpas, Daniel Rueckert, and Berç Rustem. A weighted mirror descent algorithm for nonsmooth convex optimization problem. *Journal of Optimization Theory and Applications*, 170(3): 900-915, 2016. doi: 10.1007/s10957-016-0963-5.
- Yuri Manin and Matilde Marcolli. Homotopy theoretic and categorical models of neural information networks. *Compositionality*, Volume 6 (2024), September 2024. ISSN 2631-4444. doi: 10.46298/compositionality-6-4. URL <http://dx.doi.org/10.46298/compositionality-6-4>.
- Gary Marcus. Deep learning: A critical appraisal. *arXiv preprint arXiv:1801.00631*, 2018. URL <https://arxiv.org/abs/1801.00631>.
- John McCarthy and Patrick J. Hayes. Some philosophical problems from the standpoint of artificial intelligence. In B. Meltzer and D. Michie (eds.), *Machine Intelligence 4*, pp. 463–502. Edinburgh University Press, 1969.
- Shibin Mei, Chenglong Zhao, Shengchao Yuan, and Bingbing Ni. Towards bridging sample complexity and model capacity. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36 of *AAAI’22 Technical Tracks*, pp. 1972–1980, June 2022. doi: 10.1609/aaai.v36i2.20092.
- Song Mei and Andrea Montanari. The generalization error of random features regression: Precise asymptotics and double descent curve. *arXiv preprint arXiv:1908.05355*, 2019a.
- Song Mei and Andrea Montanari. The generalization error of random features regression: Precise asymptotics and double descent curve. *arXiv preprint*, 2019b. URL <https://arxiv.org/abs/1908.05355>.
- Song Mei and Andrea Montanari. The generalization error of random features regression: Precise asymptotics and double descent curve, 2020. URL <https://arxiv.org/abs/1908.05355>.

- Peter B. Miltersen, Jaikumar Radhakrishnan, and Ingo Wegener. On converting CNF to DNF. *Theoretical Computer Science*, 347(1-2):325–335, 2005.
- Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning*. The MIT Press, 2012. ISBN 026201825X.
- Christoph Molnar, Giuseppe Casalicchio, and Bernd Bischl. *Quantifying Model Complexity via Functional Decomposition for Better Post-hoc Interpretability*, pp. 193–204. Springer International Publishing, 2020a. ISBN 9783030438234. doi: 10.1007/978-3-030-43823-4_17. URL http://dx.doi.org/10.1007/978-3-030-43823-4_17.
- Christoph Molnar, Gunnar König, Julia Herbringer, Timo Freiesleben, Susanne Dandl, Christian A. Scholbeck, Giuseppe Casalicchio, Moritz Grosse-Wentrup, and Bernd Bischl. General pitfalls of model-agnostic interpretation methods for machine learning models. *arXiv preprint arXiv:2007.04131*, 2020b.
- Preetum Nakkiran. More data can hurt for linear regression: Sample-wise double descent, 2019a. URL <https://arxiv.org/abs/1912.07242>.
- Preetum Nakkiran. More data can hurt for linear regression: Sample-wise double descent. arXiv:1912.07242, 2019b. URL <https://arxiv.org/abs/1912.07242>. arXiv preprint.
- Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep Double Descent: Where Bigger Models and More Data Hurt, December 2019. URL <http://arxiv.org/abs/1912.02292>. arXiv:1912.02292 [cs, stat].
- Preetum Nakkiran, Prayaag Venkat, Sham Kakade, and Tengyu Ma. Optimal regularization can mitigate double descent (openreview entry). OpenReview forum submission, 2020. URL <https://openreview.net/forum?id=7R7fAoUygoa>.
- Preetum Nakkiran, Prayaag Venkat, Sham Kakade, and Tengyu Ma. Optimal regularization can mitigate double descent, 2021. URL <https://arxiv.org/abs/2003.01897>.
- Brady Neal. On the bias-variance tradeoff: Textbooks need an update, 2019. URL <https://arxiv.org/abs/1912.08286>.
- Brady Neal, Sarthak Mittal, Aristide Baratin, Vinayak Tantia, Matthew Scicluna, Simon Lacoste-Julien, and Ioannis Mitliagkas. A modern take on the bias-variance tradeoff in neural networks. *arXiv preprint arXiv:1810.08591*, 2018.
- Arkadii S. Nemirovski and David B. Yudin. *Problem Complexity and Method Efficiency in Optimization*. Wiley-Interscience Series in Discrete Mathematics. John Wiley & Sons, Chichester, UK, 1983. ISBN 9780471103455.
- Jiawang Nie, Zhi Yang, and Guoyin Zhou. The saddle point problem of polynomials. *Foundations of Computational Mathematics*, 22(5):1133–1169, 2021. doi: 10.1007/s10208-021-09526-8. URL <https://link.springer.com/article/10.1007/s10208-021-09526-8>.
- Amanda Olmin and Fredrik Lindsten. Towards understanding epoch-wise double descent in two-layer linear neural networks, 2024. URL <https://arxiv.org/abs/2407.09845>.
- Liam Paninski. Statistics 4107: Intro to Math Stat (fall 2005), 2005. URL <https://sites.stat.columbia.edu/liam/teaching/4107-fall05/>.
- Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009. ISBN 9780521895606. doi: 10.1017/CBO9780511803161.
- Mohammad Pezeshki, Amartya Mitra, Yoshua Bengio, and Guillaume Lajoie. Multi-scale feature learning dynamics: Insights for double descent, 2021. URL <https://arxiv.org/abs/2112.03215>.
- David Pfau. A generalized bias-variance decomposition for bregman divergences. Technical report, 2013.
- Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets, 2022. URL <https://arxiv.org/abs/2201.02177>.

- Victor Quétu and Enzo Tartaglione. Can we avoid Double Descent in Deep Neural Networks? In *2023 IEEE International Conference on Image Processing (ICIP)*, pp. 1625–1629, October 2023a. doi: 10.1109/ICIP49359.2023.10222624. URL <http://arxiv.org/abs/2302.13259>. arXiv:2302.13259 [cs].
- Victor Quétu and Enzo Tartaglione. Can we avoid Double Descent in Deep Neural Networks? In *2023 IEEE International Conference on Image Processing (ICIP)*, pp. 1625–1629, October 2023b. doi: 10.1109/ICIP49359.2023.10222624. URL <http://arxiv.org/abs/2302.13259>. arXiv:2302.13259 [cs].
- Garvesh Raskutti and Sayan Mukherjee. The information geometry of mirror descent. *arXiv Preprint*, 2013. arXiv:1310.7780.
- M. Rattray, D. Saad, and Shun-ichi Amari. Natural gradient descent for on-line learning. *Physical Review Letters*, 81:5461–5464, 1998. doi: 10.1103/PhysRevLett.81.5461.
- Herbert Robbins and Sutton Monro. A stochastic approximation method. *Annals of Mathematical Statistics*, 22(3):400–407, 1951. doi: 10.1214/aoms/1177729586.
- Paul M. Romer. Mathiness in the theory of economic growth. *American Economic Review*, 105(5):89–93, 2015.
- Sebastian Ruder. An overview of gradient descent optimization algorithms, June 2017. URL <http://arxiv.org/abs/1609.04747>. arXiv:1609.04747 [cs].
- Carl Runge. Über empirische funktionen und die interpolation zwischen äquidistanten ordinaten. *Zeitschrift für Mathematik und Physik*, 46:224–243, 1901. The original Runge example showing divergence with equispaced interpolation.
- Rylan Schaeffer, Mikail Khona, Zachary Robertson, Akhilan Boopathy, Kateryna Pistunova, Jason W. Rocks, Ila Rani Fiete, and Oluwasanmi Koyejo. Double Descent Demystified: Identifying, Interpreting & Ablating the Sources of a Deep Learning Puzzle, March 2023. URL <http://arxiv.org/abs/2303.14151>. arXiv:2303.14151 [cs, stat].
- John R. Searle. Minds, brains, and programs. *Behavioral and Brain Sciences*, 3(3):417–457, 1980.
- Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, USA, 2014. ISBN 1107057132.
- Shai Shalev-Shwartz, Ohad Shamir, Nathan Srebro, and Karthik Sridharan. Learnability, stability and uniform convergence. *Journal of Machine Learning Research*, 11(90):2635–2670, 2010. URL <http://jmlr.org/papers/v11/shalev-shwartz10a.html>.
- Rahul Sharma and Alex Aiken. Bias-variance tradeoffs in program analysis. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL ’14*, pp. 127–137, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 978-1-4503-2544-8. doi: 10.1145/2535838.2535853. URL <https://doi.org/10.1145/2535838.2535853>.
- Zewen Shen and Kirill Serkh. On polynomial interpolation in the monomial basis. *SIAM Journal on Numerical Analysis*, 63(2):469–494, March 2025. ISSN 1095-7170. doi: 10.1137/23m1623215. URL <http://dx.doi.org/10.1137/23M1623215>.
- Cheng Shi, Liming Pan, Hong Hu, and Ivan Dokmanić. Homophily modulates double descent generalization in graph convolution networks, 2024. URL <https://arxiv.org/abs/2212.13069>.
- Jascha Sohl-Dickstein. The natural gradient by analogy to signal whitening, and recipes and tricks for its use. *arXiv Preprint*, 2012. arXiv:1205.1828.
- Daniel Soudry, Elad Hoffer, Mor Shpigel Nacson, Suriya Gunasekar, and Nathan Srebro. The implicit bias of gradient descent on separable data, 2024. URL <https://arxiv.org/abs/1710.10345>.
- Jann Spiess, Guido Imbens, and Amar Venugopal. Double and single descent in causal inference with an application to high-dimensional synthetic control, 2023. URL <https://arxiv.org/abs/2305.00700>.

- Tom F. Sterkenburg. Statistical learning theory and occam's razor: The core argument. *Minds and Machines*, 35(1), November 2024. ISSN 1572-8641. doi: 10.1007/s11023-024-09703-y. URL <http://dx.doi.org/10.1007/s11023-024-09703-y>.
- Lucy A. Suchman. *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge University Press, 1987. ISBN 0521388473.
- Masashi Sugiyama. *Introduction to Statistical Machine Learning*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2015. ISBN 9780128023501.
- Richard S. Sutton. The bitter lesson. Web essay / blog post, 2019. URL <https://www.incompleteideas.net/IncIdeas/BitterLesson.html>.
- Lars Pålsson Syll. Post-real economics — a severe case of mathiness. Blog post, Heterodox Economic Blogs, 2024.
- Mark K. Transtrum, Gus L. W. Hart, Tyler J. Jarvis, and Jared P. Whitehead. egad! double descent is explained by generalized aliasing decomposition, 2025. URL <https://arxiv.org/abs/2408.08294>.
- Lan V. Truong. On rademacher complexity-based generalization bounds for deep learning, 2025. URL <https://arxiv.org/abs/2208.04284>.
- L. G. Valiant. A theory of the learnable. *Commun. ACM*, 27(11):1134–1142, November 1984. ISSN 0001-0782. doi: 10.1145/1968.1972. URL <https://doi.org/10.1145/1968.1972>.
- Gido M. van de Ven, Nicholas Soures, and Dhireesha Kudithipudi. Continual learning and catastrophic forgetting, 2024. URL <https://arxiv.org/abs/2403.05175>.
- Vladimir Vapnik. *The Nature of Statistical Learning Theory*. Springer: New York, 1999.
- Huibing Wang, Jinbo Xiong, Zhiqiang Yao, Mingwei Lin, and Jun Ren. Research survey on support vector machine. In *Proceedings of the 10th EAI International Conference on Mobile Multimedia Communications (MOBIMEDIA)*. EAI, December 2017. doi: 10.4108/eai.13-7-2017.2270596.
- Jingyuan Wang, Kai Feng, and Junjie Wu. Svm-based deep stacking networks. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI-19)*, pp. 5262–5268, 2019. doi: 10.24963/ijcai.2019/731. URL <https://www.ijcai.org/proceedings/2019/731>.
- Tobias Wegel, Geelon So, Junhyung Park, and Fanny Yang. On the sample complexity of semi-supervised multi-objective learning, 2025. URL <https://arxiv.org/abs/2508.17152>.
- Ingo Wegener. *The Complexity of Boolean Functions*. John Wiley & Sons, Chichester, UK, 1987.
- Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models, 2022. URL <https://arxiv.org/abs/2206.07682>.
- Bernard Widrow and Marcian E. Hoff. Adaptive switching circuits. In *1960 IRE WESCON Convention Record, Part 4*, pp. 96–104, New York, 1960. Institute of Radio Engineers.
- Lin Xiao and Tong Zhang. A proximal stochastic gradient method with progressive variance reduction. *SIAM Journal on Optimization*, 24(4):2057–2075, 2014. doi: 10.1137/140961791.
- H. H. Yang and Shun-ichi Amari. Complexity issues in natural gradient descent method for training multilayer perceptrons. *Neural Computation*, 10(8):2137–2157, 1998. doi: 10.1162/089976698300017007.
- Tian-Le Yang and Joe Suzuki. Dropout drops double descent. *Japanese Journal of Statistics and Data Science*, 7(2):615–632, March 2024. ISSN 2520-8764. doi: 10.1007/s42081-024-00242-5. URL <http://dx.doi.org/10.1007/s42081-024-00242-5>.
- Zitong Yang, Yaodong Yu, Chong You, Jacob Steinhardt, and Yi Ma. Rethinking Bias-Variance Trade-off for Generalization of Neural Networks, December 2020. URL <http://arxiv.org/abs/2002.11328>. arXiv:2002.11328 [cs, stat].

Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola. Dive into deep learning, 2023. URL <https://arxiv.org/abs/2106.11342>.

Jiawei Zhang. Gradient Descent based Optimization Algorithms for Deep Learning Models Training, March 2019. URL <http://arxiv.org/abs/1903.03614>. arXiv:1903.03614 [cs].

Rui Zhang and Hong Wang. The equivalence relationship between kernel functions based on svm and four-layer functional networks. In De-Shuang Huang, Vito Bevilacqua, Prashan Premaratne, and Praveen Gupta (eds.), *Intelligent Computing Methodologies*, volume 8589 of *Lecture Notes in Computer Science*, pp. 81–89. Springer, 2014. doi: 10.1007/978-3-319-09339-0_10. URL https://doi.org/10.1007/978-3-319-09339-0_10.

Tong Zhang. *Mathematical Analysis of Machine Learning Algorithms*. Cambridge University Press, 2023. doi: 10.1017/9781009093057. URL <https://www.cambridge.org/core/books/mathematical-analysis-of-machine-learning-algorithms/EB9BABB05A5C312F19C38E5A01A5ECFC>.

Ya Shi Zhang. Manipulating sparse double descent, 2024. URL <https://arxiv.org/abs/2401.10686>.

APPENDIX A. VERIFICATION OF THEOREM 31

To verify the theorem itself, we test it with a standard setting without the theorem results. This is reflected in Figure 7. The result is somewhat consistent with the theorem and its latter parameter error and prediction error results, however, this time, it correlates exactly as the prediction error instead. Additionally, peculiarity occurs in the non-double descent region, with much less volatility than in the prediction of the theorem.

Algorithm 5: Linear regression with Gaussian white noise, full control.

Data: n, p, d, σ, r , fixed range $[a, b]$, $u \in [0, 1]$

Result: Threefold error measure - MSE on parameters p , d , MSE on prediction Y' , Y , and theoretical risks measure by prescription.

begin

```

Randomizer( $\cdot$ )  $\leftarrow$  PCG64( $n \leq 100$ );           Randomizing pseudo-random engine
 $\sigma \leftarrow 0.5, d \leftarrow 150$ ;           Common presetting for parameters
 $nl \leftarrow \{12, 24, 36, 50, 76, 82, 90, 100, 106, 112, 121, 150, 231, 250, 256, 280\}$ ;   Common
    full-list  $n$  count of datapoints
 $p \leftarrow 150$ ;                               Underparameterization to equal
 $p \leftarrow \text{range}(1, p)$ ;
 $[a, b] \leftarrow [-100, 100]$ ;
 $w_c \sim \text{PCG64.Uniform}([1, d], 150)$ ;
 $b_c \sim \text{PCG64.Uniform}([1, 10])$ ;
 $w_p \sim \text{PCG64.Uniform}([1, p], p)$ ;
 $b_p \sim \text{PCG64.Uniform}([1, 10])$ ;
 $X \sim \text{PCG64.Uniform}(\text{range} \leftarrow [a, b], \text{size} \leftarrow [n, d])$ ;    $n$  samples of  $d$  dimensions.
 $\text{concept}(\cdot, \cdot, \cdot) \leftarrow (d, w_c, b_c)$ ;
 $Y \leftarrow \text{concept}(X, w_c, b_c)$ ;
 $\sigma_{\text{scaled}} \leftarrow \sigma \cdot \mathbb{E}[|Y|]$ ;
 $Y \leftarrow Y + \text{PCG64} \dots \text{Normal}(\mu \leftarrow 0, \sigma \leftarrow \sigma_{\text{scaled}}, \text{size} \leftarrow |Y|)$ ;   Noisy featuring
for  $n \in nl$  do
     $w_p^* \leftarrow \text{PCG64.Permutation}(d, p)$ ;
    for  $i \leftarrow 1$  to  $|w_p|$  do
        if  $i \neq w_p^*$  then
             $w_p[i] \leftarrow 0$ , fixed zero point.
        end
    end
     $\text{model}(\cdot, \cdot, \cdot) \leftarrow (d, w_p, b_p)$ ;
     $Y \leftarrow \text{model}(X, w_p, b_p)$ ;
     $\text{approx} \leftarrow \text{LeastSquare}(\text{model}, Y', X, Y)$ , fixed  $w_p$ .
end
 $\text{MSE}_{\text{param}}(\cdot, \cdot, \cdot) \leftarrow \text{MSE}(Y, \{\text{approx}\}, \text{model})$ ;
 $\text{MSE}_{\text{pred}}(\cdot, \cdot) = \text{MSE}(Y, Y') \hat{R}_{\text{Thm}} = \text{TR}(w_c, \sigma, p, d, n)$ ;

```

end

return $\text{MSE}_{\text{param}}, \text{MSE}_{\text{pred}}, \hat{R}_{\text{Thm}}$

We use the Algorithm 5 for verification, including three modes of error measure, which will then be calculated accordingly. For gradient descent, we refer to another experiment. For such algorithm, we notice that the operational space of the model is $\mathbb{R}^n \rightarrow \mathbb{R}$. The masking of parameters then is not well-received, since the output space is far more skewed - single dimension resultant with no scaling difference because the model in question is supposed to be linear. Regarding such, it makes sense to realize that the model itself can still attain relatively stable error, because its feature compounds on the scaling parameters

To test the stability of the model, we run for a stable section of the permutations done on p . The result is outlined in 7. Interestingly, while the interpolation threshold theorized being $p = n$ does not hold, the pattern of double descent without head descent still fundamentally exists, albeit in a much more constrained region, as can see from $n = 12$ to $n = 150$. The pattern from then onward follows a reduced trend, without the chaotic behaviours seen. The result is shown in Figure 16.

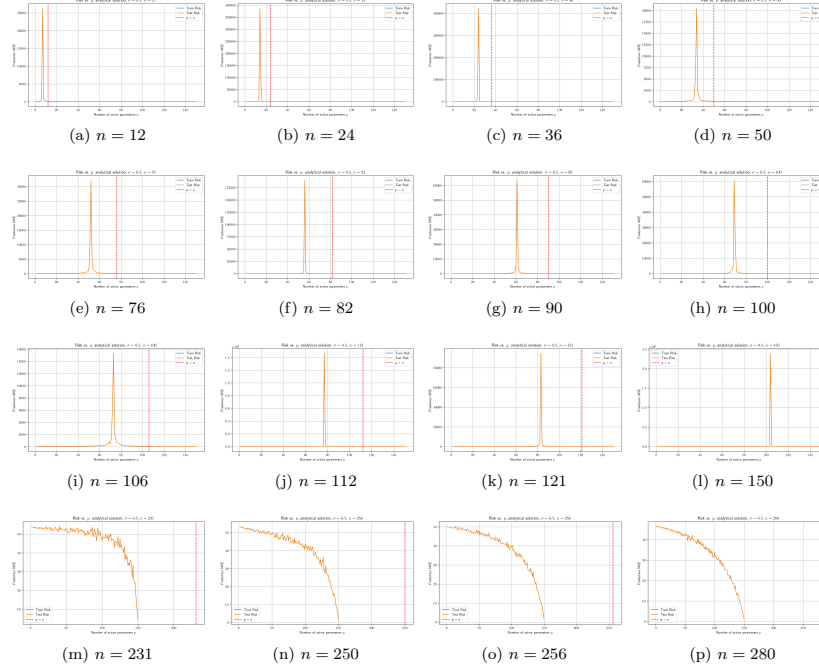


Figure 16: Risk curves for varying sample size n (dimension $d = 100$, noise $\sigma = 0.5$). Double descent is determined in similar sequence, even though the interpolation theoretical $p = n$ is actually shifted over to the right from the actual interpolation observables, in parameterization. Afterward, correlation fails, and the theoretical p is ineffective, similar to previous theorem-based test run. Sample set size range is $n \in \{12, 24, 36, 50, 76, 82, 90, 100, 106, 112, 121, 150, 231, 250, 256, 280\}$. One additional remark is that the error landscape is relatively thin in either side, making it abnormal in comparison to actual bias-variance curvature.

.0.1 REMARK

At least from this experiment, overall, results gathered using Theorem 8.1 and variations of it suggests inconsistent results. The supposed interpolation threshold has different interpretation and indication in different scenario, including both parameter-wise error and prediction-only error measure. Furthermore, verification also shows inconsistency with the interpolation threshold, albeit given the fact that the margin of error from the highest peak to the interpolation point is consistent up to certain n . Breakdown of the result from theorem, as well as double descent behaviour starts with high n counts, as well as interesting outlook of bias-variance reappearance for $p \sim 5000$, far more than what is tested in normal testing cases. Such result suggests further experiments to validate the observations, as well as statistical analysis of the entire system.

Specifically, the interpretation of the result is not perfect. Under such setting, then for the concept of the same class, so $d = 150$, any concept of $p > d$ would result in the metric taking zero value of the ‘truncated region’ where d is literally flat on the $d + 1$ onward space. Hence, the error being added on top of such error measure is the component distance of p to the flat sub $p - d$ space itself - which explains the curve going upward, as more and more risks are configured. Nevertheless, the amount of data provided, at least in terms of this model, does indeed have non-trivial effect on the model itself, and the least square solution of best fit which gives the potential minimal, hence the empirical best. Considering the results, we can see regions where the theorem ‘breaks’ of its bias-variance pattern. Further experiment to verify such claim and reconfiguration is needed. Worryingly, in this particular case, there exists no saddle point that was theorized by bias-variance tradeoff, which must then be explained accordingly. Furthermore, we also notice the troublesome fact that this particular pattern works on *same operational space* models, that is, linear models approximating linear models. This setting is fairly limited, and would not generalize well. However, we also do not know what kind of measure to take in such case. Conflicting notions and reports aid into the confusion of such topic more than can be normally observed, and the anomalous behaviours in some researches (for example, in Shi et al. (2024), we received that in GNN, there exists no trace of double descent) makes it even harder. As such, both the status of

bias-variance being false, albeit in a given range only, and double descent in masking certain behaviours of the modelling process, have taken a mysterious stance in the modern machine learning community.

While discovering the double descent phenomena, it also exposes theoretical concepts that are not organized or formalized, portions of the theory and empirical observations that are within conflict of each others, issues with definitions, theorems, and interpretation of them. Those problems would then ultimately hinder further research in such direction, as well as broader theoretical requirement of the theory of machine learning, and specifically to a larger and more complex system of deep learning.

APPENDIX A.1. STRESS TESTING SCHEMA

We introduce the concept of **stress testing**, at least in this paper setting of which we are countering at.

APPENDIX B. A WAY OF DEFINING REPRESENTATION SCHEME

Nevertheless, how to measure this complexity? We are not so sure how, but we can empirical formulate one, but will be fairly difficult to translate this to, since the representation complexity is often encoded in the sense of binary sequence or very old consideration of learning process of Kearns & Vazirani (1994). Assuming abstraction of the bits/byte practical mathematical representation of numerical encoding (of which we will have to talk about later), we present the definition for the *structural representation scheme* (basically a more applicable definition)

Definition .3 (Representation scheme, structural definition). *For a system and its object, an object's **representation scheme** $\mathcal{R}$ is a function $\mathcal{R} : (\Sigma \cup \mathcal{G})^* \rightarrow \mathcal{O}$, where Σ is the operator alphabet, or relational structure, $\mathcal{G}$ are the components, and $\mathcal{O}$ is the object. $(\Sigma \cup \mathcal{O})^*$ is the coupling of **all** configuration of the components and the operators.*

In case where scalar numbers are presented, for example, in the axis-aligned rectangle case, we will have: $\mathcal{R}_{\square}^2 : (\Sigma \cup \mathcal{G} \cup \mathbb{F})^* \rightarrow \text{Rec of all rectangle using this representation scheme, so}$

$$\mathcal{R}_{\square} = \begin{cases} \Sigma = \{\leq, \wedge\} \\ \mathcal{G} = \{l_c, p_c, a\} \\ \mathbb{F} = \mathbb{R} \end{cases} \quad (66)$$

For example, of the formula of all concepts c being axis-aligned rectangle, we can have the representation scheme in general as:

$$\sigma = \{c\} = \mathcal{R}^2 = \{a(x_a, y_a) \mid l_c^{(1)} \leq x_a \leq p_c^{(1)} \wedge l_c^{(2)} \leq x_y \leq p_c^{(2)}\} \quad (67)$$

Generally, this is called the *numerical representation scheme*, in which it is supported by a field (we do not care much about the dimension of the field, as long as we can decompose it to discrete scalar taking values), that is, $\mathcal{R} : (\Sigma \cup \mathcal{G} \cup \mathcal{F}) \rightarrow \mathcal{O}$. Any representation specification $\sigma \in (\Sigma \cup \mathcal{G} \cup \mathcal{F})$ such that $\mathcal{R}(\sigma) = c$ is a representation of c on $\mathcal{R}$, denoted σ_c , and the set of all such specification $R_c = \{\sigma_c\}$ is called the **representation space** of c on $\mathcal{R}$.

The set of all hypotheses h that is specified by certain representation scheme $\mathcal{R}_h$ is called the *hypothesis class* $\mathcal{H}$, and $\mathcal{R}_h$ is the *hypothesis class representation*. Similarly, for a concept c the set of all concepts that are represented by $\mathcal{R}_c$ is called the *concept class* $\mathcal{C}$ for the *concept representation scheme* $\mathcal{R}_c$.

The necessity of the description of a field is rather natural, especially since we are working on a numerical encoding. Generally speaking, the computer at large represents a very complex binary encoding of numerical logical operations, though via abstractions, most of them are 'cancelled out' of the fundamental details. We can almost make a *representation space* just similar to how we define vector space, though, it is more or less not so effective as it can. Using this, we can specify quite a few object classes in the same way. The first one though, we can specify the representation scheme of an input-output model. Now, for this type of definition, then the class of all **linear function representation** can be designed as

$$\mathcal{R}_L^n = \left\{ \{x_1, \dots, x_n\}, y \mid \sum_{i=1}^n w_i x_i + b \wedge w_i, x_i, b \in \mathbb{R} \right\} \quad (68)$$

From this, we can notice that there exists the notion of *size* for the object class. In one way or another, we have abstracted of the class of all objects that can be specified in such a way that their representation falls into the range of such representation class. Then, we might want to consider the concept of a **representation**

complexity, and the size of the class, denoted $size(\mathcal{R})$. Do note that this is not defining the operational complexity, but simply the structure by itself. Which is why we might want to have the definition of an object in such system that we are considering.

Two representations $\mathcal{R}_A$ and $\mathcal{R}_B$, they are said to be **equivalent** if one can convert objects from the first representation to the second one, and vice versa. Then two representations are said to be **equal** if their size is the same, assume that the representation scheme is equivalent. This prompts us to define the notion of the size of the representation, but before that, a general insight will be the follow through - you have to be able to reduce one to the others, and reverse. By then, essentially, *you cannot compare apple to orange*, that is. Generally, the size of a representation class is defined the smallest representation of the object in the underlying representation. So, for your example of the linear function, then

$$size(c) = \min_{\sigma \in R_c} \{size(\sigma)\} \quad (69)$$

which again, prompt us *again*, to figure out the notion for $size(\sigma)$. Before then, and defining the object's descriptions, let's try to construct a few more 'mathematical construct' of the same type. A subtle remark can be made here, that the representation only, again, specify the parameters used to represent it, and the string accompanied by such representation. The overall total repetition, for example, of a certain variable, and operation succeeding, is totally irrelevant. I just get the formula as a shorthand for that scheme up there, as it is. Which again, means that to specify it, you need both the components and how it is connected. Abstractly speaking, and generally speaking so.

A fundamental flaw in such configuration is that for something like *data-dependent representation*, one might say that its representational complexity is infinite, which is, at least under such definition, totally true. Nevertheless, we shall then see, that even under such situation, the main mass of which is 'representative' of the concept would be finite, for there are noisy and dense data in which does not add more information than not to the concept class.

We can examine for now the representation complexity up to operation-level as defined for polynomial space.

APPENDIX C - EXPERIMENTAL SETTING

Aside from the usual setting that was settled above preliminary sections, to contain this experiment, formalism in the experimenting concept is needed. For now, loose speaking and loosely contriving such, this experiment section and further experiment sections will follow the same design template, which is illustrated in Figure 17. Unless explicitly said so, we conform to this template.

There are certainly a lot of things to talk about of, at least in conclusion of this particular structure. Explicitly, now we can illustrate partially, where each component of the complexity measure might measure, kind of learning criteria that bounds the learning setting, depends on the cofactors it receives. Firstly, let's see the PAC-learning criteria. The PAC-learning criteria specifically target the sample set, the supervising error, and the assumed structure of the concept class. Hence, we can see that there exists three main components, indicated in Figure ???. A lot of details are missing, which is typical of the setting by itself.

APPENDIX D - STATISTICAL LEARNING THEORY PERSPECTIVE

By statistical learning theory, we observed that there are plenty tradeoffs, most notably the empirical-generalization tradeoff, though we would have to clarify what those terms are. Furthermore, there is also what can be seen to be the analogue bias-variance tradeoff in such sense, is the *approximation-estimation tradeoff* of statistical learning theory, though it must be clarified that they are strictly not the same, as seen from Brown & Ali (2024).

Informally, the empirical-generalization tradeoff can be said to be the balancing act between achieving the true generalization and the empirical best. To achieve the generalization best, the empirical best, for finite hypothesis $\mathcal{H}$ (Mohri et al. (2012)),

$$\lim_{n \rightarrow \infty} \left[\mathbb{E}_{\mathcal{S}_n \sim \mathcal{D}^m} (\hat{R}_{\mathcal{S}(h)}) \right] = R(h), \quad h \in \mathcal{H} \quad (70)$$

This follows by the law of large number, and the statistical-probabilistic interpretation of the learning setting. However, the tradeoff is not straightforward - it is rather considered between, for example, the time-complexity, sample-space size, and generalization best's reach of the model. That is, for large the dataset is, the easier it is to get a sample space that represents more and more of the actual concept. However, in exchange, time-complexity increases, and thereby, the ergonomic aspect must be also considered. It is also concerned of the fact that the empirical best is not the same as even the generalization best, thereby, one will want their

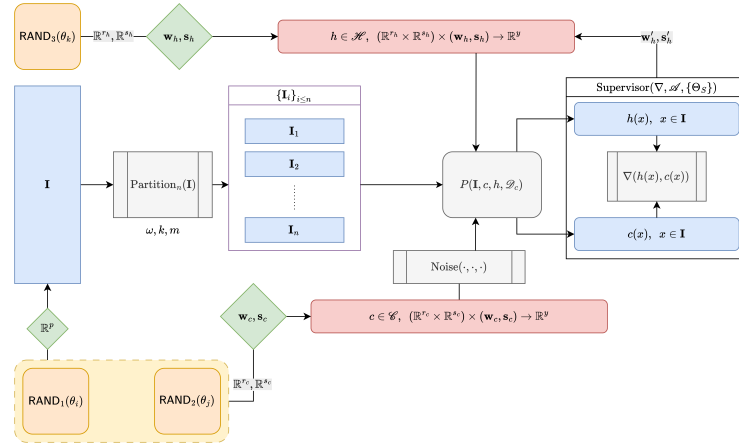


Figure 17: **Diagrammatic view for the ambiance space of the modelling scenario setting.** Under classical learning theory and consideration, such scheme is used for almost every aspect possible of the learner's action. $c \in \mathcal{C}, h \in \mathcal{H}$ described by the tuple $(\mathbf{w}, \mathbf{s})$ of main (weight) parameters and special parameters (for example, bias), $\mathbf{I}$ as the input space. The 3-tuple (ω, k, m) is used for controlling the partitioning (for $k = 2$ is the train-test split). Others include $P(\mathbf{I}, c, h, \mathcal{D}_c)$ as the theorized action sequence (where the concept is exhibited, or the where the observational space is formed) of supposed distribution $\mathcal{D}_c$, the supervisor $\text{Supervisor}(\nabla, \mathcal{A}, \{\Theta_S\})$ for the loss class ∇ , algorithm $\mathcal{A}$, and $\{\Theta_S\}$ of special parameters for the supervisor. Additionally, we also include the supposed randomized state generation, $\text{RAND}_i(\theta_{i,j,k})$ of distinct controlling parameters and arbitrary pseudo-random shape.

hypothesis class to extend. By doing this, they extend the outreach of the hypothesis class, however make it unstable under random, iterative process typically used to train such model to target.

This is also noticed and formulated in the concept of approximation-estimation tradeoff. We refer to Mohri et al. (2012); Lafon & Thomas (2024) for some analysis and mentions.

Let $\mathcal{H}$ be a family of functions mapping $\mathcal{X} \rightarrow \{1, -1\}$. This is the particular case of **binary classification**, in which can be straightforwardly extended to different tasks and loss functions. The *excess error* of a hypothesis $h \in \mathcal{H}$, is the difference between its error $R(h)$ and the Bayes error R^* . This can be decomposed to be the following:

$$R(h) - R^* = \left(R(h) - \inf_{h \in \mathcal{H}} R(h) \right) + \left(\inf_{h \in \mathcal{H}} R(h) - R^* \right) \quad (71)$$

The first bracket contains the **estimation error**, and the second bracket contains what is called the **approximation error**. The estimation error depends on the hypothesis h selected. It measures the error of h with respect to the infimum of the error achieved by hypotheses in $\mathcal{H}$, or that of the best-in-class hypothesis h^* when that infimum is reached. The approximation error measures how well the Bayes error can be approximated using $\mathcal{H}$. It is a property of the hypothesis set $\mathcal{H}$, a measure of its richness.

APPENDIX E - OBSERVING GRADIENT DESCENT DECOMPOSITION

While the interpretation of what the bias-variance tradeoff, the usual training-versus-test curve, and statistical learning consideration, another interesting aspect to consider (as well for double descent), is how bias-variance decomposition acts on gradient descent and its components. This claim is perhaps reinforced by the work of Adlam & Pennington (2020) which indicates that different bias-variance decompositions effects differently on the total system mechanism.

The main term of a gradient descent algorithm is the gradient of the loss function $\mathcal{L}(h(x), y)$. For the hypothesis h , the number of instances supplied denoted by k has three main cases: $k = 1$ for stochastic descent, $k \in (1, m)$ for batch gradient descent, and $k = m$ for standard gradient descent, for a given space of m samples. The gradient in such case is calculated using the empirical risk on m size, that is:

$$\mathcal{L}_k(h(x), y) = \hat{R}_{S[k]}(h) = \frac{1}{k} \sum_{i=1}^k \ell(h(x_i), y_i) \quad (72)$$

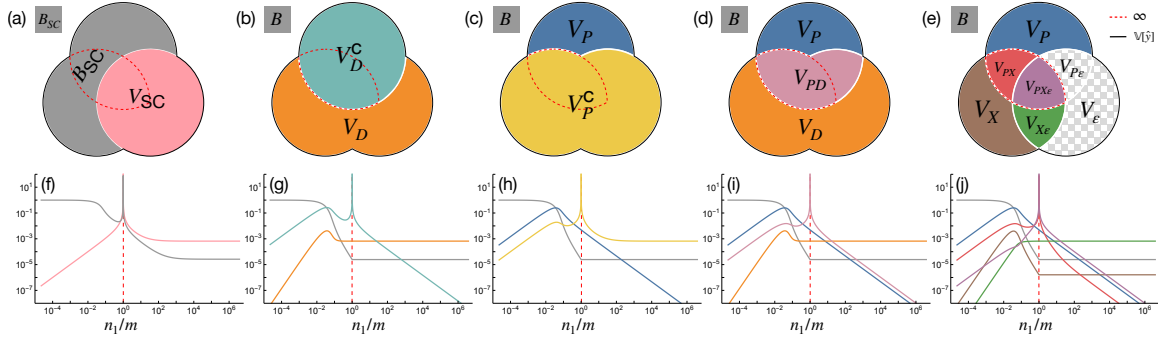


Figure 18: **(a-e) The different bias-variance decompositions.** (f-j) Corresponding theoretical predictions for $\gamma = 0$, $\phi = 1/16$ and $\sigma = \tanh$ with $\text{SNR} = 100$ as the model capacity varies across the interpolation threshold (dashed red). (a,f) The semi-classical decomposition of Hastie et al. (2019); Mei & Montanari (2019a) has a nonmonotonic and divergent bias term, conflicting with standard definitions of the bias. (b,g) The decomposition of Neal et al. (2018) utilizing the law of total variance interprets the diverging term V_D^C as “variance due to optimization”. (c,h) An alternative application of the law of total variance suggests the opposite, *i.e.* the diverging term V_P^C comes from “variance due to sampling”. (d,i) A bivariate symmetric decomposition of the variance resolves this ambiguity and shows that the diverging term is actually V_{PD} , *i.e.* “the variance explained by the parameters and data together beyond what they explain individually.” (e,j) A trivariate symmetric decomposition reveals that the divergence comes from two terms, V_{PX} and $V_{PX\epsilon}$ (outlined in dashed red), and shows that label noise exacerbates but does not cause double descent. Since $V_\epsilon = V_{P\epsilon} = 0$, they are not shown in (j). Taken from Adlam & Pennington (2020)

We use the classical bias-variance tradeoff at first. Since the gradient is a linear operator, we have:

$$\begin{aligned}\nabla_k(\mathcal{L}) &= \nabla_k \left((\mathbb{E}\hat{y}(\mathbf{x}) - \mathbb{E}y(\mathbf{x}))^2 + \mathbb{V}[\hat{y}(\mathbf{x})] + \mathbb{V}[y(\mathbf{x})] \right) \\ &= \nabla_k (\mathbb{E}[\hat{y}(\mathbf{x})] - \mathbb{E}[y(\mathbf{x})])^2 + \nabla_k \mathbb{V}[\hat{y}(\mathbf{x})] + \nabla_k \mathbb{V}[y(\mathbf{x})] \\ &= \nabla_k (\mathbb{E}[\hat{y}(\mathbf{x})] - \mathbb{E}[y(\mathbf{x})])^2 + \nabla_k \mathbb{V}[y(\mathbf{x})]\end{aligned}\quad (73)$$

This effectively decompose the gradient into three parts that influence the overall gradient calculation. Because $\nabla_k \mathbb{V}[y(\mathbf{x})]$ calculates the gradient of irreducible error, which is supposed to be constant of the sample space, it is then equal 0. Suppose a sample $\mathbf{x}_k$ of size km , of partition k . Hence, we have gradient descent based on k th run over the entire dataset, permuted. Then, the loss function calculates in the form

$$\nabla_{k,n}(\mathcal{L}) = \nabla_{k,n} \left[\frac{1}{n} \sum_{i \leq n} \hat{y}(x_{kn+i}) - \frac{1}{n} \sum_{i \leq n} y(x_{kn+i}) \right]^2 + \nabla_{k,n} \left[\frac{1}{n} \sum_{i=1}^n y(x_i)^2 - \left(\frac{1}{n} \sum_{i=1}^n y(x_i) \right)^2 \right] \quad (74)$$

The indices is now changed to (k, n) for $m/k = n$ as the i th partitioned, ordered run, thereby $kn + i$ runs for the index. If we assume a simplified structure θ_t of parameters $y(x_i)$ depends on, we can reduce it further.

$$\nabla_{k,n}(\mathcal{L}) = 2 \left(\frac{1}{n} \sum_{i \leq n} \hat{y}(x_{kn+i}) - \frac{1}{n} \sum_{i \leq n} y(x_{kn+i}) \right) \left(\frac{1}{n} \sum_{i \leq n} \nabla_k \hat{y}(x_{kn+i}) - \frac{1}{n} \sum_{i \leq n} \nabla_k y(x_{kn+i}) \right) \quad (75)$$

$$+ \frac{2}{n} \sum_{i=1}^n y(x_i) \nabla_k y(x_i) - \frac{2}{n^2} \left(\sum_{i=1}^n y(x_i) \right) \left(\sum_{i=1}^n \nabla_k y(x_i) \right) \quad (76)$$

$$= 2 \left((\mathbb{E}_n[\hat{Y}] - \mathbb{E}_n[Y]) (\mathbb{E}_n[\hat{G}_k] - \mathbb{E}_n[G_k]) + \mathbb{E}_n[Y G_k] - \mathbb{E}_n[Y] \mathbb{E}_n[G_k] \right) \quad (77)$$

$$= 2 \left((\mathbb{E}_n[\hat{Y}] - \mathbb{E}_n[Y]) (\mathbb{E}_n[\nabla_k \hat{Y}] - \mathbb{E}_n[\nabla_k Y]) + \text{Cov}_n(Y, \nabla_k Y) \right) \quad (78)$$

$$= 2 \left(\mathbb{E}_n[\hat{Y}] - \mathbb{E}_n[Y] \right) \left(\mathbb{E}_n[\nabla_k \hat{Y}] - \mathbb{E}_n[\nabla_k Y] \right) + 2 \text{Cov}_n(Y, \nabla_k Y) \quad (79)$$

for

$$\mathbb{E}_n[Z] = \frac{1}{n} \sum_{i=1}^n Z_i, \quad \text{Cov}_n(U, V) = \mathbb{E}_n[UV] - \mathbb{E}_n[U] \mathbb{E}_n[V]. \quad (80)$$

Here, we can see two terms multiplication, for the first one being the correlation between the expected error $\mathbb{E}_n[\hat{Y}] - \mathbb{E}_n[Y]$ and the gradient $\mathbb{E}_n[\nabla_k \hat{Y}] - \mathbb{E}_n[\nabla_k Y]$. This measures the correlation between the expected error, and the gradient measure. The second term is exactly the empirical covariance between the target Y and the true gradient component. Since we are taking a fixed concept case, we can simply disregard the truth gradient hence $\mathbb{E}_n[\nabla_k Y]$, for which the term is reduced to $2 \left(\mathbb{E}_n[\hat{Y}] - \mathbb{E}_n[Y] \right) \left(\mathbb{E}_n[\nabla_k \hat{Y}] \right)$. Under this same assumption applied to the covariance term, we obtain

$$\nabla_{k,n}(\mathcal{L}) = \frac{2}{n} \sum_{i=1}^n (\hat{Y}_i - Y_i) \partial_k \hat{Y}_i = 2 \left(\left(\mathbb{E}_n[\hat{Y}] - \mathbb{E}_n[Y] \right) \mathbb{E}_n[\nabla_k \hat{Y}] \right) \quad (81)$$

Looking into some particular algebraic representation, we can write this simply as

$$\nabla_{k,n} \mathcal{L} = 2 \mathbb{E}_n \left[(\hat{Y} - Y) \nabla_k \hat{Y} \right] \quad (82)$$

$$= 2 \left(\underbrace{\left(\mathbb{E}_n[\hat{Y}] - \mathbb{E}_n[Y] \right) \mathbb{E}_n[\nabla_k \hat{Y}]}_{[\mathbb{E}(\hat{Y}, Y)]^{1/2}} + \underbrace{\mathbb{E}_n[\hat{Y} \nabla_k \hat{Y}] - \mathbb{E}_n[\hat{Y}] \mathbb{E}_n[\nabla_k \hat{Y}]}_{\text{Cov}_n(\hat{Y}, \nabla_k \hat{Y})} \right) \quad (83)$$

$$- \underbrace{\mathbb{E}_n[Y \nabla_k \hat{Y}] - \mathbb{E}_n[Y] \mathbb{E}_n[\nabla_k \hat{Y}]}_{\text{Cov}_n(Y, \nabla_k \hat{Y})} \quad (84)$$

The first term can be attributed to self-correct of bias-error gradient correlation, while the two covariance term suggests correlation between model randomness and its gradient, and the last one deal with the truth-gradient correlation. What those gradients mean will have to be tested in some preliminary models. Generally, we should not expect the result to be quite effective. Similar decomposition can be conducted for the other decomposition, as suggested, however, the gradient effect is particularly interesting, and will have to be resolved.

APPENDIX F - DETAILS OF SUPERVISED LEARNING STRUCTURE

In a typical machine learning setting, we obtain assume the following figure 19. This includes the ground, input space (the setting of which the observations are observed), the concept $c \in \mathcal{C}$, the hypothesis $c \in \mathcal{H}$, and a supervisor ∇ that ‘correct’ the behaviours of the hypothesis to match with c . This is called a supervised learning setting, in which the learning part is governed by the supervisor and the observations from the concept.

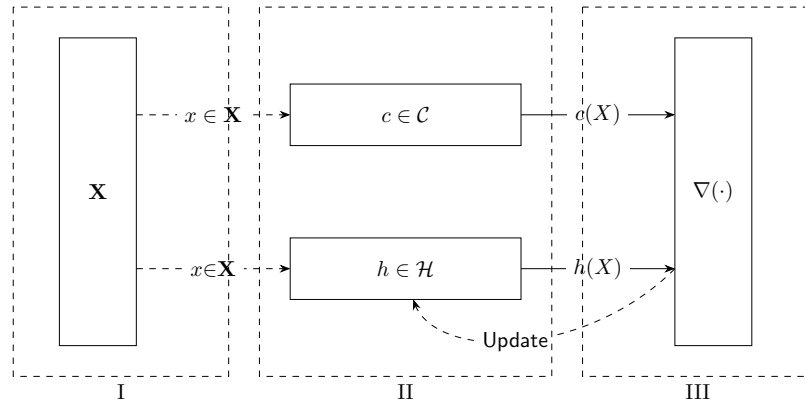


Figure 19: An illustration of the (supervised) statistical process. Phase III contains two parts: First is the evaluation $\nabla(h, c)$ according to the data $\mathcal{D}$, and second is the Update process to re-align c to the actual target.

PHASE 1.

Begin with the construction and initialization of the feature space $\mathcal{X}^\infty$. By the assumption of a probabilistic process, $\mathcal{X}_m$ or simply $\mathcal{X}$ representing the dataset is assumed to be sampled, or appeared from the distribution $\mathcal{D}(p, \cdot)$ for p an arbitrary probabilistic system.

The *ground space*, or in literature generally called **feature space**, is created. This results in the first component of the tuple D , being $\mathcal{X} \subseteq \mathbb{R}^{m \times k}$, where $|\mathcal{X}| = m$, but $\text{size}(x) = k$, for $x \in \mathcal{X}$. We assume there exists the random variable x of a given distribution $\mathcal{D}$, such that the set $\mathcal{X} \sim \mathcal{D}$ of all observation is given by an underlying distribution. A well-known assumption in this case is that $x \in \mathcal{X}$ has no dependent components. That is, for example, there does not exist any function $\psi_x : \{x_i\} \rightarrow \{x_j\}$, for $i \neq j$. (Thought, this only helps in Bayesian priori analysis). Furthermore, by specifying that $\mathcal{X}$ belongs to a specific distribution, we argue of the assumption that $x \in \mathcal{X}$ is *independently and identically distributed* (i.i.d.), that is, for a given random sampling interpretation $S \subset \mathcal{X} \sim \mathcal{D}$, all data is sampled with the events space governed by the distribution $\mathcal{D}$ for every instance, and the existence of x_i to x_j for $i \neq j$ is none.

The **role** of the distribution configuration is important. If $\mathcal{D}$ is uniform, that is, $D \sim \mathcal{U}(a, b)$, then we can disregard the aspect of $\mathcal{X}$ with random variables. For $[a, b]$ to be $(-\infty, +\infty)$, the problem changed to a *regression problem*. If $\mathcal{D}$ is some other distribution function, then the problem of *representative data* and *population size* becomes apparent, which requires statistical analysis to be taken into consideration.

PHASE 2.

For the constructed feature space $\mathcal{X}$, let $EX(c, \mathcal{D})$ be a procedure (or *oracle*) acting on $\mathcal{C}$ and $\mathcal{X}$ to output $\langle x, c(x) \rangle$. The hypothesis then contains a similar procedure $EX_{\mathcal{H}}(h, \mathcal{D})$, such that to output $\langle x, h(x) \rangle$. We call this the *inference phase*.

The feature space $\mathcal{X}$ is provided to h . We assume c is processed of the same process, resulted in $\mathcal{Y}_c$. Then, $h(\mathcal{X})$ outputs the set of hypothesis' target $\mathcal{Y}_h$. Thereby, we are approximating the *process* c itself. We can approach this in two main ways¹⁵: either *deterministic* or *probabilistic*¹⁶. We define such as followed. For the constructed feature space $\mathcal{X}$, let $EX(c, \mathcal{D})$ be a procedure (or *oracle*) acting on $\mathcal{C}$ and $\mathcal{X}$ to output $\langle x, c(x) \rangle$. The hypothesis then contains a similar procedure $EX_{\mathcal{H}}(h, \mathcal{D})$, such that to output $\langle x, h(x) \rangle$. We call this the *inference phase*.

Definition .4 (Deterministic - discriminative modelling). A **deterministic model** h_d assumes its internal space as followed. For any $h \in \mathcal{H}_d$ of deterministic model, h is characterized by the mapping $h_d : \mathcal{X} \rightarrow \mathcal{Y}_h$, such that $h \in C^n$ of n -differential space.

Similarly, if the interpretation of the model is *probabilistic*, we have the following definition of probabilistic-based model.

Definition .5 (Probabilistic - generative modelling). A **probabilistic model** h_p assumes its internal space as followed. For any $h \in \mathcal{H}_p$ of all probabilistic hypothesis, h is characterized by the probability distribution $\Gamma(p_X(X))$, where Γ is the discrete decision process outside the probabilistic interpretation.^a

^aThe discrete decision process Γ is very simple to argue. For example, given a Naive Bayes model with $p(C_k | D[i])$ of the dataset, the probability distribution is regarded as the argument

$$Y^* = \arg \max_{c_k \in C} P_{\theta}(c_k | x) = \arg \max_{c_k \in C} P_{\theta}(x | c_k) \cdot P(c_k)$$

Notice that the process and the learning setting is probabilistic, however, the interpretation of the hypothesis h can be either deterministic or probabilistic, or anything else. This puts the flexibility into the setting of the model, and permits us to consider various representation of the model structure. Furthermore, the hypothesis for the learning process is evaluated relative to the same probabilistic setting, in which the training takes place, and we allow hypotheses that are only approximation of the target concept Sugiyama (2015); Kearns & Vazirani (1994).

The similarity between both of the two general types is that the model is characterized by their **parameters**, both **model-specific parameters** and **hyperparameters**.

PHASE 3.

There exists an evaluator (or *supervisor*) $\nabla(h, c)$ that evaluates specific *loss framework* $\ell\{h(x), c(x)\}$ based on available information, and a *update modifier* $U(\ell, \mathcal{A})$ of certain objective to algorithm $\mathcal{A}$.

¹⁵Note that, in some aspect, this is similar to the usage of mathematical simulation of certain dynamical system, or *any* system that has certain patterns or relations observable.

¹⁶The model itself can be entirely deterministic, while the learning process is not. in fact, one of the very first assumption and axiomatic view of the learning problem is that learning occurs in a *probabilistic setting*.

In this step of the procedural setting, the *supervisor* includes a loss function, ℓ , which takes discrete arguments, and ‘binarily’ evaluate the discrete result between h and c . This loss function is supposed to have a global minimum, or at least a certain region of minimal approximation. We have the following definition.

Definition .6 (Loss function). *A loss function is a non-negative function $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow [0, +\infty)$.*

In general, the following is supposed of the loss function:

Conjecture .1 (Loss function convexity). *For any supervisor ∇ of a learner framework $\mathcal{L}(\mathcal{H}, \mathcal{A})$, the loss function ℓ is assumed to be a p -converging function, or as a convex function. That is, for $\ell : \mathbb{R}^n \rightarrow \mathbb{R}$, then ℓ is convex on its domain, or is locally convex on $[a, b] \subset \mathbb{R}^n$, if, for $x, y \in \mathbb{R}^n$ of such interval, and for $\lambda \in [0, 1]$, it satisfies*

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y) \quad (85)$$

Under a single structure, that is, for example, a class of mapping $\mathbb{R}^n \rightarrow \mathbb{R}$, there can exist many loss functions to be considered. Take for example the setting of regression analysis. Then, ℓ can be either the absolute error, $|h(x) - c(x)|$, or the L^2 norm, $\|h(x) - c(x)\|_2$. Different loss function provides different path and landscape, though they still share somewhat similar interpolation patterns, or either some might be subjected to, under the consideration of a loss landscape, local minima than others. Furthermore, the form and representation of loss functions is not discrete - but rather based of its interpretation and the supposition of the underlying notion required for the loss function to operate in. For example, if the setting is *generative*, the loss function would take the form of a *divergence measure* between two probability distribution. Notice however, that the action potential provided by the model in their operation disappears, or rather, is not considered, when using the loss function.

Using such loss function, for the case of no noises or interference uncertainty, if the goal is to minimize the empirical risk on such dataset, then we call this the method of *empirical risk minimization*, or ERM. This is born out of the consideration that in practice, the oracle $EX(c, \mathcal{D})$, and the actual generalization error $R(h)$ is not obtainable.

APPENDIX F - GEMAN’S DERIVATION IN GEMAN ET AL. (1992)

Proof. We present the original derivation of bias-variance tradeoff and its analysis from Geman et al. (1992). Geman’s paper is, by his own word, concerned of *parametric models*, i.e. hypothesis class without strong assumption of parameters defined, though the definition of bias-variance tradeoff afterward and its justification is made in the sense of parametric model. We partially clarify¹⁷ this with the following definition as customary.

Definition .7 (Parameterization). *A **parametric model** is one that can be parameterized by a finite number of parameters. In general, for a hypothesis class $\mathcal{H}$ of all parametric hypothesis is then expressed as:*

$$\mathcal{H} = \{f(x; \theta) : \theta \in \Theta \subset \mathbb{R}^d\} \quad (86)$$

where Θ is called the parameter space. A **nonparametric model** is one which cannot be parameterized by a fix number of parameters.

Suppose of a training dataset $\mathcal{D}$ of N 2-tuples $(\mathbf{x}_i, y_i)$, that is:

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\} \quad |\mathcal{D}| = N, \mathbf{x} \in \mathbb{R}^d, y \in \mathbb{R} \quad (87)$$

¹⁷Generally, there is no definite definition on what would be considered non-parametric. Though, an example might be drawn from the (vanilla) neural network and Gaussian Process Regression (GPR)

The regression problem is to construct a function $f(\mathbf{x}) \rightarrow y$ based on $\mathcal{D}$, which is denoted $f(\mathbf{x}; \mathcal{D})$ to show this dependency. Then,

$$\begin{aligned}
 \mathbb{E}[(y - f(\mathbf{x}; \mathcal{D}))^2 | \mathbf{x}, \mathcal{D}] &= \mathbb{E}\left[\left((y - \mathbb{E}[y | \mathbf{x}] + (\mathbb{E}[y | \mathbf{x}] - f(\mathbf{x}; \mathcal{D})))^2 | \mathbf{x}, \mathcal{D}\right]\right] \\
 &= \mathbb{E}\left[(y - \mathbb{E}[y | \mathbf{x}])^2 | \mathbf{x}, \mathcal{D}\right] + \left(\mathbb{E}[y | \mathbf{x}] - f(\mathbf{x}; \mathcal{D})\right)^2 \\
 &\quad + 2\mathbb{E}\left[(y - \mathbb{E}[y | \mathbf{x}]) | \mathbf{x}, \mathcal{D}\right] \cdot (\mathbb{E}[y | \mathbf{x}] - f(\mathbf{x}; \mathcal{D}))^2 \\
 &= \mathbb{E}\left[(y - \mathbb{E}[y | \mathbf{x}]) | \mathbf{x}, \mathcal{D}\right] + (\mathbb{E}[y | \mathbf{x}] - f(\mathbf{x}; \mathcal{D}))^2 \\
 &\geq \mathbb{E}\left[(y - \mathbb{E}[y | \mathbf{x}])^2 | \mathbf{x}, \mathcal{D}\right]
 \end{aligned} \tag{88}$$

Hence, we decompose it to:

$$\mathbb{E}[(y - f(\mathbf{x}; \mathcal{D}))^2 | \mathbf{x}, \mathcal{D}] = \mathbb{E}[(y - \mathbb{E}[y | \mathbf{x}])^2 | \mathbf{x}, \mathcal{D}] + (f(\mathbf{x}; \mathcal{D}) - \mathbb{E}[y | \mathbf{x}])^2 \tag{89}$$

Where $\mathbb{E}[(y - \mathbb{E}[y | \mathbf{x}])^2 | \mathbf{x}, \mathcal{D}]$ is regarded to be a relative constant of variance on y given $\mathbf{x}$. Taking the expectation on $\mathcal{D}$, we gain:

$$\begin{aligned}
 \mathbb{E}_{\mathcal{D}}\left\{\mathbb{E}[(y - f(\mathbf{x}; \mathcal{D}))^2 | \mathbf{x}, \mathcal{D}]\right\} &= \mathbb{E}_{\mathcal{D}}\left\{\mathbb{E}[(y - \mathbb{E}[y | \mathbf{x}])^2 | \mathbf{x}, \mathcal{D}] + (f(\mathbf{x}; \mathcal{D}) - \mathbb{E}[y | \mathbf{x}])^2\right\} \\
 &= \mathbb{E}_{\mathcal{D}}\left\{\mathbb{E}[(y - \mathbb{E}[y | \mathbf{x}])^2 | \mathbf{x}, \mathcal{D}]\right\} + \\
 &= \mathbb{E}_{\mathcal{D}}\left\{(f(\mathbf{x}; \mathcal{D}) - \mathbb{E}[y | \mathbf{x}])^2\right\}
 \end{aligned} \tag{90}$$

The second term is of importance, and is further decomposed to:

$$\begin{aligned}
 &\mathbb{E}_{\mathcal{D}}\left\{(f(\mathbf{x}; \mathcal{D}) - \mathbb{E}[y | \mathbf{x}])^2\right\} \\
 &= \mathbb{E}_{\mathcal{D}}\left\{(f(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})] + \mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})] - \mathbb{E}[y | \mathbf{x}])^2\right\} \\
 &= \mathbb{E}_{\mathcal{D}}\left[(f(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})])^2\right] + \mathbb{E}_{\mathcal{D}}\left[(\mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})] - \mathbb{E}[y | \mathbf{x}])^2\right] + \\
 &\quad 2\mathbb{E}_{\mathcal{D}}\left[f(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})]\right] \cdot (\mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})] - \mathbb{E}[y | \mathbf{x}]) \\
 &= \underbrace{\{\mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})] - \mathbb{E}[y | \mathbf{x}]\}}_{\text{bias term}} + \underbrace{\mathbb{E}_{\mathcal{D}}\left\{(f(\mathbf{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(\mathbf{x}; \mathcal{D})])^2\right\}}_{\text{variance term}}
 \end{aligned} \tag{91}$$

which yields the desired bias-variance tradeoff. $\square$